

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

Thiết Kế Và Thi Công Ổ Cắm Điện
Giám Sát Điện Năng Và Điều Khiển Từ Xa
Qua Internet

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **Phạm Năng Đức Duy**

MSSV: 22139009

Huỳnh Thiện Khải

MSSV: 22139032

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

Thiết Kế Và Thi Công Ổ Cắm Điện
Giám Sát Điện Năng Và Điều Khiển Từ Xa
Qua Internet

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Giảng viên hướng dẫn: **Huỳnh Hoàng Hà**

Sinh viên: **Phạm Năng Đức Duy**

MSSV: 22139009

Huỳnh Thiện Khải

MSSV: 22139032

TP. HỒ CHÍ MINH – 06/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: T. thiết kế và Thi công ở' cấm điện giám sát điện năng
và điều khiển từ xa qua Internet

Sinh viên: + Phạm Năng Đức Duy

MSSV: 22139009

+ Huỳnh Thiên Khải

MSSV: 22139032

Hướng dẫn: Huỳnh Hoàng Hà

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

- Mục tiêu cụ thể

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

- Phương pháp hợp lý

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

- phù hợp mục tiêu đề tài

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

- Kết luận phù hợp

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

- Bố cục hợp lý

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

- Đúng chuẩn

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

- Chưa phát hiện trùng lặp

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tất. nghiêm túc

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt - Được phép bảo vệ

Tp. HCM, ngày 24 tháng 06 năm 2024

Người nhận xét

(Ký và ghi rõ họ tên)

Huỳnh Hoàng Hà

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM

KHOA ĐIỆN – ĐIỆN TỬ

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên 1: **Phạm Năng Đức Duy**

MSSV: 22139009

Họ tên sinh viên 2: **Huỳnh Thiện Khải**

MSSV: 22139032

Ngành: **Công nghệ Kỹ thuật Điện tử - Truyền thông**

Giảng viên hướng dẫn: **Huỳnh Hoàng Hà**

1. Tên đề tài: Thiết kế và thi công ổ cắm điện giám sát điện năng và điều khiển từ xa qua Internet.

2. Các số liệu, tài liệu ban đầu: vi điều khiển ESP32, module đo PZEM-004T, relay hai kênh, nền tảng MQTT HiveMQ Cloud và web dashboard.

3. Nội dung thực hiện: nghiên cứu lý thuyết IoT và các giao thức truyền nhận; thiết kế phần cứng và mạch in; lập trình firmware ESP32; kiểm thử, đo đạc và đánh giá kết quả.

4. Ngày giao đề tài:/...../2026

5. Ngày hoàn thành:/...../2026

Giảng viên hướng dẫn

Huỳnh Hoàng Hà

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp này, nhóm thực hiện đã nhận được rất nhiều sự giúp đỡ quý báu. Trước hết, nhóm xin bày tỏ lòng biết ơn chân thành và sâu sắc tới thầy Huỳnh Hoàng Hà, người đã tận tình hướng dẫn, định hướng và góp ý cho nhóm trong suốt quá trình nghiên cứu và hoàn thiện đề tài.

Nhóm cũng xin gửi lời cảm ơn tới quý thầy cô trong Khoa Điện - Điện tử, Trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh, đã trang bị cho nhóm những kiến thức nền tảng và tạo điều kiện thuận lợi để nhóm hoàn thành đồ án.

Cuối cùng, nhóm xin cảm ơn gia đình và bạn bè đã luôn động viên, ủng hộ trong suốt thời gian học tập và thực hiện đề tài. Mặc dù đã cố gắng hoàn thiện, đồ án vẫn khó tránh khỏi thiếu sót, nhóm rất mong nhận được sự góp ý của quý thầy cô để đề tài được hoàn chỉnh hơn.

Xin chân thành cảm ơn!

Nhóm thực hiện đề tài

TÓM TẮT

Trước nhu cầu ngày càng tăng về tiết kiệm điện năng và quản lý thiết bị điện từ xa trong gia đình, đề tài thực hiện thiết kế và thi công một ổ cắm điện hai kênh có khả năng đo lường điện năng và điều khiển, giám sát từ xa qua Internet. Thiết bị được xây dựng quanh vi điều khiển ESP32 tích hợp WiFi, kết hợp module đo PZEM-004T để đo các thông số điện và relay hai kênh để đóng ngắt độc lập hai ổ cắm.

Về phần mềm, firmware trên ESP32 kết nối tới máy chủ MQTT HiveMQ Cloud có mã hóa TLS, công bố dữ liệu đo dưới dạng JSON, nhận và phản hồi lệnh điều khiển, tự bảo vệ an toàn điện và đệm dữ liệu khi mất kết nối. Dữ liệu được hiển thị và điều khiển qua giao diện web dashboard hỗ trợ nhiều người dùng, hẹn giờ, cảnh báo và thống kê điện năng.

Kết quả kiểm thử cho thấy thiết bị đo điện áp với sai số khoảng 0,1% và đo công suất sát giá trị thực sau khi tính đến phần điện tự tiêu thụ, điều khiển từ xa có độ trễ nhỏ và cơ chế tự kết nối lại hoạt động ổn định. Sản phẩm chứng minh tính khả thi của một ổ cắm tự chủ, an toàn và có thể mở rộng.

MỤC LỤC

LỜI CẢM ƠN.....	iii
TÓM TẮT	iv
DANH MỤC TỪ VIẾT TẮT	ix
DANH MỤC BẢNG.....	xi
DANH MỤC HÌNH.....	xiii
CHƯƠNG 1: TỔNG QUAN	1
1.1 LÝ DO CHỌN ĐỀ TÀI.....	1
1.2 MỤC TIÊU	4
1.3 PHƯƠNG PHÁP NGHIÊN CỨU	5
1.4 NỘI DUNG NGHIÊN CỨU.....	6
1.5 GIỚI HẠN	6
1.6 BỐ CỤC.....	7
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	8
2.1 TỔNG QUAN VỀ IOT VÀ GIÁM SÁT ĐIỆN NĂNG	8
2.1.1 Internet vạn vật (IoT)	8
2.1.2 Giám sát điện năng và mô hình ổ cắm điện	9
2.2 CÁC GIAO THỨC VÀ NỀN TẢNG SỬ DỤNG.....	10
2.2.1 Giao thức MQTT và bảo mật TLS	10
2.2.2 Nền tảng HiveMQ Cloud	11
2.2.3 Mô hình lưu trữ dữ liệu qua máy chủ backend	12
2.2.4 Giao thức đồng bộ thời gian NTP	13
2.2.5 Định dạng dữ liệu JSON.....	13
2.2.6 Hệ thống tệp LittleFS và lưu đệm dữ liệu offline	14
2.3 CƠ SỞ LÝ THUYẾT PHẦN MỀM HỆ THỐNG WEB	15

2.3.1	Giao thức WebSocket.....	15
2.3.2	Cơ chế xác thực và phân quyền trên cloud.....	17
2.3.3	Cơ sở dữ liệu quan hệ PostgreSQL và kỹ thuật ORM	19
2.3.4	Kiến trúc ứng dụng web NestJS và Next.js.....	20
2.4	GIỚI THIỆU LINH KIỆN.....	21
2.4.1	Vi điều khiển ESP32	22
2.4.2	Module đo điện năng PZEM-004T	23
2.4.3	Module relay 2 kênh.....	25
2.4.4	Nút nhấn và LED báo trạng thái.....	27
2.4.5	Khởi nguồn 220VAC – 5VDC	29
2.4.6	Các linh kiện phụ trợ khác.....	30
	CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....	31
3.1	GIỚI THIỆU	31
3.2	THIẾT KẾ HỆ THỐNG	32
3.2.1	Sơ đồ khối hệ thống.....	32
3.2.2	Sơ đồ nguyên lý toàn mạch	33
3.2.3	Thiết kế chi tiết từng khối	35
3.3	KIẾN TRÚC PHẦN MỀM VÀ LỰA CHỌN CÔNG NGHỆ	45
3.3.1	Kiến trúc phần mềm tổng quan	45
3.3.2	Tầng hiển thị (Presentation Tier).....	48
3.3.3	Tầng xử lý logic (Logic Tier).....	50
3.3.4	Tầng cơ sở dữ liệu (Database Tier)	52
3.3.5	Tầng truyền thông (Communication Tier)	54
3.4	THIẾT KẾ CƠ SỞ DỮ LIỆU.....	56
3.4.1	Mô hình quan hệ và vai trò các bảng dữ liệu	56
3.4.2	Đặc tả chi tiết các bảng dữ liệu quan hệ.....	57

3.5 THIẾT KẾ GIAO DIỆN API VÀ GIAO THỨC TRUYỀN THÔNG	64
3.5.1 Thiết kế các giao diện lập trình REST API.....	64
3.5.2 Thiết kế các sự kiện truyền thông thời gian thực WebSocket.....	65
3.5.3 Thiết kế cấu trúc các bản tin giao tiếp MQTT	66
3.6 THIẾT KẾ CÁC GIẢI THUẬT	67
3.6.1 Giải thuật kết nối WiFi không chặn	67
3.6.2 Giải thuật lưu trữ ngoại tuyến và đồng bộ (LittleFS Local).....	68
3.6.3 Giải thuật lọc nhiễu và tiết kiệm băng thông	68
3.6.4 Giải thuật tích lũy điện năng và gộp dữ liệu 3 cấp.....	69
3.6.5 Giải thuật điều khiển chịu lỗi Command Timeout và Rollback.....	71
3.6.6 Giải thuật quét Heartbeat phát hiện Node ngoại tuyến	73
3.6.7 Giải thuật cảnh báo sự cố điện an toàn và gửi Email	74
3.6.8 Giải thuật hẹn giờ và lên lịch tự động	75
CHƯƠNG 4: THI CÔNG HỆ THỐNG.....	78
4.1 GIỚI THIỆU	78
4.2 THI CÔNG PHẦN CỨNG.....	78
4.2.1 Thiết kế và chế tạo bo mạch in.....	79
4.2.2 Hàn lắp linh kiện và các module	87
4.2.3 Đóng gói sản phẩm.....	90
4.3 PHẦN MỀM.....	94
4.3.1 Cài đặt và cấu hình môi trường	94
4.3.2 Hiện thực hóa các API backend và cơ sở dữ liệu.....	94
4.3.3 Hiện thực hóa cơ chế truyền thông thời gian thực	96
4.3.4 Hiện thực hóa giao diện Web Dashboard.....	97
4.3.5 Hiện thực hóa cơ chế hẹn giờ và lập lịch tự động.....	98
4.3.6 Hiện thực hóa công cụ dọn dẹp dữ liệu lỗi.....	98

4.4 LƯU ĐỒ GIẢI THUẬT	99
4.4.1 Lưu đồ giải thuật khởi tạo và cấu hình mạng.....	99
4.4.2 Lưu đồ giải thuật vòng lặp chính.....	101
4.4.3 Lưu đồ giải thuật kết nối WiFi không chặn.....	102
4.4.4 Lưu đồ giải thuật lưu trữ đệm ngoại tuyến LittleFS.....	103
4.4.5 Lưu đồ điều khiển Toggle thiết bị và Rollback khi Timeout	105
4.4.6 Lưu đồ giải thuật quét Heartbeat phát hiện node offline.....	106
4.4.7 Lưu đồ giải thuật cảnh báo sự cố điện và gửi email tự động	108
4.4.8 Lưu đồ giải thuật gom cụm và tích lũy điện năng.....	109
4.4.9 Lưu đồ giải thuật hẹn giờ và lên lịch tự động	111
CHƯƠNG 5: NHẬN XÉT, ĐÁNH GIÁ KẾT QUẢ	113
5.1 BỐ TRÍ THÍ NGHIỆM VÀ TẢI KIỂM THỬ	114
5.2 KIỂM THỬ TẢI ĐÈN, QUẠT VÀ TRẠNG THÁI Ổ CẮM	115
5.3 PHƯƠNG PHÁP ĐO VÀ XỬ LÝ SỐ LIỆU	117
5.4 KẾT QUẢ ĐO TẢI 1 - ĐÈN BÀN	119
5.5 KẾT QUẢ ĐO TẢI 2 - ĐÈN CÔNG SUẤT LỚN.....	122
5.6 KIỂM CHỨNG ĐIỆN NĂNG TIÊU THỤ CỦA TẢI.....	125
5.7 SO SÁNH SAI SỐ ĐO LƯỜNG.....	128
5.8 KIỂM THỬ BẬT/TẮT TỪ DASHBOARD	129
5.9 KIỂM THỬ HẸN GIỜ VÀ LỊCH BẬT/TẮT	132
5.10 ĐỘ TRỄ ĐIỀU KHIỂN VÀ KHẢ NĂNG KẾT NỐI LẠI	137
5.11 NHẬN XÉT VÀ ĐÁNH GIÁ TỔNG QUÁT	139
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	140
6.1 KẾT LUẬN	140
6.2 HƯỚNG PHÁT TRIỂN	141
TÀI LIỆU THAM KHẢO	143

DANH MỤC TỪ VIẾT TẮT

AC	Alternating Current (Dòng điện xoay chiều)
AP	Access Point (Điểm truy cập)
API	Application Programming Interface
CT	Current Transformer (Biến dòng)
DC	Direct Current (Dòng điện một chiều)
ĐATN	Đồ án tốt nghiệp
EVN	Tập đoàn Điện lực Việt Nam
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
IoT	Internet of Things (Internet vạn vật)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LED	Light Emitting Diode
LittleFS	Little Fail-Safe Filesystem
MAC	Media Access Control
MQTT	Message Queuing Telemetry Transport
NTP	Network Time Protocol
NVS	Non-Volatile Storage
OAuth	Open Authorization
ORM	Object Relational Mapping
PCB	Printed Circuit Board (Bo mạch in)
PF	Power Factor (Hệ số công suất)

QoS	Quality of Service
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFC	Request for Comments
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver Transmitter
UUID	Universally Unique Identifier
WiFi	Wireless Fidelity

DANH MỤC BẢNG

Bảng 2.1: Thông số kỹ thuật của module ESP32-WROOM-32	23
Bảng 2.2: Thông số kỹ thuật của module PZEM-004T phiên bản 3.0	24
Bảng 2.3: Thông số kỹ thuật của module relay 2 kênh.....	26
Bảng 3.1: Bảng phân bổ chân kết nối của ESP32.....	35
Bảng 3.2: Ước tính dòng tiêu thụ của mạch điều khiển.....	36
Bảng 3.3: So sánh ESP32-WROOM-32 với một số vi điều khiển khác	38
Bảng 3.4: So sánh ESP32-WROOM-32 với các dòng ESP32 khác	39
Bảng 3.5: So sánh PZEM-004T với các phương án đo điện năng khác	40
Bảng 3.6: So sánh lựa chọn công nghệ frontend cho dashboard	49
Bảng 3.7: So sánh lựa chọn công nghệ backend.....	51
Bảng 3.8: So sánh lựa chọn hệ quản trị cơ sở dữ liệu.....	52
Bảng 3.9: So sánh lựa chọn thư viện ORM	53
Bảng 3.10: So sánh lựa chọn công nghệ realtime của dashboard và backend.....	54
Bảng 3.11: So sánh lựa chọn giao thức IoT giữa backend và ESP32.....	55
Bảng 3.12: Vai trò các bảng dữ liệu chính trong hệ thống	56
Bảng 3.13: Đặc tả cấu trúc bảng users.....	57
Bảng 3.14: Đặc tả cấu trúc bảng houses	58
Bảng 3.15: Đặc tả cấu trúc bảng house_members	58
Bảng 3.16: Đặc tả cấu trúc bảng house_join_requests	58
Bảng 3.17: Đặc tả cấu trúc bảng house_permission_requests	59
Bảng 3.18: Đặc tả cấu trúc bảng rooms	59
Bảng 3.19: Đặc tả cấu trúc bảng nodes.....	60
Bảng 3.20: Đặc tả cấu trúc bảng devices	60
Bảng 3.21: Đặc tả cấu trúc bảng logs	61

Bảng 3.22: Đặc tả cấu trúc bảng hourly_energy.....	62
Bảng 3.23: Đặc tả cấu trúc bảng daily_energy	62
Bảng 3.24: Đặc tả cấu trúc bảng monthly_energy.....	62
Bảng 3.25: Đặc tả cấu trúc bảng device_schedules	63
Bảng 3.26: Đặc tả các REST API cốt lõi trong hệ thống.....	64
Bảng 3.27: Đặc tả các sự kiện WebSocket trong hệ thống.....	65
Bảng 3.28: Đặc tả các topic MQTT trong hệ thống.....	66
Bảng 4.1: Danh sách linh kiện và module chính của mạch	78
Bảng 5.1: Thống kê các chức năng chính của hệ thống	113
Bảng 5.2: Kết quả 10 lần đo điện áp, dòng điện và công suất với tải 1.....	122
Bảng 5.3: Kết quả 10 lần đo điện áp, dòng điện và công suất với tải 2.....	124
Bảng 5.4: Kiểm chứng điện năng tiêu thụ tải 2 trong khoảng 1 giờ.....	126
Bảng 5.5: Kiểm tra giá tiền điện ước tính sau khi chạy tải 2.....	127
Bảng 5.6: Giá trị trung bình và sai số của hai tải thử nghiệm	128
Bảng 5.7: Kết quả kiểm thử bật/tắt và đồng bộ trạng thái	132
Bảng 5.8: Kết quả kiểm thử hẹn giờ và lịch bật/tắt	137
Bảng 5.9: Kết quả quan sát độ trễ và khả năng kết nối lại	138

DANH MỤC HÌNH

Hình 1.1: Sản lượng điện thương phẩm của Việt Nam giai đoạn 2020-2023	1
Hình 2.1: Kiến trúc ba lớp của hệ thống IoT áp dụng cho đề tài.....	9
Hình 2.2: Mô hình publish/subscribe của MQTT trong hệ thống	11
Hình 2.3: Luồng dữ liệu giữa thiết bị, broker MQTT và dashboard web.....	13
Hình 2.4: Ví dụ bản tin JSON trạng thái do ESP32 gửi lên broker	14
Hình 2.5: Cơ chế quản lý dữ liệu trên flash bằng LittleFS	15
Hình 2.6: Quy trình HTTP và truyền dữ liệu hai chiều qua WebSocket.....	16
Hình 2.7: Luồng xác thực và truy cập tài nguyên bằng JSON Web Token.....	18
Hình 2.8: Luồng cấp mã ủy quyền trong cơ chế OAuth 2.0	19
Hình 2.9: Cơ chế dữ liệu giữa Entity TypeScript và cơ sở dữ liệu quan hệ	20
Hình 2.10: Kiến trúc backend NestJS và cơ chế Dependency Injection	21
Hình 2.11: Module ESP32-WROOM-32.....	22
Hình 2.12: Module đo điện năng PZEM-004T bản 100A (biến dòng CT)	24
Hình 2.13: Module relay 2 kênh SONGLE SRD-05VDC-SL-C	26
Hình 2.14: Nút nhấn tích hợp đèn LED (loại R16-503BD).....	28
Hình 2.15: Module nguồn HLK-PM01B (220VAC → 5VDC)	29
Hình 2.16: IC ổn áp AMS1117-3.3V	30
Hình 3.1: Sơ đồ khối tổng thể của hệ thống	32
Hình 3.2: Sơ đồ nguyên lý toàn mạch (thiết kế trên Altium Designer).....	34
Hình 3.3: Sơ đồ nguyên lý khối nguồn (HLK-PM01 + AMS1117).....	35
Hình 3.4: Sơ đồ nguyên lý khối xử lý trung tâm ESP32-WROOM-32.....	37
Hình 3.5: Sơ đồ nối dây khối đo lường PZEM-004T (UART, L, N, CT)	40
Hình 3.6: Sơ đồ nguyên lý khối relay 2 kênh và đầu nối ổ cắm.....	41
Hình 3.7: Sơ đồ nguyên lý khối nút nhấn và mạch LED báo	43

Hình 3.8: Sơ đồ nguyên lý khối nạp CH340C và mạch tự reset/nạp.....	44
Hình 3.9: Sơ đồ kiến trúc phần mềm tổng quan của hệ thống.....	47
Hình 3.10: Cấu trúc tầng hiển thị của dashboard web.....	48
Hình 3.11: Cấu trúc tầng xử lý logic trên backend NestJS.....	50
Hình 3.12: Luồng tích lũy và gộp dữ liệu điện năng theo giờ, ngày, tháng.....	70
Hình 3.13: Luồng điều khiển thiết bị có ACK và rollback khi timeout.....	72
Hình 4.1: Sơ đồ bố trí mạch in (PCB layout) thiết kế trên Altium.....	80
Hình 4.2: Mô hình ba chiều của bo mạch dựng trên Altium.....	84
Hình 4.3: Mặt trước bo mạch sau khi gia công, chưa lắp linh kiện.....	85
Hình 4.4: Mặt sau bo mạch sau khi gia công, chưa lắp linh kiện.....	86
Hình 4.5: Bo mạch sau khi hàn lắp linh kiện và đấu nối các module.....	88
Hình 4.6: Hai loại dây sử dụng trong thi công.....	89
Hình 4.7: Vỏ hộp nhựa kích thước 150 mm x 150 mm x 50 mm.....	91
Hình 4.8: Sản phẩm hoàn chỉnh sau khi đóng gói.....	93
Hình 4.9: Lưu đồ giải thuật khởi tạo và cấu hình mạng của thiết bị nhúng.....	100
Hình 4.10: Lưu đồ giải thuật vòng lặp chính của thiết bị nhúng.....	101
Hình 4.11: Lưu đồ giải thuật kết nối WiFi không chặn.....	102
Hình 4.12: Lưu đồ giải thuật lưu trữ đệm ngoại tuyến LittleFS.....	104
Hình 4.13: Lưu đồ điều khiển Toggle thiết bị và Rollback khi Timeout.....	105
Hình 4.14: Lưu đồ giải thuật quét Heartbeat phát hiện node ngoại tuyến.....	107
Hình 4.15: Lưu đồ giải thuật cảnh báo sự cố điện và gửi email tự động.....	108
Hình 4.16: Lưu đồ giải thuật luân chuyển và dọn dẹp dữ liệu điện năng.....	110
Hình 4.17: Lưu đồ giải thuật xử lý tác vụ hẹn giờ và lên lịch tự động.....	111
Hình 5.1: Bố trí đo thực tế với thiết bị, tải và công tơ đối chứng.....	115
Hình 5.2: Trạng thái hai tải đều tắt.....	116
Hình 5.3: Trạng thái chỉ tải đèn hoạt động.....	116

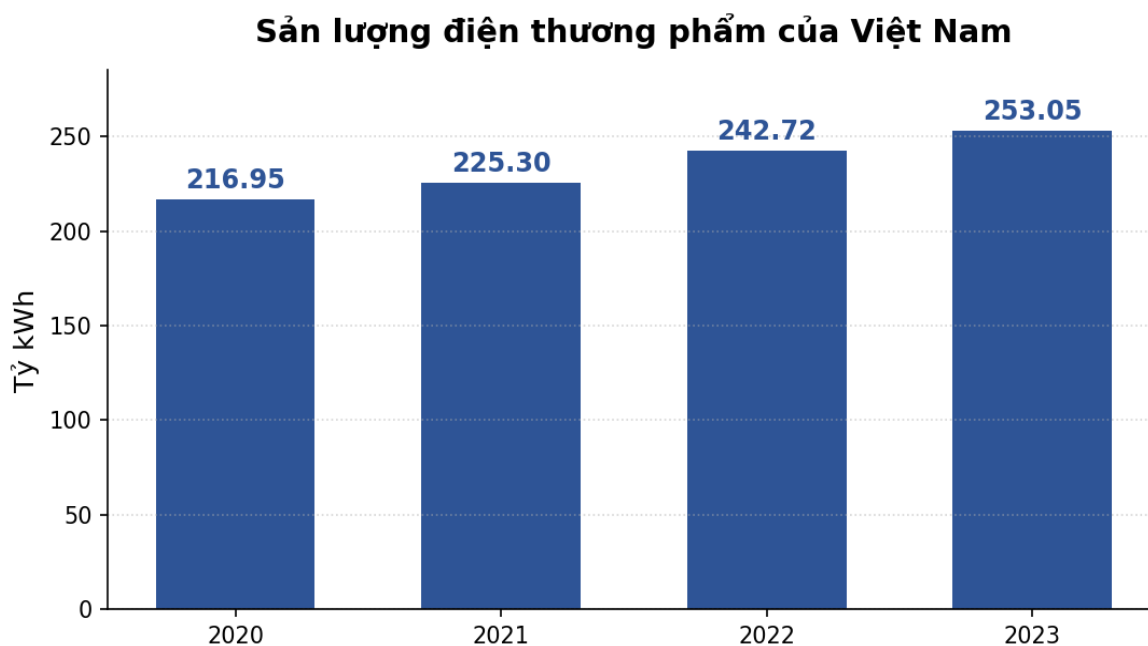
Hình 5.4: Trạng thái đèn và quạt cùng hoạt động.....	117
Hình 5.5: Trạng thái một tải đã tắt trong quá trình kiểm thử.....	117
Hình 5.6: Đo điện áp đầu ra tại ổ cắm của thiết bị bằng đồng hồ vạn năng.....	118
Hình 5.7: Đo điện áp tại ổ điện lưới để đối chiếu với đầu ra thiết bị	118
Hình 5.8: Dashboard hiển thị thông số PZEM khi đo tải 1	119
Hình 5.9: Công tơ đối chứng điện áp khi đo tải 1.....	120
Hình 5.10: Công tơ đối chứng công suất khi đo tải 1	120
Hình 5.11: Ampe kìm đối chứng dòng điện khi đo tải 1	121
Hình 5.12: Giá trị nền của ampe kìm trước khi đo tải	121
Hình 5.13: Dashboard hiển thị thông số PZEM khi đo tải 2	123
Hình 5.14: Công tơ đối chứng công suất khi đo tải 2	123
Hình 5.15: Ampe kìm đối chứng dòng điện khi đo tải 2	124
Hình 5.16: Biểu đồ hiển thị điện năng tiêu thụ của tải 2 sau khoảng 1 giờ.....	125
Hình 5.17: Giá tiền điện ước tính trên web sau khi chạy tải 2 khoảng 1 giờ	127
Hình 5.18: Dashboard hiển thị trạng thái ổ cắm trước khi thao tác.....	130
Hình 5.19: Thao tác đổi trạng thái thiết bị trên dashboard	130
Hình 5.20: Dashboard cập nhật trạng thái sau khi điều khiển	131
Hình 5.21: Thiết bị hoạt động đồng thời với giao diện giám sát	131
Hình 5.22: Thiết lập hẹn giờ bật trên dashboard	132
Hình 5.23: Trạng thái thiết bị sau khi kiểm thử hẹn giờ bật.....	134
Hình 5.24: Thiết lập lịch tắt trên dashboard	135
Hình 5.25: Trạng thái thiết bị sau khi kiểm thử lịch tắt.....	136
Hình 5.26: Theo dõi trạng thái thiết bị khi kiểm thử độ trễ và kết nối lại	138

CHƯƠNG 1

TỔNG QUAN

1.1 LÝ DO CHỌN ĐỀ TÀI

Trong những năm gần đây, nhu cầu sử dụng điện năng tại Việt Nam liên tục tăng cao do quá trình đô thị hóa, sự gia tăng nhanh chóng của các thiết bị điện gia dụng và xu hướng hiện đại hóa đời sống. Theo Tập đoàn Điện lực Việt Nam, trong giai đoạn 2016-2020, sản lượng điện thương phẩm của cả nước tăng trưởng bình quân khoảng 9,67%/năm, cao gấp khoảng 1,6 lần tốc độ tăng trưởng GDP [1]. Đà tăng tiếp tục trong những năm gần đây: sản lượng điện thương phẩm năm 2023 đạt 253,05 tỷ kWh, tăng 4,26% so với năm 2022 (Hình 1.1) [2]. Trong khi đó, giá bán lẻ điện sinh hoạt được tính theo bậc thang lũy tiến nên càng dùng nhiều, đơn giá càng cao, khiến chi phí tiền điện trở thành mối quan tâm thường trực của các hộ gia đình. Vì vậy, người dùng ngày càng cần theo dõi và quản lý lượng điện mà từng thiết bị tiêu thụ để dùng điện tiết kiệm hơn.



Hình 1.1: Sản lượng điện thương phẩm của Việt Nam giai đoạn 2020-2023

Tuy nhiên, các ổ cắm điện thông thường hiện nay chỉ thực hiện chức năng cấp nguồn đơn thuần, không cung cấp bất kỳ thông tin nào về mức tiêu thụ điện năng cũng như không cho phép người dùng điều khiển từ xa. Người sử dụng không thể biết được một thiết bị đang tiêu thụ bao nhiêu công suất, đã tiêu thụ bao nhiêu điện năng hay ước tính được chi phí tương ứng. Bên cạnh đó, nhiều thiết bị vẫn tiêu thụ điện ở chế độ chờ (standby) gây lãng phí mà người dùng khó nhận biết; việc quên tắt thiết bị khi ra khỏi nhà còn tiềm ẩn nguy cơ lãng phí điện và mất an toàn (cháy nổ do quá tải, chập điện).

Trong khi đó, công nghệ Internet vạn vật (IoT) và các vi điều khiển tích hợp WiFi giá rẻ như ESP32 đã mở ra khả năng xây dựng những thiết bị điện thông minh có thể đo lường, giám sát và điều khiển từ xa qua mạng Internet. Các giao thức truyền nhận dữ liệu thời gian thực như MQTT cùng dịch vụ broker đám mây (HiveMQ Cloud) giúp kết nối thiết bị với người dùng ổn định và an toàn, từ đó hiện thực hóa mô hình nhà thông minh (smart home).

Xét về hạ tầng kết nối và điều khiển, các giải pháp nhà thông minh hiện nay có thể quy về ba hướng tiếp cận, mỗi hướng có ưu và nhược điểm riêng. Hướng thứ nhất là hệ thống đi dây theo các chuẩn công nghiệp như KNX hoặc các giải pháp độc quyền của Crestron: độ tin cậy và ổn định rất cao nhờ đường truyền vật lý chuyên dụng, nhưng chi phí đầu tư lớn và phải thi công đồng bộ ngay từ khi xây dựng, nên rất khó lắp bổ sung cho những căn nhà đã hoàn thiện. Hướng thứ hai là các thiết bị không dây dựa trên đám mây đóng của những hãng như Tuya, Xiaomi hay SmartLife: giá rẻ, lắp đặt nhanh, song toàn bộ dữ liệu trạng thái và thói quen sử dụng của gia đình buộc phải gửi về máy chủ của hãng đặt ở nước ngoài, gây lo ngại về quyền riêng tư và sự lệ thuộc vào hệ sinh thái của nhà sản xuất. Hướng thứ ba là các nền tảng mã nguồn mở tự vận hành cục bộ như Home Assistant: làm chủ được dữ liệu và rất linh hoạt, nhưng đòi hỏi người dùng có kiến thức kỹ thuật và phải duy trì một máy chủ vật lý chạy liên tục 24/7 trong nhà, làm tăng chi phí và độ phức tạp vận hành [26].

Từ phân tích trên, đề tài lựa chọn một hướng dung hòa với trọng tâm đặt ở thiết bị đầu cuối. Ổ cắm được xây dựng quanh vi điều khiển ESP32 tích hợp WiFi, có thể

lắp trực tiếp lên hạ tầng điện sẵn có của hộ gia đình mà không cần đi dây BUS hay cải tạo công trình như giải pháp có dây. Thiết bị giao tiếp với người dùng qua giao thức nhẹ MQTT và một dashboard web do chính nhóm phát triển, nhờ đó dữ liệu được lưu trên hạ tầng riêng của nhóm thay vì máy chủ bên thứ ba ở nước ngoài, khắc phục nhược điểm về chủ quyền dữ liệu của hướng đám mây đóng. Để bớt phụ thuộc vào Internet, firmware còn được viết sao cho người dùng vẫn điều khiển được tại chỗ bằng nút nhấn vật lý và thiết bị tự thực hiện các cơ chế bảo vệ an toàn điện ngay trên phần cứng, hướng tới mức tự chủ ở thiết bị mà không buộc người dùng phải vận hành một máy chủ cục bộ phức tạp như hướng tự vận hành.

Tình hình nghiên cứu trong nước. Ở trong nước, đã có nhiều đề tài, đồ án nghiên cứu về điều khiển và giám sát thiết bị điện ứng dụng công nghệ IoT. Nhóm đã tìm hiểu và tham khảo đồ án *"Mô hình ứng dụng IoT trong giám sát và điều khiển các thiết bị điện trong nhà"* của Lê Đình Quang Thắng và Lê Văn Hoan [3], qua đó nắm được cách thức kết nối vi điều khiển với máy chủ để bật/tắt và giám sát trạng thái thiết bị từ xa. Bên cạnh đó, đề tài *"Thiết kế ổ cắm điện thông minh điều khiển qua WiFi sử dụng ADE7753"* của Trường Đại học Bách khoa Hà Nội [4] đã hiện thực một ổ cắm có khả năng đo các thông số điện (điện áp, dòng điện, công suất, điện năng) bằng IC chuyên dụng ADE7753 và điều khiển qua WiFi. Phần lớn các đề tài trong nước mới tập trung vào bật/tắt và giám sát cơ bản; một số có đo điện năng nhưng phần lớn chưa tích hợp đầy đủ các tiện ích như hẹn giờ, cảnh báo quá ngưỡng hay ước tính hóa đơn theo biểu giá bậc thang.

Tình hình nghiên cứu ngoài nước. Ở ngoài nước, nhiều công trình khoa học đã nghiên cứu hệ thống đo lường và giám sát điện năng dựa trên ESP32 và IoT. El-Khozondar và cộng sự [6] đã thiết kế, chế tạo một hệ thống giám sát điện năng chi phí thấp dùng vi điều khiển ESP32 kết hợp cảm biến dòng và áp, truyền dữ liệu lên máy chủ qua Internet và ghi nhận chính xác điện áp, dòng điện, công suất tác dụng cùng điện năng tích lũy. Dzahir và Chia [5] tập trung đánh giá mức tiêu thụ năng lượng của ESP32 khi truyền dữ liệu thời gian thực bằng giao thức MQTT, cho thấy MQTT phù hợp cho các ứng dụng IoT cần truyền dữ liệu liên tục và ổn định. Nalbalwar và cộng

sự [7] xây dựng công tơ điện thông minh dùng vi điều khiển và module đo điện năng PZEM-004T, có khả năng giám sát thời gian thực và phát hiện gian lận điện. Các nghiên cứu này cung cấp cơ sở về kiến trúc phần cứng, phương pháp đo và giao thức truyền dữ liệu để nhóm tham khảo và kế thừa.

Kế thừa kết quả trong và ngoài nước, đồng thời xuất phát từ nhu cầu thực tế về một thiết bị vừa đo điện chính xác vừa điều khiển, giám sát từ xa với đầy đủ tiện ích, nhóm chọn đề tài "**Thiết kế và thi công ổ cắm điện giám sát điện năng và điều khiển từ xa qua Dashboard**".

Khác với phần lớn các đề tài đã khảo sát (vốn tập trung vào điều khiển bật/tắt và giám sát cơ bản), đề tài này hướng tới một thiết bị gộp nhiều chức năng trên cùng một sản phẩm: đo đầy đủ sáu thông số điện (điện áp, dòng điện, công suất, điện năng, tần số, hệ số công suất) bằng module PZEM-004T; truyền dữ liệu và nhận lệnh thời gian thực qua giao thức MQTT bảo mật TLS thông qua broker HiveMQ Cloud, kết nối với một dashboard web do nhóm tự phát triển để lưu trữ lịch sử, cấu hình và phục vụ nhiều người dùng; cùng các tiện ích hẹn giờ, hẹn giờ đếm ngược, cảnh báo quá ngưỡng công suất và ước tính hóa đơn tiền điện theo biểu giá bậc thang của EVN được cung cấp ở lớp web. Riêng ở mức thiết bị, firmware tích hợp cơ chế tự bảo vệ an toàn điện (tự ngắt khi quá dòng, quá áp, sụt áp) và cho phép điều khiển tại chỗ bằng nút nhấn vật lý kể cả khi mất kết nối. Tất cả được triển khai trên hai kênh ổ cắm điều khiển độc lập, cho phép người dùng vừa chủ động quản lý điện năng, vừa điều khiển thiết bị mọi lúc, mọi nơi qua điện thoại hoặc máy tính, qua đó giúp dùng điện an toàn, tiết kiệm và tiện lợi hơn.

1.2 MỤC TIÊU

Mục tiêu tổng quát của đề tài là thiết kế và thi công hoàn chỉnh một ổ cắm điện hai kênh sử dụng vi điều khiển ESP32, có khả năng đo lường điện năng và điều khiển, giám sát từ xa qua Internet. Để hiện thực hóa mục tiêu đó, hệ thống trước hết phải đo và hiển thị chính xác các thông số điện áp, dòng điện, công suất, điện năng tiêu thụ, tần số và hệ số công suất của tải thông qua module PZEM-004T. Trên cơ sở các số

liệu đo được, thiết bị cho phép đóng và ngắt độc lập hai ổ cắm bằng nút nhấn vật lý tại chỗ hoặc qua giao diện dashboard trên Internet, kèm phản hồi xác nhận để bảo đảm lệnh được thực thi đúng. Dữ liệu đo cùng trạng thái thiết bị được truyền lên đám mây qua giao thức MQTT thông qua broker HiveMQ Cloud để dashboard web hiển thị và lưu lịch sử tiêu thụ, nhờ đó người dùng theo dõi được từ xa theo thời gian thực. Bên cạnh các chức năng cơ bản, hệ thống còn cung cấp những tiện ích ở lớp web như hẹn giờ bật tắt, hẹn giờ đếm ngược, cảnh báo khi công suất vượt ngưỡng và ước tính hóa đơn tiền điện theo biểu giá bậc thang của EVN. Cuối cùng, thiết bị phải hoạt động ổn định trong thời gian dài, tự động kết nối lại khi mất WiFi hoặc MQTT và bảo đảm an toàn điện cho người sử dụng.

1.3 PHƯƠNG PHÁP NGHIÊN CỨU

Để đạt được các mục tiêu trên, đề tài kết hợp giữa nghiên cứu lý thuyết, thực nghiệm thiết kế và thi công, cùng đo đạc đánh giá thực tế, được triển khai theo ba giai đoạn nối tiếp nhau. Ở giai đoạn nghiên cứu lý thuyết, nhóm tìm hiểu kiến trúc của một hệ thống IoT, nguyên lý đo các đại lượng điện xoay chiều của module PZEM-004T qua giao tiếp UART, cùng các giao thức và nền tảng truyền nhận dữ liệu như MQTT kèm bảo mật TLS, broker HiveMQ Cloud, đồng bộ thời gian NTP và định dạng JSON, lấy đó làm cơ sở cho việc thiết kế phần cứng và lập trình firmware. Tiếp theo, ở giai đoạn thực nghiệm thiết kế và thi công, sơ đồ nguyên lý của mạch được vẽ trên phần mềm Altium Designer rồi chuyển thành mạch in để gia công, hàn lắp linh kiện và đóng gói sản phẩm, song song đó firmware cho ESP32 được lập trình trên nền Arduino để đọc dữ liệu đo, điều khiển relay, quét nút nhấn, duy trì kết nối WiFi và MQTT cùng các giải thuật tự bảo vệ an toàn điện. Cuối cùng, ở giai đoạn đo đạc và đánh giá, hệ thống được chạy thử nghiệm tích hợp để kiểm tra độ chính xác của phép đo thông qua đối chứng với đồng hồ đo chuẩn, đo độ trễ điều khiển từ xa trên các môi trường mạng khác nhau, đồng thời đánh giá khả năng hoạt động ổn định và cách hệ thống xử lý các tình huống mất kết nối, từ đó rút ra nhận xét và hoàn thiện thiết bị.

1.4 NỘI DUNG NGHIÊN CỨU

Trên cơ sở mục tiêu và phương pháp nêu trên, nội dung nghiên cứu của đề tài được triển khai trên ba mặt: lý thuyết, phần cứng và phần mềm. Về lý thuyết, đề tài nghiên cứu tổng quan về IoT, mô hình giám sát điện năng và các giao thức, nền tảng truyền nhận dữ liệu được sử dụng trong hệ thống. Về phần cứng, đề tài lựa chọn linh kiện và thiết kế các khối chức năng của thiết bị gồm khối nguồn, khối xử lý trung tâm ESP32, khối đo lường PZEM-004T, khối relay hai kênh và khối nút nhấn cùng LED, sau đó thiết kế sơ đồ nguyên lý trên Altium Designer, thi công, hàn lắp và đóng gói sản phẩm. Về phần mềm, đề tài lập trình firmware cho ESP32 và phối hợp với dashboard web để điều khiển, giám sát từ xa. Sau khi hoàn thiện, hệ thống được kiểm thử và đánh giá về độ chính xác đo lường, độ trễ điều khiển và độ ổn định, rồi tổng hợp thành báo cáo.

1.5 GIỚI HẠN

Trong phạm vi của một đồ án tốt nghiệp, đề tài được giới hạn ở một số nội dung sau. Hệ thống hoạt động với điện lưới dân dụng một pha 220VAC, tần số 50Hz, phục vụ các thiết bị gia dụng công suất nhỏ và vừa. Thiết bị gồm hai kênh ổ cắm điều khiển độc lập, mỗi kênh chịu dòng tải tối đa khoảng 10 A theo thông số của relay, trong khi module đo PZEM-004T bản 100A có dư biên đo cho dòng tổng của cả hai kênh, và việc đo điện năng được thực hiện chung cho cả hai ổ chứ chưa đo riêng từng ổ. Hệ thống cần có kết nối WiFi và Internet để thực hiện các chức năng điều khiển, giám sát từ xa. Về công cụ phát triển, sơ đồ nguyên lý và mạch in được thiết kế trên Altium Designer, còn firmware được biên dịch và nạp bằng Arduino IDE, trong khi một số dịch vụ đám mây sẵn có như broker HiveMQ Cloud chỉ được sử dụng làm trung gian truyền tin chứ không thuộc phạm vi tự xây dựng. Sản phẩm được đóng gói trong vỏ hộp nhựa tự chế, tập trung vào tính năng và độ ổn định, chưa tối ưu về kiểu dáng công nghiệp.

1.6 BỐ CỤC

Nội dung của đồ án được trình bày trong sáu chương. Chương 1 trình bày tổng quan về đề tài, gồm lý do chọn đề tài, mục tiêu, phương pháp và nội dung nghiên cứu, giới hạn cùng bố cục của đồ án. Chương 2 giới thiệu cơ sở lý thuyết, bao gồm tổng quan về IoT và giám sát điện năng, các giao thức và nền tảng sử dụng, cơ sở lý thuyết của phần mềm web và các linh kiện chính của hệ thống. Chương 3 trình bày quá trình thiết kế hệ thống, từ yêu cầu thiết kế và sơ đồ khối tổng thể đến thiết kế chi tiết từng khối chức năng cùng sơ đồ nguyên lý toàn mạch. Chương 4 trình bày quá trình thi công phần cứng cùng kiến trúc và lưu đồ giải thuật của phần mềm điều khiển. Chương 5 trình bày kết quả phần cứng, giao diện và kết quả đo lường, kiểm thử, kèm theo phân tích sai số và đánh giá hệ thống. Cuối cùng, Chương 6 tổng kết các kết quả đạt được và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

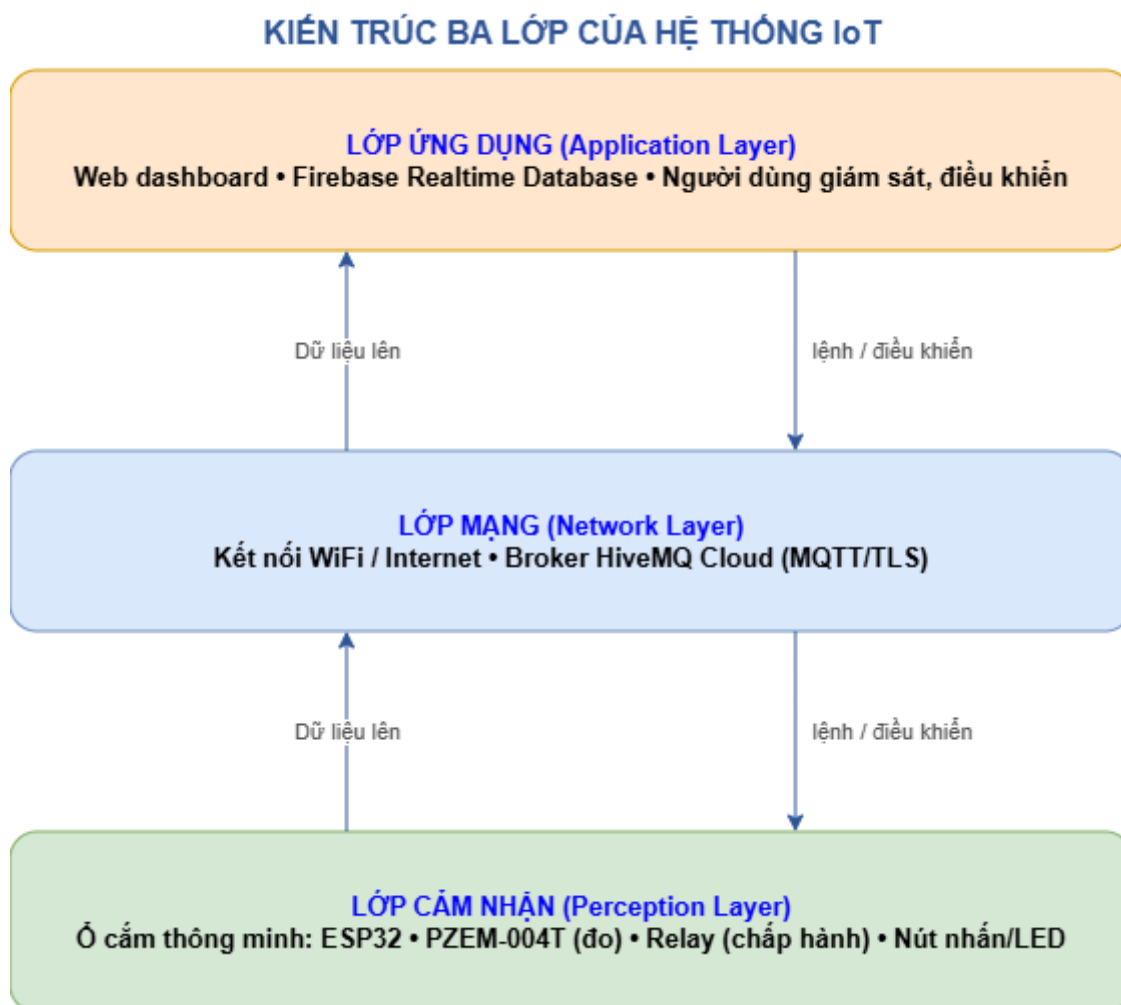
Chương này trình bày các kiến thức nền tảng được vận dụng trong đề tài, theo trình tự từ khái niệm chung (IoT, giám sát điện năng) đến các giao thức và nền tảng truyền nhận dữ liệu, cơ chế lưu đệm dữ liệu trên thiết bị nhúng, cơ sở lý thuyết của phần mềm web, và cuối cùng là các linh kiện cấu thành hệ thống. Với mỗi công nghệ và linh kiện, ngoài việc giới thiệu, phần trình bày còn nêu lý do lựa chọn trong bối cảnh của đề tài để làm rõ tính phù hợp của phương án được chọn.

2.1 TỔNG QUAN VỀ IOT VÀ GIÁM SÁT ĐIỆN NĂNG

2.1.1 Internet vạn vật (IoT)

Internet vạn vật (Internet of Things – IoT) là một hệ thống các thiết bị tính toán, máy móc, đối tượng vật lý được gắn cảm biến, phần mềm và khả năng kết nối mạng, cho phép chúng thu thập và trao đổi dữ liệu với nhau cũng như với người dùng qua Internet mà không cần hoặc cần rất ít sự can thiệp của con người. Nhờ IoT, các thiết bị trong đời sống hằng ngày có thể được giám sát và điều khiển từ xa, tự động hóa nhiều tác vụ và cung cấp dữ liệu phục vụ phân tích, ra quyết định.

Một hệ thống IoT điển hình thường được tổ chức thành ba lớp chính: lớp cảm nhận (perception layer) gồm các cảm biến và cơ cấu chấp hành để thu thập dữ liệu và tác động lên môi trường; lớp mạng (network layer) đảm nhiệm việc truyền dữ liệu qua các công nghệ như WiFi, Ethernet, 4G/5G; và lớp ứng dụng (application layer) gồm máy chủ đám mây, cơ sở dữ liệu và giao diện người dùng để lưu trữ, xử lý và hiển thị thông tin. Dựa vào đó đề tài: ổ cắm điện (ESP32 + PZEM-004T + relay) là thiết bị ở lớp cảm nhận; kết nối WiFi đến broker HiveMQ Cloud là lớp mạng; còn dashboard web giám sát, điều khiển đóng vai trò lớp ứng dụng. Cách phân lớp này cũng định hướng cho việc chia khối phần cứng ở Chương 3. Kiến trúc ba lớp nói trên được minh họa ở Hình 2.1 [24].



Hình 2.1: Kiến trúc ba lớp của hệ thống IoT áp dụng cho đề tài

2.1.2 Giám sát điện năng và mô hình ổ cắm điện

Giám sát điện năng là quá trình đo lường, thu thập và phân tích các thông số điện (điện áp, dòng điện, công suất, điện năng tiêu thụ...) của thiết bị hoặc hệ thống điện để theo dõi mức tiêu thụ, phát hiện bất thường và tối ưu hóa việc sử dụng điện. Khi kết hợp với công nghệ IoT, việc giám sát có thể thực hiện theo thời gian thực và từ xa, giúp người dùng chủ động quản lý chi phí và nâng cao an toàn.

Ổ cắm điện là thiết bị trung gian giữa nguồn điện và tải, vừa đóng/ngắt nguồn cho tải, vừa tích hợp khả năng đo điện năng và kết nối mạng để điều khiển, giám sát từ xa. Trong đề tài, thiết bị được thiết kế hai kênh ổ cắm điều khiển độc lập, dùng chung một module đo PZEM-004T để đo tổng các thông số điện; lựa chọn này được phân

tích kỹ ở Chương 3 trên cơ sở cân đối giữa chi phí, độ phức tạp mạch và yêu cầu sử dụng.

2.2 CÁC GIAO THỨC VÀ NỀN TẢNG SỬ DỤNG

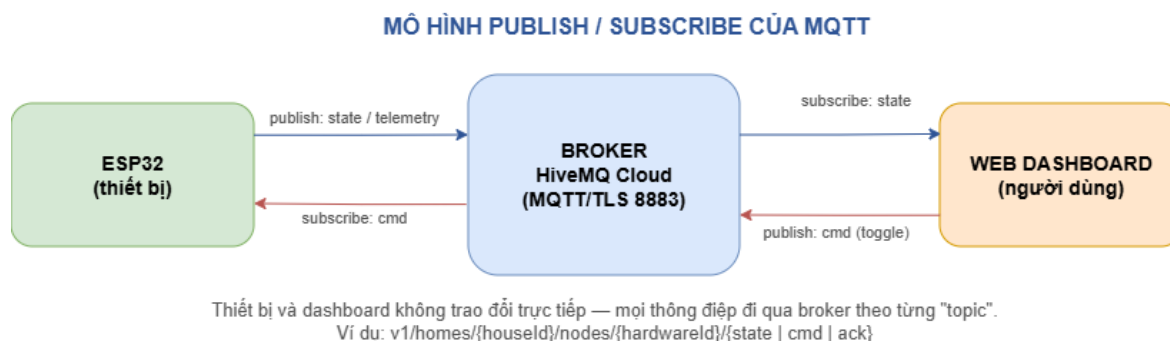
2.2.1 Giao thức MQTT và bảo mật TLS

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông điệp nhẹ, hoạt động theo mô hình xuất bản – đăng ký (publish/subscribe), được thiết kế cho các thiết bị có tài nguyên hạn chế và đường truyền băng thông thấp, không ổn định [10]. Trong mô hình này, các thiết bị không trao đổi trực tiếp mà thông qua một máy chủ trung gian gọi là broker: bên gửi (publisher) đăng dữ liệu lên một chủ đề (topic), bên nhận (subscriber) đăng ký nhận từ topic đó, và broker chuyển tiếp thông điệp đến đúng các bên đăng ký (Hình 2.2). MQTT hỗ trợ ba mức chất lượng dịch vụ QoS 0/1/2 và có thể duy trì kết nối thường trực để nhận lệnh tức thì [25].

Lý do chọn MQTT cho khâu điều khiển. So với cách để dashboard liên tục hỏi trạng thái thiết bị theo kiểu HTTP polling, MQTT có ưu thế rõ rệt cho bài toán điều khiển thời gian thực: kết nối thường trực giúp lệnh bật/tắt từ web tới ổ cắm có độ trễ thấp mà không phải dò hỏi liên tục; mô hình publish/subscribe cho phép một thay đổi trạng thái được đẩy ngay tới mọi client đang theo dõi; gói tin nhỏ nên tiết kiệm tài nguyên cho ESP32 và băng thông. Đây là lý do MQTT được chọn làm kênh điều khiển và truyền số liệu nhanh trong đề tài, thay vì HTTP [27], [28].

Triển khai thực tế trong đề tài. ESP32 kết nối tới broker qua MQTT trên nền TLS (cổng 8883) có xác thực bằng tài khoản/mật khẩu. TLS giúp mã hóa kênh truyền khi dữ liệu đi qua Internet; tuy nhiên firmware hiện tại dùng chế độ setInsecure(), tức là chưa kiểm chứng chứng chỉ máy chủ, nên mức xác thực thực tế vẫn chủ yếu dựa trên tài khoản và mật khẩu MQTT. Thiết bị đăng ký (subscribe) topic lệnh để nhận yêu cầu đóng/ngắt từ dashboard và phản hồi ACK sau khi thực thi; đồng thời publish trạng thái và thông số đo khoảng 2 giây (sau mỗi lần đọc PZEM) và ngay sau mỗi lần đổi relay để dashboard luôn đồng bộ. Đây là phương án phù hợp trong phạm vi đồ án, nhưng khi

triển khai thực tế lâu dài cần bổ sung kiểm chứng chứng chỉ CA để hoàn thiện lớp bảo mật.



Hình 2.2: Mô hình publish/subscribe của MQTT trong hệ thống

Qos, Keep Alive và Last Will. MQTT cung cấp ba mức chất lượng dịch vụ (QoS): QoS 0 gửi nhiều nhất một lần và không xác nhận; QoS 1 bảo đảm nhận ít nhất một lần (có xác nhận, có thể trùng); QoS 2 bảo đảm nhận đúng một lần qua bắt tay bốn bước. Trong đề tài, luồng số liệu đo gửi đều đặn dùng QoS 0 cho nhẹ và nhanh, vì một bản tin lỗi sẽ sớm được thay bằng bản kế tiếp. Bên cạnh đó, cơ chế Keep Alive cho phép thiết bị và broker định kỳ trao đổi gói PINGREQ/PINGRESP để phát hiện mất kết nối; đi kèm là bản tin di chúc (Last Will and Testament) mà thiết bị đăng ký trước, để khi nó rớt mạng đột ngột thì broker tự thay mặt công bố trạng thái offline, giúp dashboard nhận biết ngay. ESP32 trong đề tài đăng ký bản tin di chúc này ở mức QoS 1 để bảo đảm thông điệp offline luôn tới được hệ thống.

2.2.2 Nền tảng HiveMQ Cloud

HiveMQ Cloud là dịch vụ MQTT broker dạng đám mây (managed broker), tương thích đầy đủ chuẩn MQTT và hỗ trợ kết nối bảo mật TLS. Trong đề tài, HiveMQ Cloud được chọn làm broker trung tâm thay vì tự dựng một broker (ví dụ Mosquitto) trên máy chủ riêng. Lý do: broker tự dựng đòi hỏi một máy chủ chạy liên tục, có địa chỉ IP tĩnh hoặc dịch vụ DNS động và phải tự cấu hình bảo mật, gây tốn kém và khó bảo trì trong phạm vi một đồ án; trong khi HiveMQ Cloud cung cấp sẵn hạ tầng ổn định, có TLS, quản lý tài khoản và gói miễn phí đủ dùng, giúp nhóm tập trung

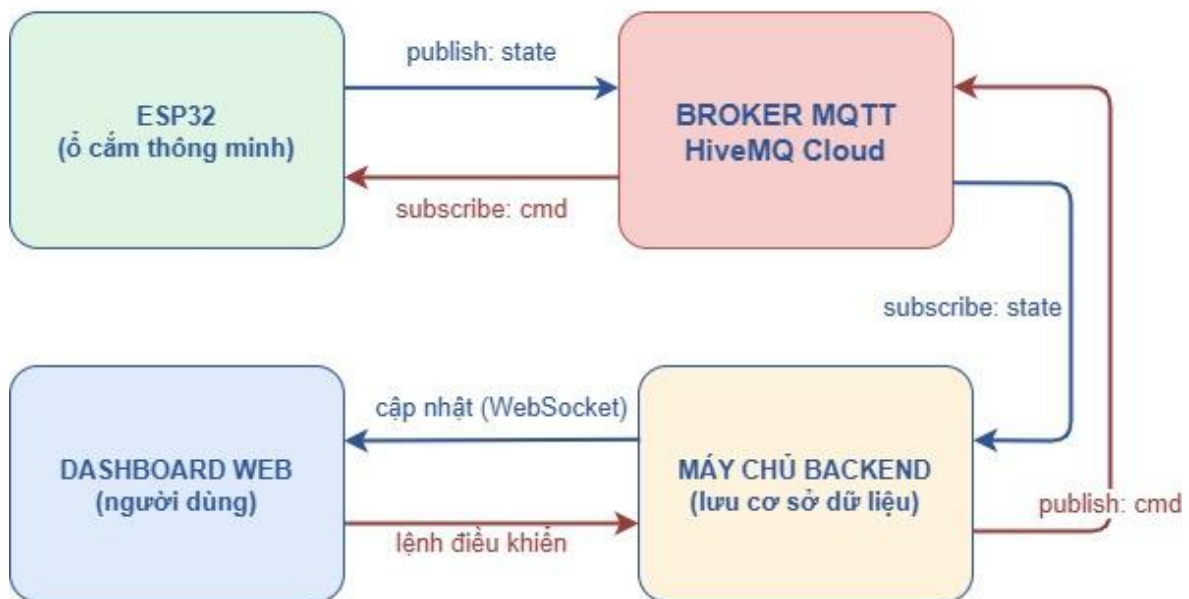
vào thiết bị và ứng dụng. Nhờ broker đặt trên Internet, ESP32 và dashboard có thể kết nối từ bất kỳ mạng nào mà không cần cấu hình mở cổng trên router gia đình [29].

2.2.3 Mô hình lưu trữ dữ liệu qua máy chủ backend

Bản thân broker MQTT chỉ làm nhiệm vụ trung chuyển thông điệp theo thời gian thực mà không lưu trữ dữ liệu lâu dài. Do đó, để phục vụ truy vấn lịch sử tiêu thụ, vẽ biểu đồ và lưu cấu hình của hệ thống, một máy chủ backend đặt trên đám mây được bố trí làm nhiệm vụ đăng ký (subscribe) các topic của thiết bị, tiếp nhận bản tin đo lường và trạng thái rồi ghi vào cơ sở dữ liệu. Toàn bộ luồng dữ liệu giữa thiết bị, broker MQTT và dashboard được minh họa ở Hình 2.3.

Vì sao ESP32 chỉ đẩy dữ liệu lên MQTT? Cách tách vai trò này giúp firmware gọn nhẹ và ổn định: vì điều khiển chỉ cần kết nối tới một đầu mối duy nhất là broker MQTT để vừa gửi số liệu đo, vừa nhận lệnh điều khiển, thay vì phải tự quản lý nhiều kết nối cơ sở dữ liệu hay xử lý truy vấn lịch sử vốn tốn bộ nhớ và dễ gây mất ổn định trên thiết bị nhúng. Mọi tác vụ lưu trữ bền vững, thống kê, phân quyền và phục vụ nhiều người dùng được chuyển lên lớp backend và dashboard web. Do cuốn báo cáo này là bản gộp của toàn hệ thống, phần thiết kế chi tiết backend, cơ sở dữ liệu và dashboard được trình bày ở các chương thiết kế/thi công phía sau; riêng mục này chỉ dừng ở cơ sở lý thuyết và ranh giới trao đổi dữ liệu giữa ESP32, broker và backend.

LUỒNG DỮ LIỆU GIỮA THIẾT BỊ VÀ DASHBOARD



Hình 2.3: Luồng dữ liệu giữa thiết bị, broker MQTT và dashboard web

2.2.4 Giao thức đồng bộ thời gian NTP

NTP (Network Time Protocol) là giao thức đồng bộ thời gian qua mạng, cho phép thiết bị lấy thời gian chuẩn từ các máy chủ thời gian trên Internet. Đối với ổ cắm, thời gian chính xác cần thiết để gắn nhãn thời gian (timestamp) cho mỗi bản tin đo lường và sự kiện trước khi gửi lên broker, nhờ đó backend lưu lịch sử và dựng biểu đồ tiêu thụ theo đúng mốc thời gian thực tế. ESP32 đồng bộ thời gian qua NTP và đặt theo múi giờ Việt Nam (GMT+7); nếu không có NTP, đồng hồ nội bộ của vi điều khiển sẽ sai lệch dần khiến nhãn thời gian của dữ liệu không còn chính xác.

2.2.5 Định dạng dữ liệu JSON

JSON (JavaScript Object Notation) là định dạng trao đổi dữ liệu dạng văn bản, nhẹ, dễ đọc với con người và dễ phân tích với máy. Dữ liệu JSON tổ chức theo cặp khóa – giá trị, phù hợp để đóng gói các thông số đo lường và trạng thái thiết bị trước khi truyền qua MQTT. Trong đề tài, các bản tin của ESP32 được tạo và phân tích bằng thư viện ArduinoJson, giúp dữ liệu trao đổi giữa thiết bị, broker và dashboard có cấu

trúc thống nhất, dễ mở rộng thêm trường mà không phá vỡ định dạng cũ. Hình 2.4 minh họa một bản tin trạng thái thực tế mà ESP32 đóng gói và gửi lên broker.

```
{
  "node_id": "A1B2C3D4E5F6",
  "hardware_id": "A1B2C3D4E5F6",
  "firmware_version": "4.0",
  "timestamp": "2026-06-18T14:30:05Z",
  "voltage": 220.5,
  "current": 0.84,
  "power": 184.2,
  "devices": [
    { "pin": 5, "status": "on", "type": "relay",
      "name": "Relay 1", "power_consumption_ma": 736 },
    { "pin": 18, "status": "off", "type": "relay",
      "name": "Relay 2" }
  ]
}
```

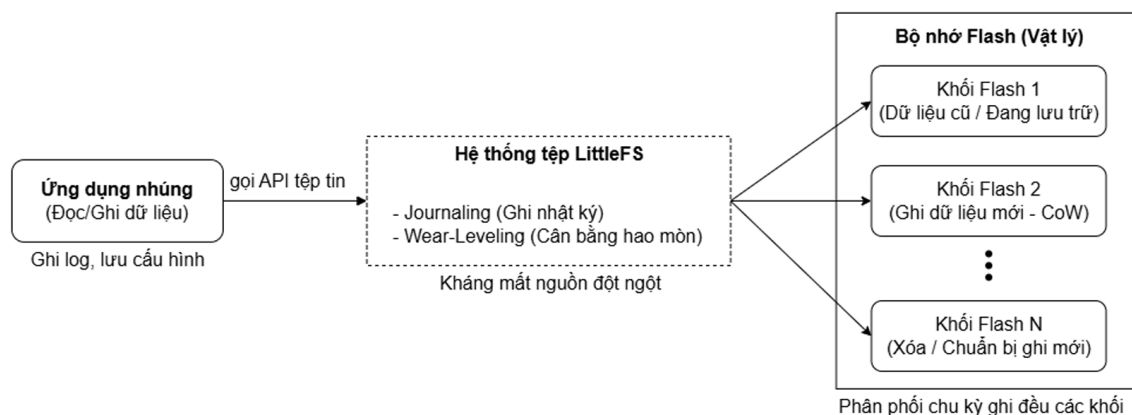
Hình 2.4: Ví dụ bản tin JSON trạng thái do ESP32 gửi lên broker

2.2.6 Hệ thống tệp LittleFS và lưu đệm dữ liệu offline

LittleFS là hệ thống tệp nhỏ gọn, hướng tới vi điều khiển và bộ nhớ flash, được thiết kế với các đặc tính như chống lỗi khi mất nguồn, cân bằng hao mòn khối nhớ và giới hạn mức sử dụng RAM/ROM [23]. Các đặc điểm này phù hợp với ESP32 vì dữ liệu cần ghi cục bộ thường có kích thước nhỏ nhưng phải chịu rủi ro mất nguồn hoặc mất kết nối mạng trong quá trình vận hành.

Trong hệ thống ổ cắm, LittleFS được dùng làm bộ đệm telemetry khi thiết bị mất MQTT. Khi chưa kết nối được broker, firmware mở tệp /offline.csv trên flash và ghi nối tiếp các bản ghi gồm thời gian, điện áp, dòng điện, công suất và điện năng tiêu thụ. Khi WiFi/MQTT phục hồi, thiết bị đọc lần lượt từng dòng trong tệp này, đóng gói lại thành bản tin JSON và publish lên topic trạng thái; sau khi đồng bộ xong, tệp offline được xóa để giải phóng bộ nhớ. Nhờ đó hệ thống không bỏ mất toàn bộ dữ liệu đo trong các khoảng mất mạng ngắn.

Cơ chế ghi dữ liệu lên flash qua LittleFS được minh họa ở Hình 2.5. Cách làm này không thay thế cơ sở dữ liệu trên backend, mà chỉ đóng vai trò lớp đệm tạm thời tại thiết bị để tăng khả năng chịu lỗi của hệ thống.



Hình 2.5: Cơ chế quản lý dữ liệu trên flash bằng LittleFS

2.3 CƠ SỞ LÝ THUYẾT PHẦN MỀM HỆ THỐNG WEB

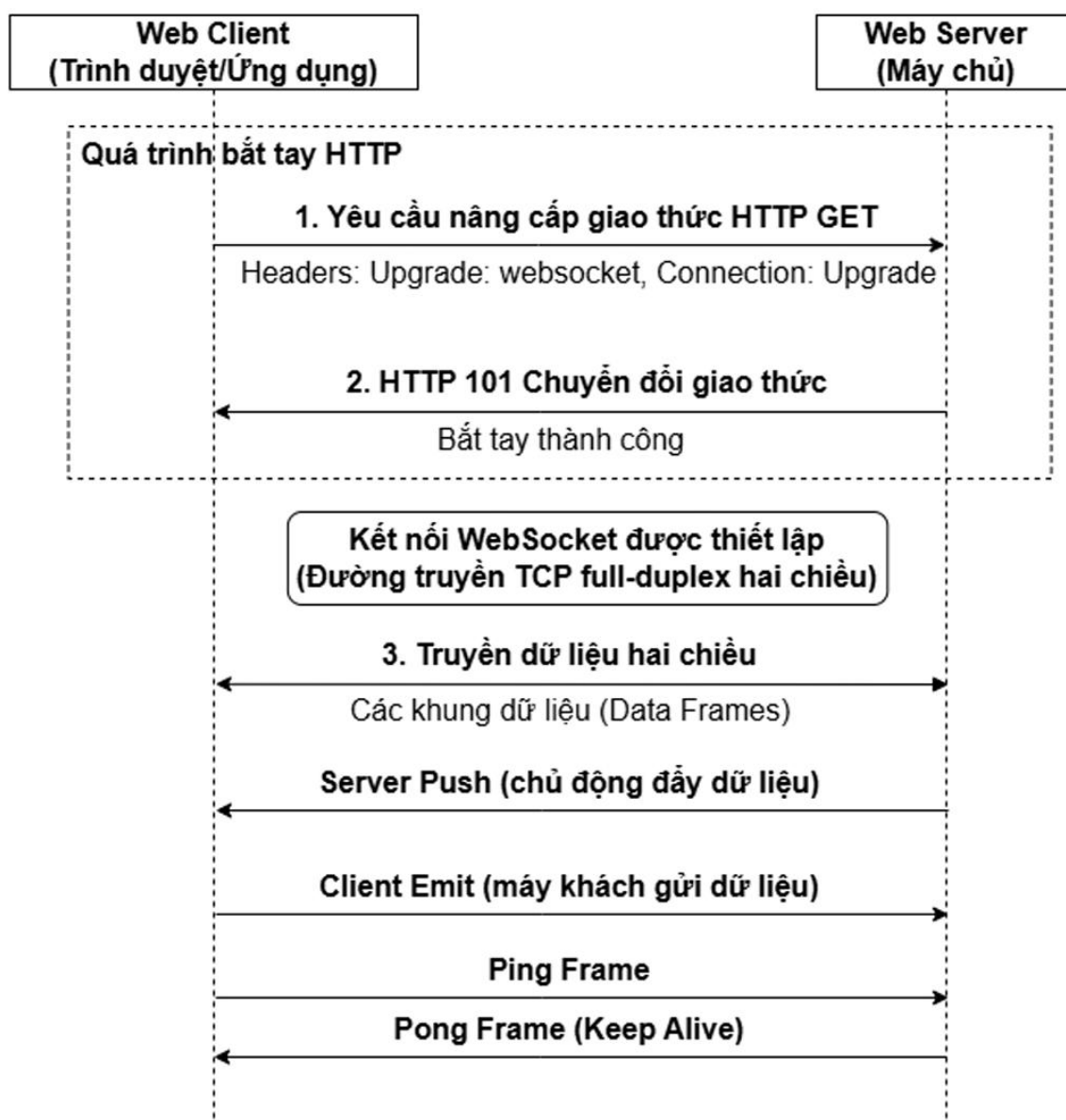
Bên cạnh thiết bị nhúng ESP32, hệ thống còn có phần phần mềm web đảm nhận xác thực người dùng, quản lý nhà/phòng/thiết bị, lưu trữ dữ liệu đo và hiển thị trạng thái vận hành theo thời gian thực. Vì vậy, phần cơ sở lý thuyết của web tập trung vào bốn nhóm nền tảng chính: kênh truyền thời gian thực WebSocket, cơ chế xác thực và phân quyền trên cloud, cơ sở dữ liệu quan hệ kết hợp ORM, cùng kiến trúc triển khai backend NestJS và frontend Next.js.

2.3.1 Giao thức WebSocket

WebSocket là giao thức truyền thông hai chiều toàn phần (full-duplex) hoạt động trên một kết nối TCP duy nhất, được chuẩn hóa trong RFC 6455 [11]. Khác với HTTP truyền thống, trong đó trình duyệt phải gửi yêu cầu rồi mới nhận phản hồi, WebSocket cho phép cả client và server chủ động truyền dữ liệu sau khi kết nối được thiết lập. Quá trình thiết lập bắt đầu bằng một yêu cầu HTTP có tiêu đề Upgrade; nếu máy chủ

chấp nhận, kết nối được chuyển sang trạng thái WebSocket và duy trì liên tục để trao đổi các khung dữ liệu có kích thước nhỏ.

Quy trình bắt tay HTTP và thiết lập kênh truyền hai chiều của WebSocket được thể hiện ở Hình 2.6.



Hình 2.6: Quy trình bắt tay HTTP và truyền dữ liệu hai chiều qua WebSocket

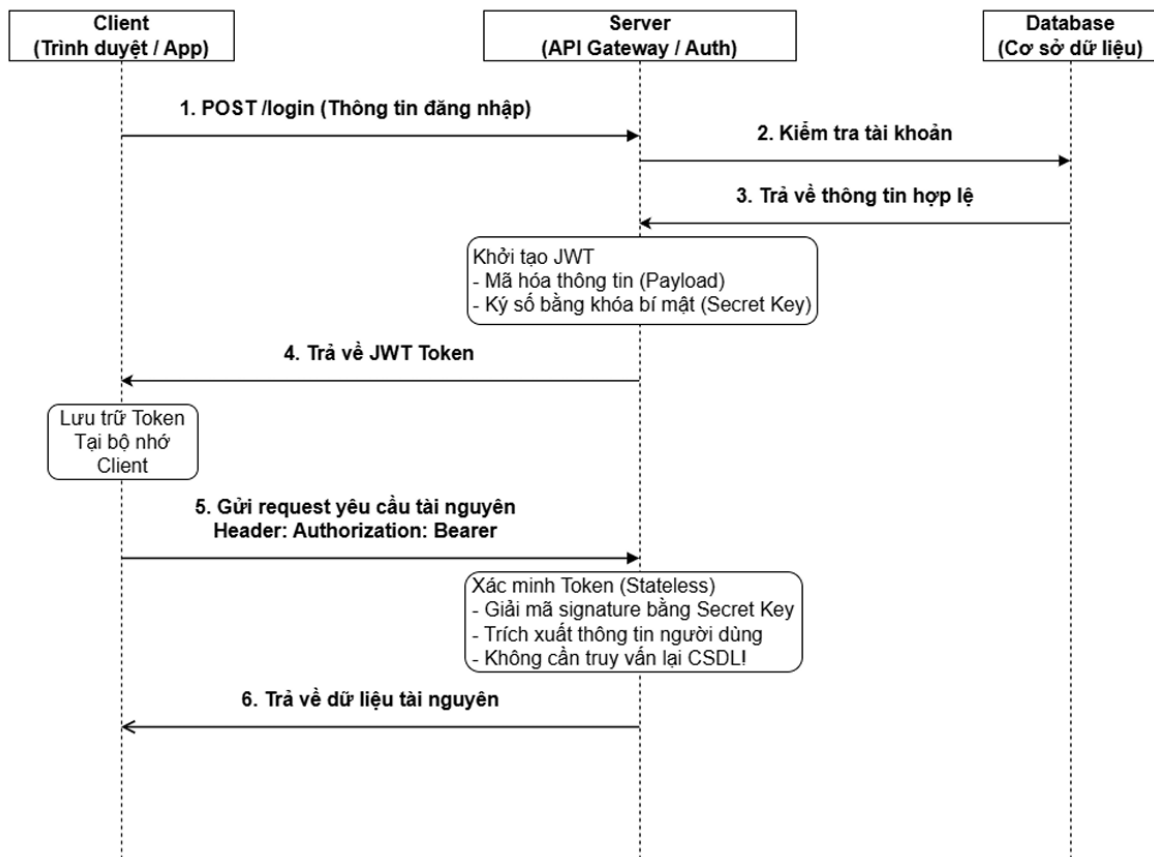
Đối với hệ thống ổ cắm, WebSocket phù hợp với yêu cầu cập nhật nhanh trạng thái relay, điện áp, dòng điện, công suất và cảnh báo bất thường lên dashboard. Khi backend nhận bản tin MQTT từ ESP32, dữ liệu không cần chờ người dùng tải lại trang hoặc gửi yêu cầu thăm dò định kỳ; máy chủ có thể đẩy sự kiện mới trực tiếp đến giao diện web. Trong triển khai thực tế, Socket.IO thường được dùng phía NestJS và Next.js để bổ sung cơ chế tự kết nối lại, phân nhóm client theo nhà hoặc thiết bị, và phát sự kiện chọn lọc đến đúng người dùng đang có quyền truy cập [22].

2.3.2 Cơ chế xác thực và phân quyền trên cloud

Vì hệ thống cho phép nhiều người dùng cùng quản lý nhiều nhà và nhiều thiết bị, cơ chế xác thực và phân quyền là lớp bảo vệ bắt buộc. Xác thực trả lời câu hỏi “người dùng là ai”, còn phân quyền xác định “người dùng được phép thao tác gì” trên từng tài nguyên cụ thể. Nếu thiếu lớp này, một người dùng không hợp lệ có thể xem dữ liệu tiêu thụ điện hoặc gửi lệnh điều khiển relay trái phép.

JSON Web Token (JWT) là cơ chế thường dùng cho xác thực phi trạng thái, được chuẩn hóa trong RFC 7519 [16]. Sau khi đăng nhập thành công, máy chủ cấp cho người dùng một token có ba phần: Header mô tả thuật toán ký, Payload chứa các thông tin định danh như mã người dùng và vai trò, còn Signature dùng để kiểm tra token có bị sửa đổi hay không. Ở các yêu cầu tiếp theo, frontend gửi token kèm theo request; backend chỉ cần kiểm tra chữ ký và thời hạn token để xác minh người dùng, không phải lưu session trong bộ nhớ máy chủ.

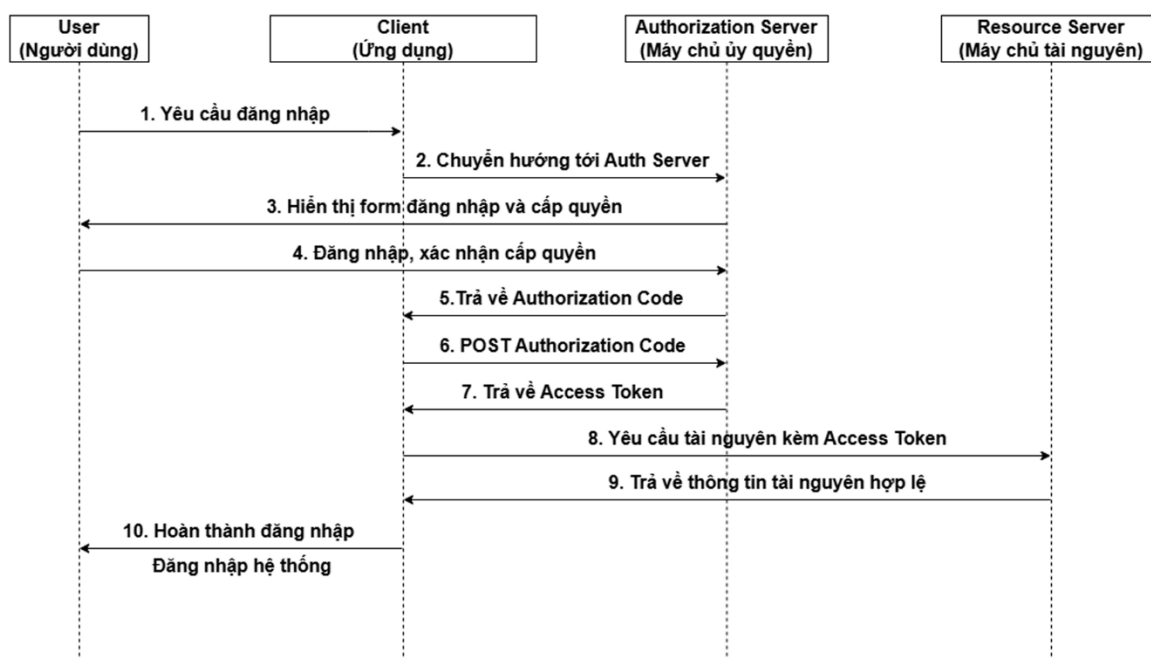
Luồng xác thực phi trạng thái bằng JWT được trình bày ở Hình 2.7.



Hình 2.7: Luồng xác thực và truy cập tài nguyên bằng JSON Web Token

OAuth 2.0 hỗ trợ đăng nhập thông qua nhà cung cấp bên thứ ba như Google và được đặc tả trong RFC 6749 [17]. Người dùng được chuyển đến Google để xác nhận danh tính, sau đó backend nhận mã ủy quyền và trao đổi lấy thông tin tài khoản cần thiết. Cách này giúp hệ thống không phải tự lưu mật khẩu người dùng, giảm rủi ro bảo mật. Trên lớp phân quyền, mô hình RBAC (Role-Based Access Control) gán quyền theo vai trò như Owner, Member hoặc Guest trong từng nhà. Owner có thể quản lý thành viên và cấu hình thiết bị; Member có thể điều khiển hoặc theo dõi theo quyền được cấp; Guest chỉ nên có quyền xem hoặc thao tác hạn chế. Nhờ đó dữ liệu giữa các hộ gia đình được cô lập rõ ràng.

Luồng cấp mã ủy quyền khi đăng nhập bằng Google OAuth 2.0 được thể hiện ở Hình 2.8.



Hình 2.8: Luồng cấp mã ủy quyền trong cơ chế OAuth 2.0

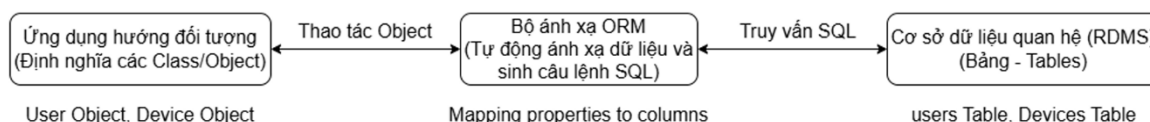
2.3.3 Cơ sở dữ liệu quan hệ PostgreSQL và kỹ thuật ORM

Cơ sở dữ liệu quan hệ lưu trữ dữ liệu dưới dạng các bảng có quan hệ với nhau thông qua khóa chính và khóa ngoại. Với hệ thống ổ cắm, mô hình này phù hợp vì dữ liệu có cấu trúc rõ ràng: người dùng thuộc nhà, nhà có phòng, phòng chứa node thiết bị, node có các kênh relay và các bản ghi đo điện năng theo thời gian. PostgreSQL được lựa chọn trong nhiều hệ thống web nhờ khả năng lưu trữ ổn định, hỗ trợ truy vấn mạnh và tuân thủ các tính chất giao dịch ACID: nguyên tử, nhất quán, cô lập và bền vững [18].

Tính ACID đặc biệt quan trọng khi hệ thống ghi nhật ký đo lường, cập nhật trạng thái thiết bị hoặc thay đổi quyền thành viên. Ví dụ, một thao tác thêm thiết bị vào phòng phải được ghi đầy đủ các quan hệ cần thiết; nếu xảy ra lỗi giữa chừng, cơ sở dữ liệu cần khôi phục về trạng thái trước đó thay vì để lại dữ liệu dang dở. Điều này giúp dữ liệu vận hành ổ cắm và lịch sử tiêu thụ điện có độ tin cậy cao hơn.

Object-Relational Mapping (ORM) là kỹ thuật ánh xạ giữa các lớp đối tượng trong mã nguồn backend và các bảng trong cơ sở dữ liệu quan hệ. Với TypeORM, các bảng như User, House, Room, Node hoặc Device có thể được mô tả bằng Entity trong TypeScript; quan hệ một-nhiều hoặc nhiều-nhiều được định nghĩa trực tiếp trong mã nguồn. TypeORM cũng hỗ trợ migration để thay đổi cấu trúc bảng có kiểm soát khi hệ thống phát triển, giảm rủi ro sửa thủ công cơ sở dữ liệu và giúp mã nguồn backend dễ bảo trì hơn [19].

Cơ chế ánh xạ giữa mô hình đối tượng trong mã nguồn và bảng dữ liệu quan hệ được minh họa ở Hình 2.9.

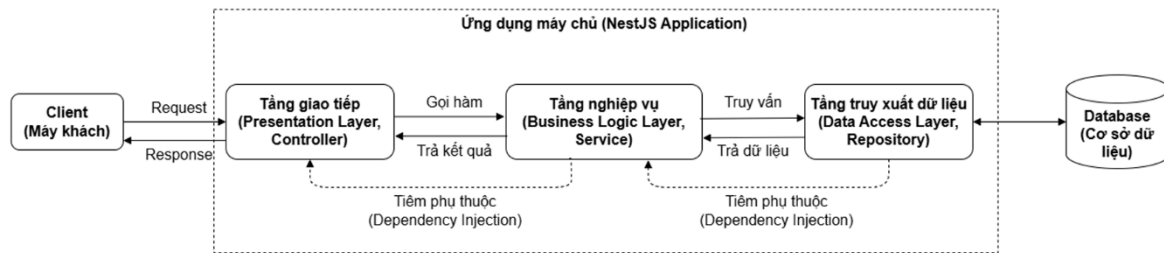


Hình 2.9: Cơ chế ánh xạ dữ liệu giữa Entity TypeScript và cơ sở dữ liệu quan hệ

2.3.4 Kiến trúc ứng dụng web NestJS và Next.js

Backend sử dụng NestJS trên nền Node.js. NestJS là framework phía máy chủ hỗ trợ TypeScript và cung cấp kiến trúc ứng dụng có tính mô-đun [20]. Trong hệ thống, mã nguồn thường được tổ chức thành Module, Controller, Service và lớp truy xuất dữ liệu. Controller tiếp nhận yêu cầu HTTP hoặc sự kiện WebSocket, Service xử lý nghiệp vụ, còn Repository hoặc ORM đảm nhận giao tiếp với cơ sở dữ liệu. Cơ chế Dependency Injection giúp các thành phần phụ thuộc lẫn nhau thông qua interface hoặc service được khai báo, làm cho mã nguồn dễ kiểm thử và dễ mở rộng khi bổ sung các chức năng như quản lý nhà, quản lý phòng, xử lý MQTT, ghi log hoặc gửi cảnh báo email.

Cách tổ chức phân tầng và cơ chế tiêm phụ thuộc của backend NestJS được minh họa ở Hình 2.10.



Hình 2.10: Kiến trúc phân tầng backend NestJS và cơ chế Dependency Injection

Trong kiến trúc tổng thể, backend NestJS đóng vai trò trung gian giữa thiết bị ESP32, broker MQTT, cơ sở dữ liệu PostgreSQL và dashboard. ESP32 publish dữ liệu đo lên HiveMQ; backend subscribe các topic trạng thái, xử lý nghiệp vụ, ghi dữ liệu cần thiết vào PostgreSQL, sau đó phát sự kiện qua WebSocket để cập nhật giao diện. Khi người dùng bấm điều khiển trên dashboard, backend kiểm tra quyền truy cập rồi publish lệnh MQTT xuống topic điều khiển tương ứng. Cách tổ chức này giúp thiết bị nhúng không phải giao tiếp trực tiếp với cơ sở dữ liệu hoặc trình duyệt, đồng thời giữ logic bảo mật ở phía máy chủ.

Frontend sử dụng Next.js kết hợp React để xây dựng dashboard. Next.js hỗ trợ nhiều cơ chế kết xuất như kết xuất phía máy chủ, kết xuất phía client và các thành phần server/client trong cùng ứng dụng web [21]. Với đề tài, các phần cần tải nhanh có thể được dựng trước từ máy chủ, trong khi các thành phần động như biểu đồ, trạng thái relay hoặc thông báo thời gian thực được cập nhật ở trình duyệt. Sau khi trình duyệt nhận HTML ban đầu, quá trình hydration gắn logic JavaScript vào giao diện để người dùng có thể tương tác như một ứng dụng web hiện đại. Sự kết hợp giữa HTTP REST cho các thao tác cấu hình/ lịch sử và WebSocket cho dữ liệu tức thời giúp giao diện vừa có cấu trúc rõ ràng, vừa phản hồi nhanh với thay đổi từ thiết bị thực tế.

2.4 GIỚI THIỆU LINH KIỆN

Phần này giới thiệu các linh kiện chính theo cấu trúc: giới thiệu, thông số kỹ thuật và ứng dụng trong đề tài. Cách chọn các chi tiết và các thông số (dòng tải, công

suất nguồn, dòng kích relay...) được trình bày sâu hơn ở Chương 3; tại đây nêu ngắn gọn lý do mỗi linh kiện phù hợp để giữ mạch trình bày liền mạch.

2.4.1 Vi điều khiển ESP32

2.4.1.1. Giới thiệu

ESP32 là dòng vi điều khiển hệ thống trên chip (SoC) do Espressif Systems phát triển, tích hợp sẵn WiFi và Bluetooth, được dùng rộng rãi trong IoT nhờ hiệu năng cao, nhiều ngoại vi và giá thành thấp [8]. Module sử dụng trong đề tài là ESP32-WROOM-32 với vi xử lý lõi kép Xtensa LX6, tích hợp sẵn WiFi và đủ chân GPIO cho các ngoại vi của ổ cắm hai kênh (hai relay, hai nút nhấn, giao tiếp UART với PZEM-004T). Hình 2.11 là ảnh module ESP32-WROOM-32 sử dụng trong đề tài. Phần so sánh và biện luận lựa chọn ESP32 so với các vi điều khiển khác được trình bày ở Chương 3.



Hình 2.11: Module ESP32-WROOM-32 (nguồn: nhà sản xuất)

2.4.1.1. Thông số kỹ thuật

Các thông số kỹ thuật chính của ESP32-WROOM-32 được liệt kê ở Bảng 2.1.

Bảng 2.1: Thông số kỹ thuật của module ESP32-WROOM-32

Thông số	Giá trị
Vi xử lý	Xtensa LX6 32-bit, lõi kép, tối đa 240 MHz
Bộ nhớ	520 KB SRAM, 448 KB ROM, 4 MB Flash (SPI)
Kết nối không dây	WiFi 802.11 b/g/n; Bluetooth v4.2 BR/EDR và BLE
Số chân GPIO	34 chân GPIO lập trình được
ADC / DAC	ADC 12-bit (18 kênh); 2 × DAC 8-bit
Ngoại vi giao tiếp	UART, SPI, I2C, I2S, PWM, cảm ứng điện dung
Điện áp hoạt động	3.3 V

2.4.1.1. Ứng dụng

Trong đề tài, ESP32 là khối xử lý trung tâm: đọc dữ liệu từ PZEM-004T qua UART, điều khiển hai relay, quét hai nút nhấn và điều khiển hai LED, duy trì kết nối WiFi và MQTT, phản hồi lệnh điều khiển từ dashboard và thực thi giải thuật tự bảo vệ an toàn điện ngay trên thiết bị. Sơ đồ phân bổ chân GPIO cụ thể được trình bày ở Chương 3.

2.4.2 Module đo điện năng PZEM-004T

2.4.2.1. Giới thiệu

PZEM-004T là module đo các thông số điện xoay chiều một pha: điện áp, dòng điện, công suất tác dụng, điện năng tiêu thụ, tần số và hệ số công suất [9]. Module giao

tiếp với vi điều khiển qua UART theo giao thức Modbus-RTU và được cách ly an toàn với phần đo điện áp lưới. Phiên bản sử dụng trong đề tài là PZEM-004T-100A dùng biến dòng (CT) dạng vòng kẹp quanh dây pha, đo gián tiếp dòng điện nên không phải cắt mạch chính và an toàn khi lắp đặt. Hình 2.12 là ảnh module PZEM-004T bản 100A dùng trong đề tài. Phần so sánh và biện luận lựa chọn PZEM-004T so với các phương án đo khác, cùng lý do chọn bản 100A, được trình bày ở Chương 3.



Hình 2.12: Module đo điện năng PZEM-004T bản 100A (biến dòng CT)

2.4.2.2. Thông số kỹ thuật

Bảng 2.2 trình bày dải đo, độ phân giải và sai số của từng đại lượng mà PZEM-004T đo được.

Bảng 2.2: Thông số kỹ thuật của module PZEM-004T phiên bản 3.0 (100A, dùng biến dòng CT)

Đại lượng đo	Dải đo / Độ phân giải / Sai số
Điện áp	80–260 VAC / 0,1 V / 0,5%
Dòng điện	0–100 A qua biến dòng CT (khởi đo 0,02 A) / 0,01 A / 0,5%
Công suất tác dụng	0–23 kW / 0,1 W / 0,5%
Điện năng	0–9999,99 kWh / 1 Wh / 0,5%
Tần số	45–65 Hz / 0,1 Hz / 0,5%
Hệ số công suất	0,00–1,00 / 0,01 / 1%
Giao tiếp	UART → Modbus-RTU, 9600 baud, 8 bit dữ liệu, 1 bit dừng, không chặn lẻ

2.4.2.2. Ứng dụng

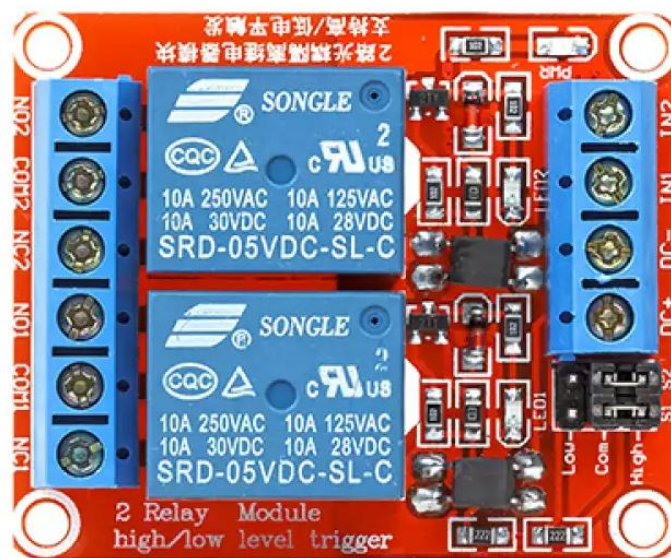
PZEM-004T là khối đo lường của hệ thống. Về khả năng phần cứng, module hỗ trợ đo điện áp, dòng điện, công suất tác dụng, điện năng tiêu thụ, tần số và hệ số công suất. Trong firmware cuối, ESP32 đọc định kỳ điện áp, dòng điện, công suất và điện năng tiêu thụ (energy_kwh) rồi đóng gói gửi lên MQTT; dashboard hiện ưu tiên hiển thị voltage/current/power cho giám sát tức thời, còn energy_kwh được dùng làm dữ liệu tích lũy để backend có thể lưu trữ hoặc xử lý tiếp. Các đại lượng tần số và hệ số công suất vẫn là khả năng của PZEM-004T nhưng chưa được đọc/publish trong code cuối, nên không trình bày như chức năng đã hoàn thiện.

2.4.3 Module relay 2 kênh

2.4.3.1. Giới thiệu

Relay là linh kiện đóng/ngắt mạch bằng tín hiệu điều khiển, cho phép vi điều khiển mức điện áp thấp (3,3 V) điều khiển tải 220 VAC. Module relay 2 kênh dùng trong đề tài [13] cho phép đóng/ngắt độc lập hai ổ cắm và có opto-coupler cách ly giữa

mạch điều khiển với mạch công suất để bảo vệ ESP32. So với relay bán dẫn (SSR), relay cơ điện có giá thành thấp hơn, sụt áp khi dẫn không đáng kể và cách ly hoàn toàn về điện; đổi lại có tiếng kêu cơ khí và tuổi thọ đóng cắt hữu hạn, chấp nhận được với tần suất đóng/ngắt của một ổ cắm gia dụng. Hình 2.13 là ảnh module relay 2 kênh sử dụng trong đề tài.



Hình 2.13: Module relay 2 kênh SONGLE SRD-05VDC-SL-C

2.4.3.2. Thông số kỹ thuật

Các thông số chính của module relay 2 kênh được tóm tắt ở Bảng 2.3.

Bảng 2.3: Thông số kỹ thuật của module relay 2 kênh

Thông số	Giá trị
Điện áp điều khiển	5 VDC
Số kênh	2 kênh độc lập
Cách ly	Opto-coupler giữa mạch điều khiển và công suất

Mức kích	Kích mức cao (active HIGH) theo cấu hình RELAY_ACTIVE_HIGH=1
Dòng tiếp điểm tối đa	10 A / 250 VAC; 10 A / 30 VDC
Model	SONGLE SRD-05VDC-SL-C (2 kênh)

2.4.3.3. Ứng dụng

Hai kênh relay được ESP32 điều khiển qua hai chân GPIO (GPIO5 và GPIO18) để đóng/ngắt nguồn cho hai ổ cắm độc lập, đáp ứng cả lệnh từ nút nhấn vật lý lẫn lệnh điều khiển từ xa qua dashboard.

2.4.4 Nút nhấn và LED báo trạng thái

2.4.4.1. Giới thiệu

Nút nhấn (push button) là cơ cấu đầu vào để người dùng điều khiển trực tiếp trên thiết bị; LED là linh kiện phát quang dùng báo trạng thái. Đề tài sử dụng nút nhấn loại R16-503BD có tích hợp sẵn đèn LED báo ngay trong thân nút: phần tiếp điểm nút nhấn nối với GPIO26 và GPIO27 để ESP32 đọc trạng thái bấm, còn phần LED không lấy dòng trực tiếp từ GPIO32/GPIO33. Thay vào đó, tín hiệu điều khiển được đưa qua điện trở vào cực B của transistor C1815; khi có tín hiệu tương ứng với trạng thái relay, C1815 dẫn và cho phép LED lấy nguồn từ đường 5 V. Cách điều khiển này giúp LED sáng ổn định hơn so với cấp trực tiếp từ mức 3,3 V của GPIO, đồng thời giảm tải dòng cho chân vi điều khiển. Hình 2.14 là ảnh nút nhấn tích hợp đèn LED loại R16-503BD.



Hình 2.14: Nút nhấn tích hợp đèn LED (loại R16-503BD) (nguồn: nhà sản xuất)

2.4.4.2. Thông số kỹ thuật

Nút nhấn loại thường mở, nối với chân GPIO có điện trở kéo, kèm xử lý chống rung (debounce) bằng phần mềm khoảng 50 ms để tránh nhận nhầm một lần nhấn thành nhiều lần. LED báo trạng thái có điện áp thuận khoảng 1,8–3,0 V, dòng làm việc 5–20 mA, nên cần mắc nối tiếp điện trở hạn dòng và dùng transistor đệm khi lấy nguồn từ đường 5 V.

Ngoài hai nút điều khiển từng kênh, thiết bị còn có công tắc nguồn tổng SW3. SW3 đóng/ngắt trực tiếp đường 220 VAC cấp cho toàn bộ thiết bị nên được xếp vào nhóm linh kiện công suất, không phải tín hiệu GPIO. LED L3 là đèn báo nguồn lấy từ đường 5 V sau khối nguồn AC-DC, dùng để cho biết mạch điều khiển đã có nguồn thấp áp hoạt động.

2.4.4.3. Ứng dụng

Mỗi nút nhấn bật/tắt một ổ tại chỗ; nhấn giữ đồng thời hai nút trong thời gian quy định (> 3 giây) sẽ kích hoạt chế độ cấu hình lại WiFi. LED tương ứng hiển thị trạng thái đóng/ngắt của từng ổ, giúp người dùng nhận biết trực quan ngay cả khi không mở dashboard.

2.4.5 Khối nguồn 220VAC – 5VDC

2.4.5.1. Giới thiệu

Khối nguồn chuyển điện áp lưới 220 VAC thành 5 VDC ổn định cấp cho ESP32, relay, PZEM-004T, LED báo nguồn L3 và các linh kiện khác. Đây là khối nền tảng quyết định độ ổn định và an toàn của toàn hệ thống. Hình 2.15 là ảnh module nguồn HLK-PM01B dùng trong đề tài.



Hình 2.15: Module nguồn HLK-PM01B (220VAC → 5VDC)

2.4.5.2. Thông số kỹ thuật

Khối nguồn dùng module chuyển đổi AC-DC HLK-PM01B [12] với thông số: điện áp vào 220 VAC, tần số 50 Hz, điện áp ra 5 VDC, công suất 3 W (5 V/0,6 A), cụ thể là module HLK-PM01B. Ngoài ra dùng thêm IC ổn áp AMS1117-3.3V để hạ 5 V xuống 3,3 V cấp cho ESP32. Cách tính tổng dòng tiêu thụ để chọn công suất nguồn được trình bày ở Chương 3.

2.4.5.3. Ứng dụng

Khối nguồn cấp điện cho toàn bộ mạch điều khiển. Khi thiết kế cần dự phòng công suất để nguồn ổn định lúc relay đóng và ESP32 phát WiFi ở công suất đỉnh, đây là điểm sẽ được tính toán định lượng ở Chương 3.

2.4.6 Các linh kiện phụ trợ khác

Ngoài các module chính ở trên, mạch còn sử dụng một số linh kiện phụ trợ để hoàn thiện chức năng nạp, điều khiển và ổn định hoạt động:

- IC ổn áp AMS1117-3.3V [15]: hạ điện áp 5V từ khối nguồn xuống 3,3V ổn định, cấp riêng cho ESP32 (Hình 2.16).
- IC CH340C [14]: chuyển đổi tín hiệu USB sang UART để nạp firmware cho ESP32 từ máy tính.
- Transistor 2N2222 và C1815: 2N2222 dùng cho mạch tự reset khi nạp firmware; C1815 đóng vai trò transistor đệm/khuếch đại dòng cho LED báo trạng thái, giúp LED lấy nguồn từ đường 5 V thay vì lấy dòng trực tiếp từ GPIO của ESP32.
- Điện trở và tụ điện: điện trở hạn dòng (cho LED, cực gốc transistor) và tụ điện lọc nguồn, giúp mạch hoạt động ổn định và giảm nhiễu.

Giá trị cụ thể của các điện trở, tụ điện cùng vai trò của chúng trong từng khối được tính toán và trình bày chi tiết ở Chương 3.



Hình 2.16: IC ổn áp AMS1117-3.3V (nguồn: nhà sản xuất)

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 GIỚI THIỆU

Trên cơ sở mục tiêu đề ra ở Chương 1 và các kiến thức nền tảng ở Chương 2, chương này trình bày quá trình thiết kế phần cứng của ổ cắm điện. Thiết kế được triển khai theo hướng phân chia hệ thống thành các khối chức năng độc lập, sau đó lựa chọn linh kiện, tính toán và biện luận cho từng khối, cuối cùng ghép nối thành sơ đồ nguyên lý hoàn chỉnh.

Về tổng thể, sản phẩm của đề tài là một ổ cắm điện hai kênh. Thiết bị lấy điện trực tiếp từ lưới 220VAC, dùng module PZEM-004T đo các thông số điện của tải, dùng hai relay đóng/ngắt độc lập hai ổ cắm, và dùng vi điều khiển ESP32 làm bộ não điều phối. ESP32 vừa đọc số liệu đo, quét nút nhấn vật lý, điều khiển relay, vừa kết nối WiFi để đẩy dữ liệu lên đám mây qua giao thức MQTT và nhận lệnh điều khiển từ dashboard web. Người dùng có thể bật/tắt và theo dõi mức tiêu thụ điện của từng ổ ngay tại chỗ bằng nút nhấn hoặc từ xa qua Internet. Toàn bộ phần điện áp cao 220VAC được tách biệt và cách ly an toàn với phần mạch điều khiển điện áp thấp.

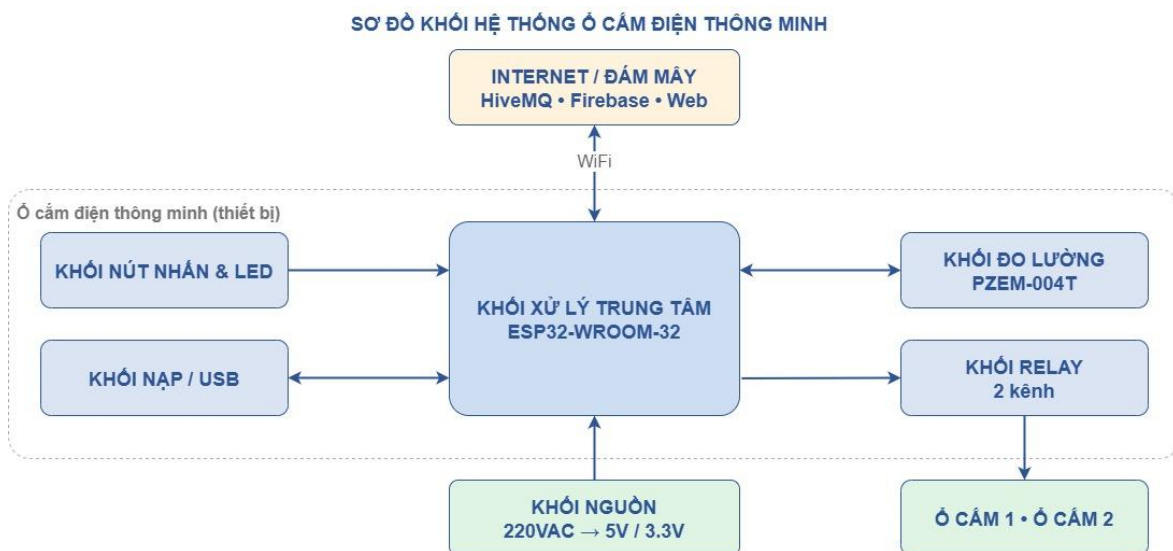
Từ yêu cầu sử dụng, hệ thống cần đáp ứng một số tiêu chí thiết kế. Trước hết, thiết bị phải đo và cung cấp được các thông số điện chính của tải; trong phiên bản dashboard hiện tại hệ thống ưu tiên hiển thị điện áp, dòng điện và công suất, còn module đo vẫn có khả năng hỗ trợ thêm điện năng, tần số và hệ số công suất khi mở rộng phần mềm. Tiếp đến, hai ổ cắm phải đóng và ngắt độc lập được cả bằng nút nhấn vật lý tại chỗ lẫn bằng lệnh điều khiển từ xa qua Internet, đồng thời thiết bị kết nối WiFi để truyền dữ liệu đo, trạng thái và nhận lệnh thông qua giao thức MQTT. Về an toàn, phần mạch điều khiển điện áp thấp phải được cách ly với phần công suất 220VAC. Cuối cùng, thiết bị cần nhỏ gọn để lắp vừa trong vỏ ổ cắm, hoạt động ổn định lâu dài và thuận tiện cho việc nạp/cập nhật firmware qua cổng USB-UART. Các

tiêu chí này là cơ sở để phân chia khối chức năng và lựa chọn linh kiện ở các mục tiếp theo.

3.2 THIẾT KẾ HỆ THỐNG

3.2.1 Sơ đồ khối hệ thống

Hệ thống được xây dựng từ sáu khối chức năng chính, liên kết với nhau như Hình 3.1. Khối nguồn cấp điện cho toàn bộ mạch điều khiển; khối xử lý trung tâm (ESP32) là bộ não điều phối mọi hoạt động; khối đo lường thu thập thông số điện; khối relay đóng/ngắt tải; khối nút nhấn và LED phục vụ điều khiển, hiển thị tại chỗ; và khối nạp/giao tiếp USB phục vụ lập trình.



Hình 3.1: Sơ đồ khối tổng thể của hệ thống

Vai trò của từng khối được mô tả như sau. Khối nguồn chuyển điện lưới 220VAC thành 5VDC bằng module HLK-PM01B rồi hạ tiếp xuống 3,3VDC bằng IC AMS1117 để cấp cho ESP32 và các linh kiện.

Khối xử lý trung tâm dùng vi điều khiển ESP32-WROOM-32, đảm nhận việc đọc dữ liệu đo, quét nút nhấn, điều khiển relay, duy trì kết nối WiFi và MQTT, đồng thời thực thi giải thuật tự bảo vệ an toàn điện.

Khối đo lường là module PZEM-004T dùng biến dòng CT 100A, đo các thông số điện của đường dây và gửi về ESP32 qua UART.

Khối relay gồm module relay hai kênh đóng và ngắt độc lập nguồn 220VAC cấp cho hai ổ cắm.

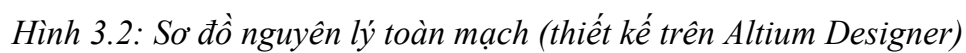
Khối nút nhấn và LED gồm hai nút nhấn điều khiển tại chỗ, hai LED báo trạng thái đóng/ngắt của từng ổ và một LED báo nguồn điều khiển 5V. Hai LED trạng thái được nối theo tín hiệu điều khiển relay nên tự sáng theo trạng thái relay.

Khối nạp và giao tiếp USB gồm mạch CH340C cùng cổng USB và mạch tự reset, cho phép nạp firmware cho ESP32 từ máy tính.

Ngoài ra, ESP32 kết nối qua WiFi tới hạ tầng đám mây gồm broker MQTT (HiveMQ Cloud) và máy chủ backend, từ đó dữ liệu được đưa lên giao diện web để người dùng giám sát và điều khiển từ xa. Trên sơ đồ, khối nguồn cấp điện cho toàn bộ phần điều khiển điện áp thấp, còn khối relay đóng/ngắt nguồn 220VAC cấp cho hai ổ cắm, qua đó thể hiện rõ ranh giới giữa phần điều khiển và phần công suất của hệ thống.

3.2.2 Sơ đồ nguyên lý toàn mạch

Trước khi đi vào thiết kế chi tiết từng khối, mục này trình bày sơ đồ nguyên lý toàn mạch của hệ thống (Hình 3.2) để có cái nhìn tổng thể về cách các khối liên kết với nhau, giúp người đọc dễ theo dõi phần thiết kế chi tiết ở sau. Sơ đồ được thiết kế trên phần mềm Altium Designer, thể hiện đầy đủ liên kết giữa các khối: nguồn HLK-PM01B và AMS1117 cấp 5V và 3,3V; ESP32-WROOM-32 ở trung tâm kết nối tới module relay, PZEM-004T, nút nhấn, LED và mạch nạp CH340C; đường 220VAC chỉ cấp cho module nguồn và đi qua tiếp điểm relay, được cách ly với phần điều khiển điện áp thấp.



Sự phân bổ các chân kết nối của ESP32 trên sơ đồ được tổng hợp ở Bảng 3.1, làm cơ sở cho việc lập trình firmware ở Chương 4. Các chân relay được cấu hình OUTPUT, các chân nút nhấn cấu hình INPUT_PULLUP, còn cặp chân GPIO19 và GPIO21 dùng cho UART giao tiếp với PZEM-004T. Hai đèn LED báo trạng thái relay không dùng chân GPIO riêng mà được nối theo hai tín hiệu điều khiển relay nên tự sáng theo trạng thái relay; LED nguồn L3 lấy từ đường 5V và không chiếm GPIO.

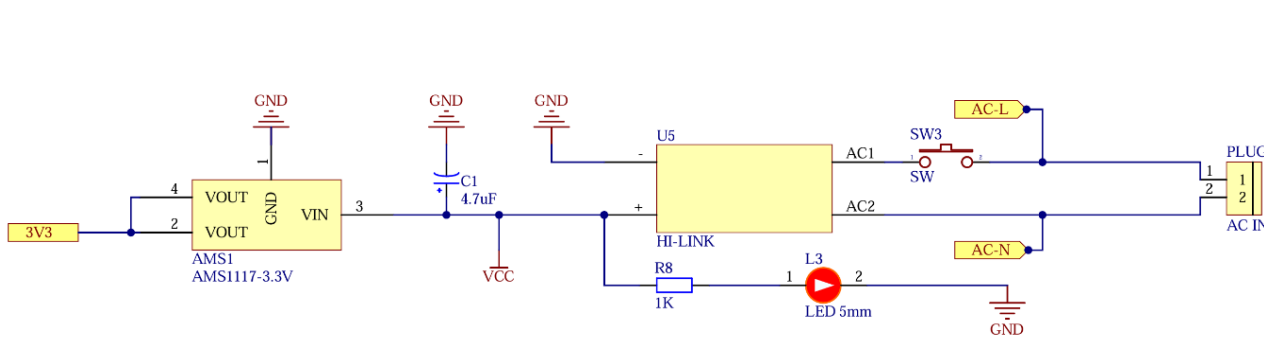
Bảng 3.1: Bảng phân bổ chân kết nối của ESP32

Chức năng	Chân GPIO	Chức năng	Chân GPIO
Relay 1	GPIO 5	Nút nhấn 1	GPIO 26
Relay 2	GPIO 18	Nút nhấn 2	GPIO 27
UART RX (đọc PZEM)	GPIO 19	UART TX (ghi PZEM)	GPIO 21

3.2.3 Thiết kế chi tiết từng khối

3.2.3.1. Khối nguồn

Khối nguồn có nhiệm vụ cấp điện ổn định cho phần mạch điều khiển (ESP32, module relay, phần TTL của PZEM-004T, LED). Cần lưu ý: khối nguồn này chỉ cấp cho mạch điều khiển, còn dòng tải của thiết bị cắm vào ổ đi trực tiếp qua tiếp điểm relay chứ không qua khối nguồn. Sơ đồ nguyên lý khối nguồn được thể hiện ở Hình 3.3.



Hình 3.3: Sơ đồ nguyên lý khối nguồn (HLK-PM01 + AMS1117)

Nhóm chọn module nguồn xung HLK-PM01B (220VAC $\rightarrow$ 5VDC, 0,6A, 3W) [12] vì: kích thước rất nhỏ gọn phù hợp lắp trong vỏ ổ cắm; tích hợp sẵn cách ly và chỉnh lưu nên an toàn và đơn giản hơn so với tự thiết kế mạch biến áp + chỉnh lưu; điện áp ra 5V đúng với mức cấp cho module relay và phần TTL của PZEM. Từ mức 5V, một IC ổn áp tuyến tính AMS1117-3.3V [15] hạ xuống 3,3V cấp riêng cho ESP32 (vốn hoạt động ở 3,3V). Hai cấp 5V và 3,3V giúp mỗi nhóm linh kiện được cấp đúng điện áp định mức.

Để kiểm tra HLK-PM01B 0,6A có đủ hay không [12], ta ước tính dòng tiêu thụ của các thành phần lấy từ nguồn 5V (Bảng 3.2).

Bảng 3.2: Ước tính dòng tiêu thụ của mạch điều khiển

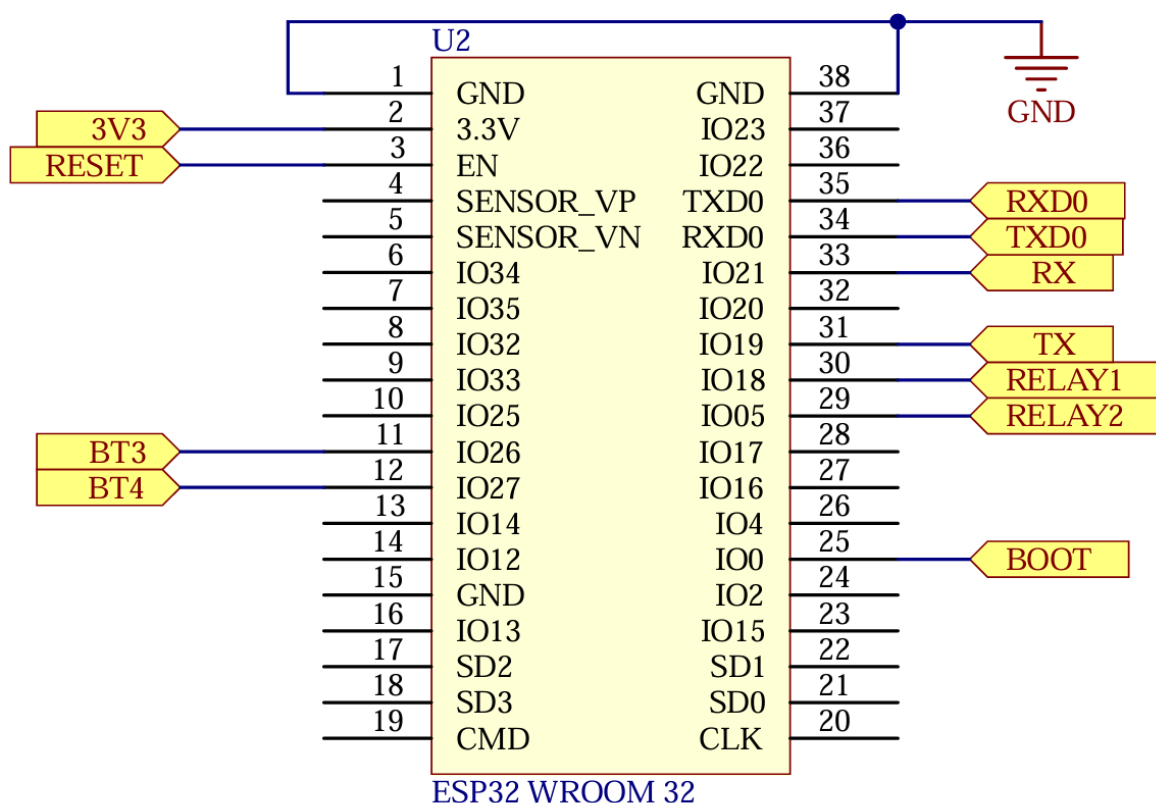
Thành phần	Dòng điển hình	Dòng đỉnh
ESP32-WROOM-32 (qua AMS1117)	~170 mA	~500 mA
Module relay 2 kênh (2 relay đóng)	~150 mA	~160 mA
3 LED báo (L1/L2 qua C1815, L3 báo nguồn)	~9 mA	~9 mA
PZEM-004T (phần giao tiếp TTL)	~15 mA	~20 mA
Tổng cộng	~343 mA	~693 mA
HLK-PM01B cung cấp	600 mA (3W)	600 mA

Nhận xét: ở chế độ hoạt động bình thường, tổng dòng khoảng 343 mA, tức chỉ chiếm khoảng 57% khả năng của nguồn, nguồn đủ và còn dự phòng. Tuy nhiên ở thời điểm đỉnh, khi ESP32 phát WiFi công suất cao đồng thời hai relay cùng đóng, dòng đỉnh tức thời có thể đạt xấp xỉ 693 mA, vượt nhẹ định mức 600 mA của HLK-PM01B. Trên thực tế các đỉnh dòng của ESP32 chỉ kéo dài cỡ micro tới mili-giây và được hấp thụ bởi tụ lọc ở ngõ ra nguồn và ngõ vào ESP32, nên hệ thống vẫn hoạt động ổn định. Để tăng biên độ an toàn, có thể bổ sung tụ điện phân dung lượng lớn (470 đến 1000 μ F) ở ngõ ra 5V để cấp dòng tức thời cho các đỉnh tải. Trong quá trình chạy thử thực

tế, hệ thống không xảy ra hiện tượng khởi động lại hay sụt áp khi hai relay cùng đóng cũng như khi ESP32 phát WiFi, cho thấy nguồn HLK-PM01B đáp ứng đủ cho mạch điều khiển.

3.2.3.2. Khối xử lý trung tâm

Khối xử lý trung tâm sử dụng module ESP32-WROOM-32 [8], đóng vai trò điều phối toàn bộ hoạt động: đọc thông số từ PZEM-004T qua UART, quét trạng thái hai nút nhấn, điều khiển hai relay, duy trì kết nối WiFi và trao đổi dữ liệu qua MQTT, đồng thời thực thi giải thuật tự bảo vệ an toàn điện và phản hồi lệnh điều khiển từ dashboard. Sơ đồ nguyên lý khối xử lý trung tâm được thể hiện ở Hình 3.4.



Hình 3.4: Sơ đồ nguyên lý khối xử lý trung tâm ESP32-WROOM-32

ESP32 được chọn sau khi cân nhắc [8] với một số vi điều khiển phổ biến khác như Arduino Uno, ESP8266 và STM32 (Bảng 3.3). Lý do quan trọng nhất là ESP32 tích hợp sẵn WiFi 2,4GHz ngay trong chip, trong khi Arduino Uno và STM32 không

có WiFi nên phải ghép thêm module thu phát rời, làm mạch phức tạp và tốn thêm chân. So với ESP8266 cũng có WiFi tích hợp, ESP32 có hai lõi xử lý (ESP8266 chỉ một lõi) nên vừa duy trì kết nối mạng vừa xử lý đo lường, điều khiển mà không nghẽn, đồng thời có nhiều RAM và GPIO hơn. Về nhu cầu chân, hệ thống cần tối thiểu 6 chân (2 relay, 2 nút nhấn, 2 UART giao tiếp PZEM; đèn LED không chiếm thêm chân vì nối chung tín hiệu điều khiển relay) và ESP32 với 34 GPIO đáp ứng dư dả. Ngoài ra ESP32 giá rẻ, rất phổ biến, thư viện và cộng đồng hỗ trợ dồi dào, phù hợp nhất với phạm vi và yêu cầu của đồ án. Sơ đồ phân bổ chân cụ thể được tổng hợp ở Bảng 3.1 (mục 3.2.2).

Bảng 3.3: So sánh ESP32-WROOM-32 với một số vi điều khiển khác

Tiêu chí	ESP32-WROOM-32	Arduino Uno	ESP8266	STM32 (F103)
Lõi xử lý	Xtensa LX6, 2 lõi, 240 MHz	AVR 8-bit, 16 MHz	L106, 1 lõi, 80 MHz	Cortex-M3, 72 MHz
SRAM	520 KB	2 KB	khoảng 80 KB	20 KB
WiFi tích hợp	Có, kèm Bluetooth	Không	Có	Không
Số chân GPIO	34	khoảng 20	ít, dùng được khoảng 11	Nhiều
Giá và độ phổ biến	Rẻ, rất phổ biến	Rẻ, phổ biến	Rẻ	Vừa, ít thư viện IoT

Trong họ ESP32, đề tài chọn dòng nguyên bản ESP32-WROOM-32 thay vì các dòng mới hơn như ESP32-C3 hay ESP32-S3 (Bảng 3.4), trên cơ sở cân đối giữa năng lực, chi phí và tính phổ biến. ESP32-C3 chỉ có một lõi RISC-V và 22 chân GPIO, tuy đủ chân nhưng năng lực xử lý và khả năng đa nhiệm kém hơn loại hai lõi. ESP32-S3 mạnh hơn với 45 GPIO và tập lệnh tăng tốc AI, nhưng những thế mạnh đó (AI, đồ họa) không cần thiết cho một ổ cắm trong khi giá lại cao hơn. ESP32-WROOM-32 có

hai lõi đủ dùng, đủ 34 GPIO, đồng thời là dòng phổ biến nhất nên giá rẻ, dễ mua, thư viện và cộng đồng hỗ trợ dồi dào.

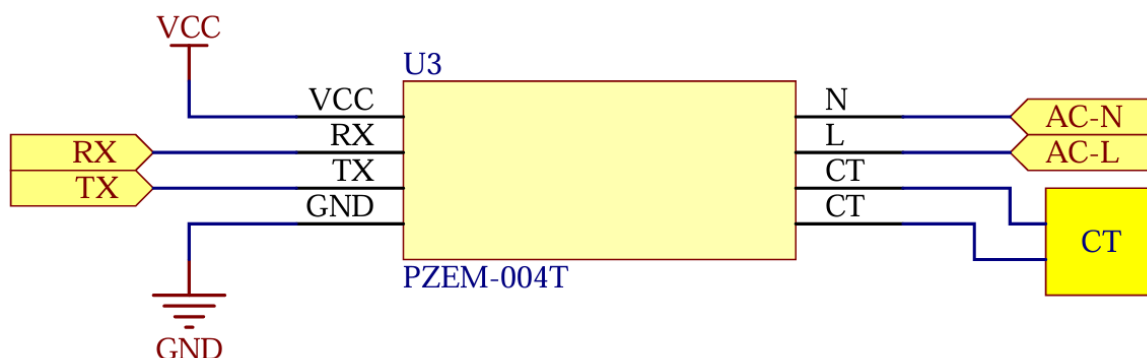
Bảng 3.4: So sánh ESP32-WROOM-32 với các dòng ESP32 khác

Tiêu chí	ESP32-WROOM-32	ESP32-C3	ESP32-S3
Lõi xử lý	Xtensa LX6, 2 lõi, 240 MHz	RISC-V, 1 lõi, 160 MHz	Xtensa LX7, 2 lõi, 240 MHz
SRAM	520 KB	400 KB	512 KB
Bluetooth	Classic + BLE 4.2	BLE 5.0	BLE 5.0
Số chân GPIO	34	22	45
Thế mạnh	Phổ biến, giá rẻ, thư viện dồi dào	Nhỏ gọn, tiết kiệm điện, có USB	Mạnh, hỗ trợ AI, nhiều GPIO

ESP32 hoạt động ở mức 3,3V; các chân GPIO xuất/nhập mức logic 3,3V. Đây là lý do khối nguồn phải tạo riêng mức 3,3V qua AMS1117 cho vi điều khiển, trong khi module relay và PZEM dùng mức 5V.

3.2.3.3. Khối đo lường

Khối đo lường dùng module PZEM-004T phiên bản 100A với biến dòng (CT) dạng vòng [9]. Module đo điện áp bằng cách lấy trực tiếp từ hai dây pha và trung tính (L, N) và đo dòng điện gián tiếp qua biến dòng kẹp quanh dây pha. Dữ liệu đo được truyền về ESP32 qua giao tiếp UART theo giao thức Modbus-RTU (9600 baud, 8N1). Sơ đồ nối dây khối đo lường được thể hiện ở Hình 3.5.



Hình 3.5: Sơ đồ nối dây khối đo lường PZEM-004T (UART, L, N, CT)

PZEM-004T được chọn cho khối đo lường [9] vì hỗ trợ đo V/I/P/kWh/Hz/PF đã hiệu chuẩn sẵn, trả về giá trị số nên tránh được sai số của khâu lấy mẫu ADC như khi tự thiết kế mạch đo bằng cảm biến dòng và mạch chia áp. Phiên bản 100A dùng biến dòng CT có ưu điểm là đo gián tiếp, chỉ cần kẹp CT quanh dây pha mà không phải cắt dòng tải đi xuyên qua module, nhờ đó an toàn và dễ lắp đặt hơn so với phiên bản 10A dùng shunt nối tiếp. Đánh đổi của bản 100A là độ phân giải ở vùng dòng nhỏ không mịn bằng bản 10A; tuy nhiên với mục tiêu giám sát tiêu thụ của đề tài thì sai số 0,5% vẫn đáp ứng tốt.

Bảng 3.5 so sánh PZEM-004T với hai phương án đo khác: dùng IC đo năng lượng chuyên dụng ADE7753 tự thiết kế mạch, và dùng cảm biến dòng ACS712 kết hợp mạch chia áp đưa vào ADC của vi điều khiển.

Bảng 3.5: So sánh PZEM-004T với các phương án đo điện năng khác

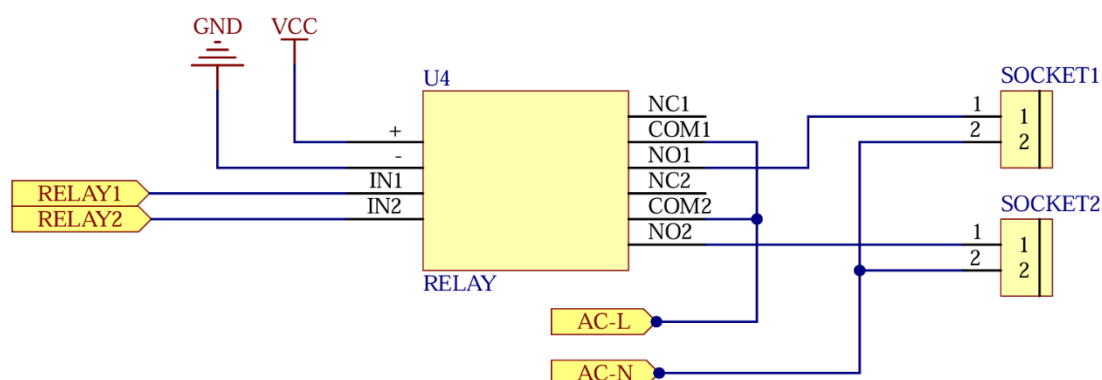
Tiêu chí	Module PZEM-004T	IC ADE7753 (tự thiết kế)	ACS712 + chia áp + ADC
Đại lượng đo	Hỗ trợ 6: V, I, P, kWh, tần số, PF; dashboard hiện dùng V/I/P	V, I, P, kWh	Chỉ dòng và áp, phải tự tính P/kWh/PF

Hiệu chuẩn	Hiệu chuẩn sẵn	Phải tự hiệu chuẩn	Phải tự hiệu chuẩn, có sai số ADC
Cách lấy số liệu	Số hóa, trả qua UART	Đọc thanh ghi qua SPI	Đọc ADC, dễ nhiễu
Cách ly an toàn	Có, cách ly với 220V	Phải tự thiết kế cách ly	Phần đo áp không cách ly tốt
Độ phức tạp	Thấp, cảm là dùng	Cao	Trung bình

Về an toàn, phần đo điện áp cao của PZEM-004T được cách ly quang với phần giao tiếp UART, nên ESP32 không tiếp xúc trực tiếp với điện áp lưới. Trong thiết kế, PZEM đo tổng dòng của cả hai ổ (đo chung) để đơn giản hóa phần cứng; phương án đo riêng từng ổ được nêu ở hướng phát triển (Chương 6).

3.2.3.4. Khối relay

Khối relay dùng module 2 kênh với relay cơ SONGLE SRD-05VDC-SL-C (tiếp điểm 10A/250VAC) [13], cho phép ESP32 ở mức 3,3V điều khiển đóng/ngắt tải 220VAC. Mỗi kênh relay được cách ly với mạch điều khiển bằng opto-coupler để bảo vệ ESP32 khỏi nhiễu và sự cố phía công suất. Sơ đồ nguyên lý khối relay và đầu nối ổ cắm được thể hiện ở Hình 3.6.



Hình 3.6: Sơ đồ nguyên lý khối relay 2 kênh và đầu nối ổ cắm

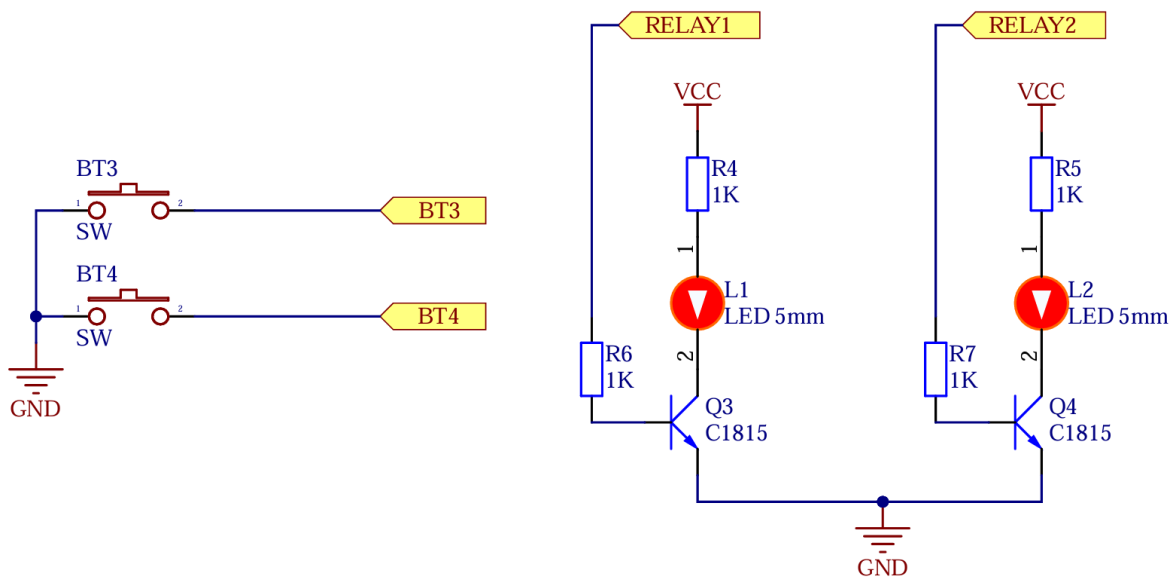
Relay cơ được chọn (thay vì relay bán dẫn SSR) vì giá thành thấp, sụt áp khi dẫn không đáng kể và cách ly hoàn toàn về điện; đổi lại có tiếng kêu đóng cắt và tuổi thọ cơ khí hữu hạn, chấp nhận được với tần suất đóng/ngắt thấp của một ổ cắm gia dụng. Module được cấu hình kích mức cao (đặt jumper sang High-trigger, tương ứng RELAY_ACTIVE_HIGH trong firmware): khi chân GPIO của ESP32 xuất mức cao, kênh relay tương ứng đóng.

Theo thông số relay SRD-05VDC-SL-C, cuộn dây 5V có điện trở xấp xỉ $70\ \Omega$ [13], nên dòng giữ mỗi relay khoảng $I = 5V / 70\Omega \approx 71\text{ mA}$, hai relay khoảng 142 mA, dòng này do nguồn 5V cấp qua transistor trên module, không lấy trực tiếp từ chân ESP32. Chân GPIO của ESP32 chỉ cấp dòng nhỏ qua opto (cỡ vài mA), nằm trong giới hạn an toàn của GPIO. Tiếp điểm relay 10A/250VAC đáp ứng tốt dòng tải của ổ cắm dân dụng.

Về đấu nối công suất: dây pha 220VAC đưa vào chân chung (COM) của relay, chân thường hở (NO) nối ra ổ cắm; khi relay đóng, tiếp điểm COM và NO thông mạch cấp điện cho ổ cắm tương ứng.

3.2.3.5. Khối nút nhấn và LED báo

Khối này gồm hai nút nhấn để người dùng bật/tắt từng ổ tại chỗ, hai LED báo trạng thái đóng/ngắt tương ứng và một LED L3 báo nguồn điều khiển 5V. Hai nút nhấn nối vào chân ESP32 cấu hình INPUT_PULLUP (mức cao khi nhả, mức thấp khi nhấn), kèm xử lý chống rung bằng phần mềm khoảng 50 ms để tránh một lần nhấn bị nhận thành nhiều lần. Sơ đồ nguyên lý khối nút nhấn và mạch LED được thể hiện ở Hình 3.7.



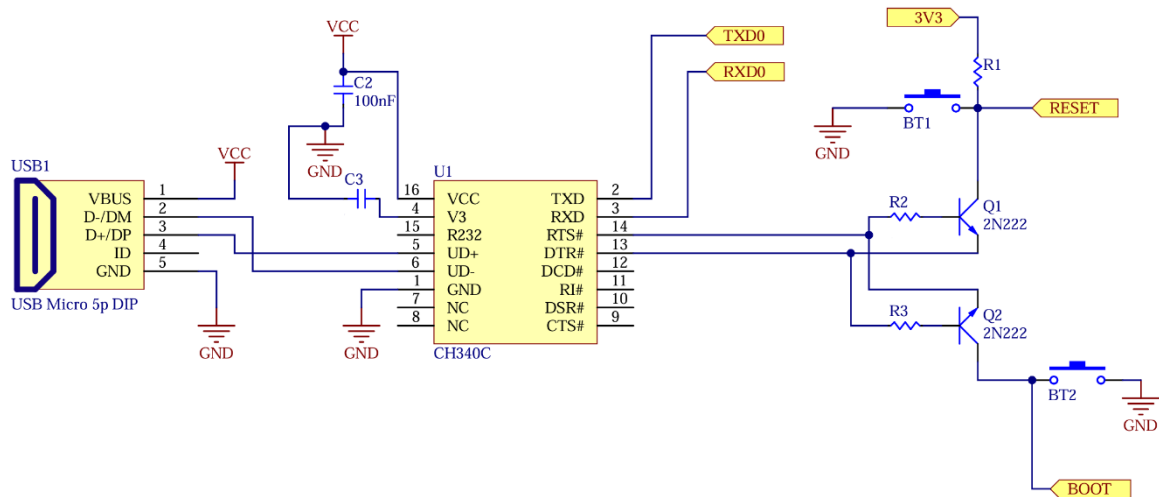
Hình 3.7: Sơ đồ nguyên lý khối nút nhấn và mạch LED báo

Khác với cách nối LED trạng thái vào một chân GPIO riêng, ở thiết kế này mỗi LED báo relay lấy tín hiệu ngay từ đường điều khiển relay tương ứng (IN1 ứng với GPIO5, IN2 ứng với GPIO18) thông qua một transistor C1815 (Q3, Q4). Khi ESP32 kéo chân điều khiển lên mức cao để đóng relay, cùng tín hiệu đó kích transistor làm LED sáng, và khi ngắt relay thì LED tắt. Nhờ vậy L1 và L2 luôn phản ánh đúng trạng thái relay mà không cần thêm chân GPIO điều khiển riêng, trong khi transistor vẫn giúp tách dòng tải LED khỏi chân tín hiệu và cho phép LED lấy điện từ nguồn 5V để sáng rõ. LED L3 được dùng riêng để báo nguồn 5V của mạch điều khiển, không lấy trực tiếp từ 220VAC.

Với LED báo có điện áp thuận khoảng 2V, mắc nối tiếp điện trở 1 kΩ tới nguồn 5V, dòng qua LED là $I = (5 - 2)/1000 \approx 3 \text{ mA}$, đủ sáng cho mục đích báo trạng thái và tiết kiệm dòng. Điện trở cực gốc 1 kΩ nối từ đường tín hiệu điều khiển (mức cao khoảng 3,3V) cho dòng base $I_B = (3,3 - 0,7)/1000 \approx 2,6 \text{ mA}$, thừa sức đưa transistor C1815 vào bão hòa để dẫn dòng collector vài mA của LED.

3.2.3.6. Khối nạp và giao tiếp USB

Khởi nạp dùng IC chuyển đổi USB sang UART CH340C [14] cùng cổng USB micro, cho phép nạp firmware cho ESP32 từ máy tính mà không cần mạch nạp chuyên dụng. CH340C chuyển tín hiệu USB [14] của máy tính thành UART nối tới chân TXD/RXD của ESP32. Sơ đồ nguyên lý khởi nạp và mạch tự reset được thể hiện ở Hình 3.8.



Hình 3.8: Sơ đồ nguyên lý khởi nạp CH340C và mạch tự reset/nạp

Để không phải bấm nút thủ công mỗi lần nạp, mạch dùng hai transistor 2N2222 ghép tín hiệu DTR/RTS từ CH340C tới chân EN (reset) và IO0 (boot) của ESP32. Khi trình nạp điều khiển DTR/RTS, hai transistor tự đưa ESP32 vào chế độ nạp rồi khởi động lại, đây là mạch auto-reset tiêu chuẩn cho ESP32, giúp việc nạp/cập nhật firmware qua USB-UART thuận tiện.

Bên cạnh sáu khối phần cứng ở trên, hệ thống còn có khối thu phát tín hiệu không dây tích hợp trong ESP32 và khối ứng dụng đám mây ở phía ngoài thiết bị, được trình bày dưới đây.

3.2.3.7. Khối thu phát tín hiệu (kết nối WiFi)

Khác với một số thiết bị phải gắn thêm module thu phát rời, ESP32-WROOM-32 đã tích hợp sẵn bộ thu phát WiFi 2,4GHz theo chuẩn IEEE 802.11 b/g/n ngay trong chip [8], nên hệ thống không cần thiết kế mạch thu phát riêng. Nhờ đó thiết bị kết nối

trực tiếp tới router WiFi của hộ gia đình rồi qua đó liên lạc với broker MQTT trên Internet. Khi bật lần đầu hoặc khi chưa có thông tin mạng, ESP32 tự chuyển sang chế độ điểm truy cập (Access Point) phát ra một mạng WiFi cấu hình để người dùng nhập tên mạng, mật khẩu và mã nhà; các thông tin này được lưu vào bộ nhớ NVS nên những lần khởi động sau thiết bị tự kết nối lại mà không phải cấu hình lại. Trong quá trình hoạt động, firmware liên tục theo dõi trạng thái kết nối và tự động kết nối lại khi mất WiFi hoặc mất MQTT, bảo đảm kênh truyền luôn ổn định.

3.2.3.8. Khối ứng dụng Internet/đám mây

Khối ứng dụng đám mây nằm ngoài phần cứng nhúng, đóng vai trò tiếp nhận, lưu trữ dữ liệu và cung cấp giao diện cho người dùng. Thiết bị chỉ kết nối tới một đầu mối duy nhất là broker MQTT HiveMQ Cloud [10] để vừa đẩy dữ liệu đo và trạng thái, vừa nhận lệnh điều khiển. Phía sau broker, một máy chủ backend đăng ký các topic của thiết bị để tiếp nhận bản tin rồi ghi vào cơ sở dữ liệu, đồng thời đẩy cập nhật theo thời gian thực lên giao diện web dashboard để người dùng giám sát, điều khiển từ xa và quản lý nhiều người dùng, nhiều thiết bị. Như vậy, phạm vi thiết kế phần cứng trong báo cáo này dừng ở mức thiết bị xuất bản dữ liệu lên broker; phần thiết kế chi tiết của máy chủ backend và dashboard web thuộc phần mềm web của hệ thống.

3.3 KIẾN TRÚC PHẦN MỀM VÀ LỰA CHỌN CÔNG NGHỆ

Sau khi hoàn thiện thiết kế phần cứng, hệ thống cần một kiến trúc phần mềm đủ rõ ràng để biến thiết bị ở cứng thành một hệ thống IoT có thể sử dụng thực tế. Nếu chỉ có ESP32, PZEM và relay, thiết bị mới dừng ở mức đo lường và đóng ngắt cục bộ. Để người dùng có thể đăng nhập, quản lý nhiều nhà, nhiều phòng, xem dữ liệu thời gian thực, đặt lịch, nhận cảnh báo và truy xuất lịch sử, hệ thống phải được tổ chức thành các tầng phần mềm có trách nhiệm riêng.

3.3.1 Kiến trúc phần mềm tổng quan

Kiến trúc phần mềm tổng quan được tổ chức theo mô hình phân tầng từ trên xuống, gồm bốn tầng chính.

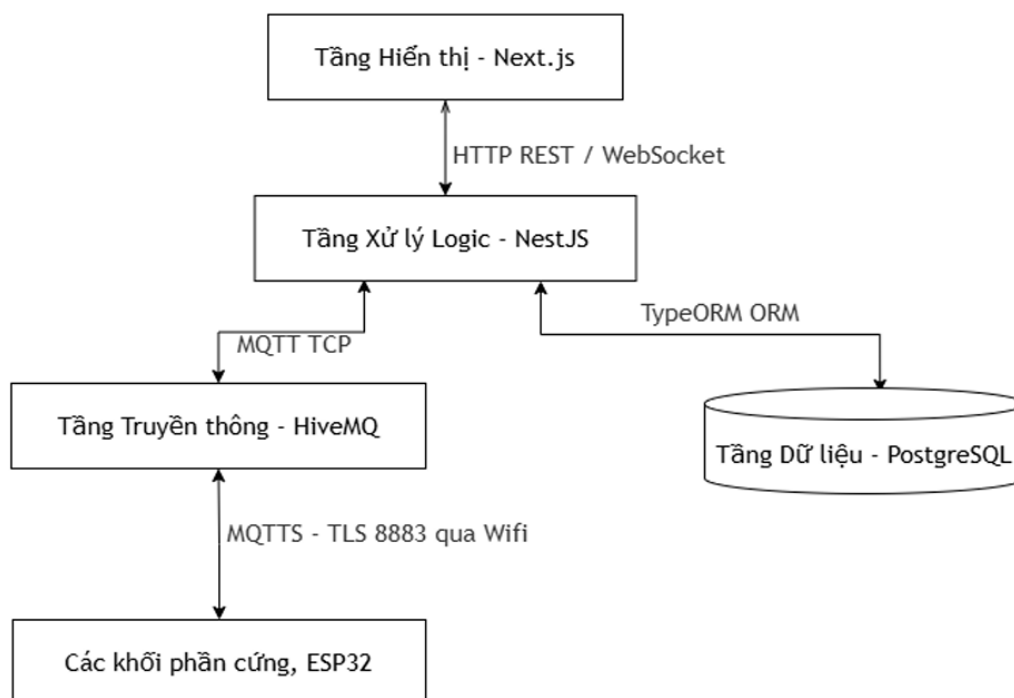
Tầng hiển thị là dashboard web Next.js, cung cấp giao diện để người dùng quan sát dữ liệu đo, bật/tắt thiết bị, quản lý nhà/phòng và thiết lập lịch tự động.

Tầng xử lý logic là backend NestJS, đóng vai trò trung tâm điều phối nghiệp vụ, kiểm tra xác thực, phân quyền, nhận dữ liệu MQTT, ghi cơ sở dữ liệu, phát WebSocket và gửi cảnh báo.

Tầng truyền thông và dữ liệu gồm HiveMQ Cloud và PostgreSQL, đảm nhiệm trung chuyển bản tin giữa thiết bị với backend, đồng thời lưu trữ dữ liệu quan hệ và dữ liệu điện năng.

Tầng thiết bị nhúng là ESP32, chịu trách nhiệm đọc PZEM-004T, điều khiển relay, xử lý nút nhấn, bảo vệ tải và gửi trạng thái lên MQTT.

Sơ đồ tổng quan trong Hình 3.9 thể hiện mối quan hệ giữa bốn tầng. Tầng hiển thị không giao tiếp trực tiếp với ESP32 mà thông qua backend. Cách này giúp mọi thao tác điều khiển đều đi qua xác thực người dùng và kiểm tra quyền. Backend không kết nối trực tiếp tới địa chỉ IP nội bộ của ESP32 mà thông qua broker MQTT, nhờ đó thiết bị có thể đặt trong mạng WiFi gia đình mà vẫn trao đổi được với cloud. PostgreSQL không nhận dữ liệu trực tiếp từ thiết bị, mà chỉ được cập nhật sau khi backend đã kiểm tra, chuyển đổi và ghi nhận sự kiện.



Hình 3.9: Sơ đồ kiến trúc phần mềm tổng quan của hệ thống

Mối liên kết giữa tầng hiển thị và tầng logic sử dụng đồng thời HTTP REST và WebSocket. HTTP REST phù hợp với các tác vụ có dạng yêu cầu - phản hồi như đăng nhập, tạo nhà, tạo phòng, tạo lịch và truy vấn lịch sử. WebSocket phù hợp với dữ liệu cần cập nhật ngay như trạng thái relay, node online/offline, công suất tức thời và nhật ký mới. Nếu chỉ dùng REST, dashboard phải polling liên tục; nếu chỉ dùng WebSocket cho toàn bộ nghiệp vụ, các thao tác cần xác thực và phân quyền sẽ khó tổ chức theo chuẩn API.

Mối liên kết giữa tầng logic và tầng truyền thông sử dụng MQTT TCP thông qua HiveMQ Cloud. Backend NestJS là một MQTT client chạy ngầm, subscribe các topic state/ack từ ESP32 và publish lệnh điều khiển xuống topic cmd.

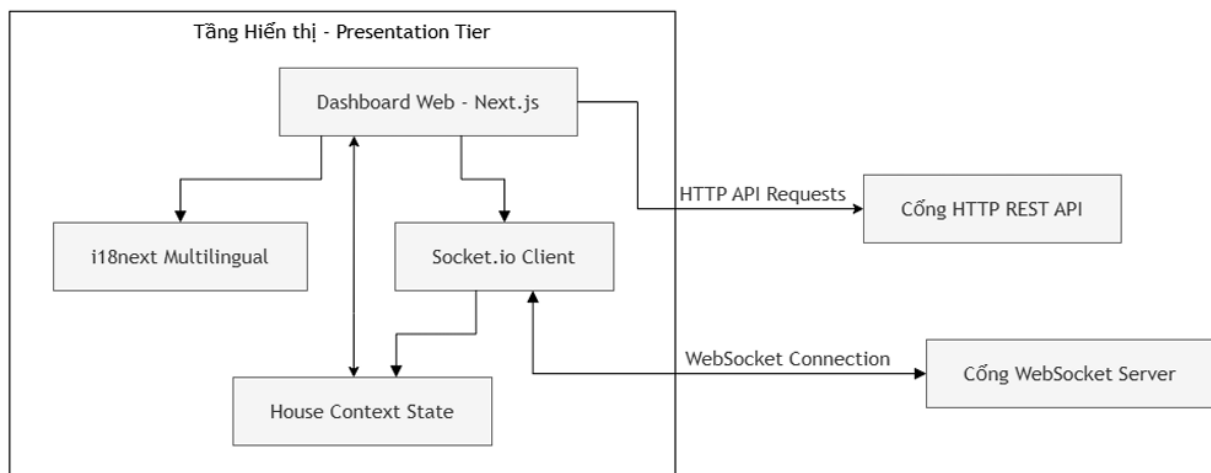
Mối liên kết giữa tầng logic và tầng dữ liệu sử dụng TypeORM ORM, giúp ánh xạ các entity TypeScript sang bảng PostgreSQL và giảm lỗi khi truy cập dữ liệu quan hệ.

Mối liên kết giữa tầng truyền thông và tầng thiết bị nhúng sử dụng MQTTS qua WiFi trên cổng 8883. Trong firmware hiện tại, ESP32 dùng TLS nhưng ở chế độ

setInsecure(), nên báo cáo chỉ khẳng định kênh truyền được mã hóa, không khẳng định thiết bị đã xác thực chứng chỉ CA đầy đủ.

3.3.2 Tầng hiển thị (Presentation Tier)

Tầng hiển thị chạy trên trình duyệt và được xây dựng bằng Next.js. Cấu trúc của tầng này được thiết kế xoay quanh dashboard, trạng thái ngôi nhà hiện tại và kênh cập nhật thời gian thực. Hình 3.10 thể hiện các thành phần chính của tầng hiển thị: Dashboard Web, House Context State, i18next Multilingual, Socket.IO Client, cổng HTTP REST API và cổng WebSocket Server.



Hình 3.10: Cấu trúc tầng hiển thị của dashboard web

Dashboard Web và House Context State có mối liên kết hai chiều. Dashboard đọc trạng thái từ context để hiển thị số đo điện áp, dòng điện, công suất, điện năng và trạng thái bật/tắt của relay. Khi người dùng thao tác trên giao diện, dashboard cập nhật yêu cầu vào context hoặc gọi API tương ứng để toàn bộ component con có cùng trạng thái. Dashboard Web gọi i18next để hiển thị đa ngôn ngữ, giúp giao diện có thể chuyển đổi giữa tiếng Việt và tiếng Anh mà không phải viết lại logic điều khiển.

Dashboard Web kết nối Socket.IO Client để nhận sự kiện thời gian thực từ backend. Khi backend nhận bản tin MQTT mới từ ESP32, Socket.IO Client cập nhật dữ liệu vào House Context State, từ đó giao diện tự render lại mà không cần tải lại trang. Với các tác vụ như đăng nhập, tải danh sách nhà/phòng, tạo lịch hoặc xem nhật

ký, Dashboard Web gửi HTTP REST request tới backend. Như vậy, tầng hiển thị tách rõ hai loại dữ liệu: dữ liệu cấu hình truy xuất qua REST và dữ liệu trạng thái tức thời nhận qua WebSocket.

Về thư viện giao diện, hệ thống dùng Tailwind CSS kết hợp Shadcn UI/Radix UI để xây dựng các thành phần như nút bật/tắt, thẻ thống kê, bảng nhật ký, hộp thoại và biểu đồ. Lựa chọn này giúp giao diện có thể tùy biến nhanh bằng class tiện ích, đồng thời vẫn giữ được các component có hành vi truy cập tốt. Dashboard gồm các màn hình chính như trang tổng quan, trang chi tiết phòng, trang điều khiển thiết bị, trang lịch sử và trang thông kê điện năng.

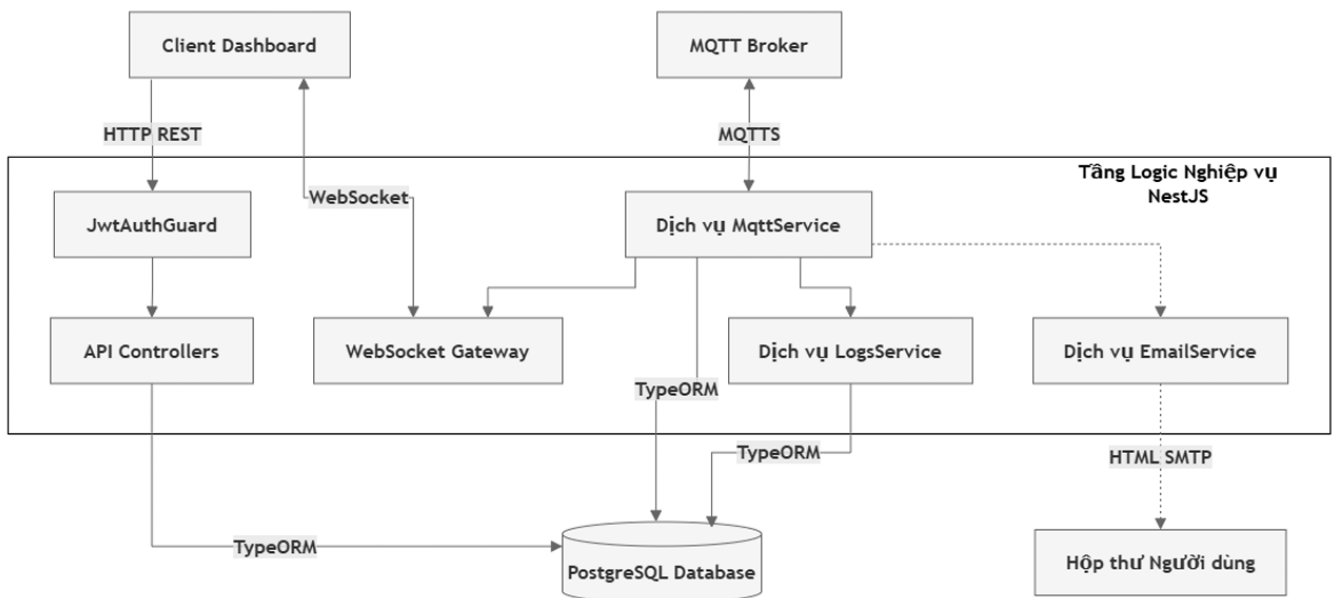
Bảng 3.6: So sánh lựa chọn công nghệ frontend cho dashboard

Tiêu chí	HTML/JavaScript thuần	ReactJS SPA	Next.js
Cơ chế hiển thị	Thao tác DOM thủ công, phù hợp trang nhỏ.	Client-side rendering qua Virtual DOM.	Kết hợp server-side rendering và client-side rendering.
Tốc độ tải đầu	Nhanh khi giao diện rất đơn giản, chậm khi logic lớn.	Phải tải bundle JavaScript trước khi hiển thị đầy đủ.	Tối ưu tải đầu tốt hơn nhờ SSR và chia nhỏ mã.
Quản lý route	Phải tự viết logic chuyển trang.	Cần thêm thư viện router.	App Router tích hợp sẵn, cấu trúc rõ.
Đa ngôn ngữ và bảo mật	Khó tổ chức đồng bộ và dễ lộ logic.	Triển khai được nhưng chủ yếu ở client.	Có middleware và server runtime hỗ trợ kiểm soát tốt hơn.
Phù hợp dashboard IoT	Không phù hợp khi có nhiều trạng thái realtime.	Phù hợp giao diện động nhưng chưa tối ưu tải đầu.	Được chọn vì cân bằng hiệu năng, cấu trúc và khả năng mở rộng [21].

Từ Bảng 3.6, HTML/JavaScript thuần bị loại vì hệ thống không chỉ có một trang bật/tắt đơn giản. Dashboard cần quản lý tài khoản, nhà, phòng, node, thiết bị, lịch, log và biểu đồ. ReactJS SPA có thể đáp ứng phần giao diện động, nhưng khi dự án lớn, toàn bộ mã client ban đầu dễ tăng kích thước và làm thời gian tải đầu kéo dài. Next.js được chọn vì vẫn giữ ưu điểm component của React, đồng thời hỗ trợ App Router, middleware và khả năng render linh hoạt. Lựa chọn này phù hợp với dashboard cần vừa tải nhanh, vừa duy trì kết nối WebSocket để cập nhật trạng thái thời gian thực.

3.3.3 Tầng xử lý logic (Logic Tier)

Tầng xử lý logic được triển khai bằng NestJS trên môi trường Node.js. Đây là lớp trung tâm của toàn hệ thống, tiếp nhận request từ dashboard, xác thực người dùng, kiểm tra quyền, giao tiếp với MQTT broker, ghi cơ sở dữ liệu, phát WebSocket và gửi email cảnh báo. Cấu trúc chi tiết của tầng logic được thể hiện ở Hình 3.11.



Hình 3.11: Cấu trúc tầng xử lý logic trên backend NestJS

Client Dashboard gửi HTTP request qua JwtAuthGuard trước khi tới API Controllers. JwtAuthGuard kiểm tra chữ ký JWT và đảm bảo request thuộc về người

dùng hợp lệ. Sau khi xác thực, API Controllers như DevicesController, RoomsController hoặc HousesController xử lý yêu cầu nghiệp vụ. Với các thao tác realtime, Client Dashboard kết nối hai chiều với WebSocket Gateway để nhận trạng thái mới sau khi backend xử lý dữ liệu.

MqttService là cầu nối giữa backend và HiveMQ Broker. Service này subscribe bản tin state/ack từ ESP32 và publish bản tin cmd xuống thiết bị. Khi MqttService nhận dữ liệu đo hoặc trạng thái relay, nó gọi LogsService và các service dữ liệu để ghi nhận vào PostgreSQL, sau đó thông báo WebSocket Gateway phát sự kiện tới các client đang ở cùng room house. Khi nhận cảnh báo điện từ trường alert trong state, backend ghi log cảnh báo và có thể gọi EmailService để gửi email cho thành viên trong nhà.

NestJS cũng là nơi tích hợp xác thực Google OAuth 2.0 và JWT. Người dùng đăng nhập bằng tài khoản thường hoặc Google, sau đó backend cấp token để truy cập API. Các quyền như quản lý phòng, quản lý node và quản lý thiết bị được kiểm tra ở tầng service dựa trên house_members. Nhờ đó, một người dùng chỉ có thể điều khiển thiết bị trong nhà mà họ thuộc về, thay vì chỉ dựa vào giao diện frontend.

Bảng 3.7: So sánh lựa chọn công nghệ backend

Tiêu chí	ExpressJS	Python/Django	NestJS
Kiến trúc mã nguồn	Nhẹ nhưng không áp đặt cấu trúc, dễ rối khi nhiều module.	MVC rõ nhưng khá nặng cho realtime IoT nhỏ.	Module, Controller, Service và Dependency Injection rõ ràng.
Hiệu năng I/O	Tốt nhờ Node.js bất đồng bộ.	Không tự nhiên bằng Node.js cho nhiều kết nối realtime liên tục.	Tốt cho HTTP, WebSocket và MQTT chạy đồng thời.
Kiểm soát kiểu dữ liệu	JavaScript thuần dễ lỗi runtime.	Python có type hint nhưng không bắt buộc.	TypeScript giúp kiểm soát DTO, entity và payload.
Khả năng mở rộng	Phải tự đặt quy	Mạnh cho web	Phù hợp hệ thống

	ước khi dự án lớn.	truyền thống, nhưng có thể dư thừa.	nhiều service nhưng vẫn thống nhất.
Lý do chọn	Không chọn vì thiếu khung chuẩn cho hệ nhiều module.	Không chọn vì không tối ưu cho stack Node realtime.	Được chọn vì đồng bộ với TypeScript, Socket.IO và MQTT [20].

Bảng 3.7 cho thấy NestJS được chọn không phải chỉ vì có nhiều tính năng, mà vì nó phù hợp với bản chất của hệ thống. Backend phải xử lý nhiều luồng bất đồng bộ: REST API từ dashboard, WebSocket tới trình duyệt, MQTT tới broker, truy vấn PostgreSQL và email cảnh báo. ExpressJS nhẹ nhưng khi số module tăng sẽ cần tự xây cấu trúc. Django mạnh nhưng thiên về web truyền thống hơn là một backend Node.js realtime. NestJS giữ ưu điểm I/O bất đồng bộ của Node.js, đồng thời đưa dự án vào cấu trúc module rõ ràng, dễ bảo trì.

3.3.4 Tầng cơ sở dữ liệu (Database Tier)

Tầng cơ sở dữ liệu lưu trữ toàn bộ dữ liệu bền vững của hệ thống: tài khoản, nhà, phòng, thành viên, quyền, node ESP32, thiết bị relay, nhật ký, dữ liệu điện năng và lịch tự động. Dữ liệu của đề tài có quan hệ chặt chẽ: một người dùng có thể tham gia nhiều nhà, một nhà có nhiều phòng, một phòng có nhiều node, một node có nhiều thiết bị, mỗi thiết bị có log và thống kê điện năng. Do đó, hệ quản trị cơ sở dữ liệu quan hệ phù hợp hơn mô hình lưu trữ tài liệu tự do.

Bảng 3.8: So sánh lựa chọn hệ quản trị cơ sở dữ liệu

Tiêu chí	MySQL	MongoDB	PostgreSQL
Mô hình dữ liệu	Quan hệ, phổ biến cho CRUD.	Tài liệu JSON linh hoạt nhưng khó ràng buộc nhiều quan hệ.	Quan hệ mạnh, có thể kết hợp JSONB khi cần.
Toàn vẹn dữ liệu	Có khóa ngoại và	Toàn vẹn quan hệ phụ thuộc nhiều	Khóa ngoại, transaction và ràng

	transaction.	vào ứng dụng.	buộc dữ liệu mạnh.
Phù hợp RBAC	Phù hợp.	Không tối ưu khi quan hệ user-house-role phức tạp.	Rất phù hợp với mô hình người dùng, nhà và quyền.
Truy vấn thống kê	Dùng được.	Linh hoạt nhưng dễ lặp dữ liệu.	Mạnh cho truy vấn log và dữ liệu theo thời gian.
Lý do chọn	Không chọn vì PostgreSQL linh hoạt hơn cho thống kê.	Không chọn vì dữ liệu quan hệ là trọng tâm.	Được chọn vì cần toàn vẹn quan hệ và truy vấn thống kê [18].

Từ Bảng 3.8 có thể thấy PostgreSQL được chọn vì hệ thống cần cả tính toàn vẹn quan hệ lẫn truy vấn thống kê. Nếu dùng MongoDB, các quan hệ như người dùng - nhà - quyền - phòng - thiết bị có thể bị lặp hoặc phải tự kiểm soát ở tầng backend. PostgreSQL cho phép dùng khóa ngoại để ràng buộc dữ liệu và transaction để đảm bảo các thao tác nhiều bảng không bị dở dang.

Bảng 3.9: So sánh lựa chọn thư viện ORM

Tiêu chí	SQL thuần	Prisma	TypeORM
Cách thao tác dữ liệu	Linh hoạt nhưng dễ lặp code.	Schema rõ, migration tốt.	Entity TypeScript ánh xạ trực tiếp sang bảng.
Tích hợp NestJS	Phải tự quản lý connection và repository.	Tích hợp được nhưng phong cách riêng.	Tích hợp tốt với module, repository và decorator.
Quan hệ bảng	Phải tự join và map dữ liệu.	Hỗ trợ tốt.	Hỗ trợ entity relation, query builder và repository.
Lý do chọn	Không chọn vì khó bảo trì khi API	Không chọn để giữ đồng bộ với entity	Được chọn vì phù hợp NestJS và

	tăng.	NestJS hiện có.	TypeScript [19].
--	-------	-----------------	------------------

Từ Bảng 3.9, TypeORM được chọn để chuẩn hóa cách backend thao tác với PostgreSQL. Khi một bản tin MQTT đến, backend có thể phải cập nhật node, device, logs và bảng thống kê điện năng. Nếu viết SQL rời rạc ở nhiều service, nguy cơ sai tên trường hoặc thiếu transaction cao hơn. TypeORM giúp entity trong mã nguồn phản ánh cấu trúc bảng, đồng thời vẫn cho phép dùng query builder với truy vấn thống kê phức tạp.

3.3.5 Tầng truyền thông (Communication Tier)

Tầng truyền thông trung chuyển dữ liệu giữa các thành phần phân tán. Có hai kênh chính: MQTT giữa ESP32 và backend, WebSocket giữa backend và dashboard. HiveMQ Cloud Broker làm trung gian cho ESP32 và backend thông qua MQTTs trên cổng 8883, đồng thời hỗ trợ Last Will and Testament để phát hiện mất kết nối bất thường. Socket.IO Gateway duy trì kết nối song công giữa trình duyệt và backend để đồng bộ trạng thái thiết bị tức thời.

Bảng 3.10: So sánh lựa chọn công nghệ realtime giữa dashboard và backend

Tiêu chí	Server-Sent Events	Native WebSocket	Socket.IO
Hướng truyền dữ liệu	Một chiều từ server tới client.	Hai chiều.	Hai chiều.
Tự kết nối lại	Có hỗ trợ cơ bản ở trình duyệt.	Phải tự lập trình.	Có cơ chế auto-reconnect.
Quản lý phòng/kênh	Không có sẵn.	Phải tự viết logic room.	Có namespaces và rooms.
Phù hợp dashboard nhiều nhà	Không phù hợp vì không gửi lệnh ngược.	Phù hợp nhưng tốn công chuẩn hóa.	Được chọn vì room theo houseId và reconnect [22].

Từ Bảng 3.10, SSE không phù hợp vì dashboard cần gửi thao tác điều khiển ngược lại backend. Native WebSocket có độ trễ thấp nhưng thiếu sẵn cơ chế room, trong khi hệ thống phải tách dữ liệu theo từng houseId để đảm bảo người dùng chỉ nhận dữ liệu của nhà mình. Socket.IO được chọn vì cung cấp room, namespace, cơ chế reconnect và mô hình sự kiện rõ ràng.

Bảng 3.11: So sánh lựa chọn giao thức IoT giữa backend và ESP32

Tiêu chí	HTTP/HTTPS REST	CoAP	MQTT
Mô hình truyền tin	Request-response, client-server.	Request-response trên UDP.	Publish-subscribe qua broker.
Bảng thông	Header lớn, kém tối ưu cho gửi thường xuyên.	Header nhỏ.	Header nhỏ, phù hợp telemetry.
Duy trì trạng thái	Không duy trì kết nối liên tục.	Không duy trì TCP liên tục.	Keep-alive trên TCP.
Điều khiển từ xa	Backend khó đẩy lệnh nếu ESP32 không mở server.	Có thể dùng nhưng ít phổ biến trong stack hiện tại.	Backend publish cmd, ESP32 subscribe topic riêng.
Phát hiện offline	Phải tự xây heartbeat.	Phải tự xử lý.	Có Last Will and Testament.
Lý do chọn	Không chọn vì nặng và polling nhiều.	Không chọn vì UDP dễ mất gói khi WiFi yếu.	Được chọn vì nhẹ, có broker và LWT [10].

Từ Bảng 3.11, MQTT phù hợp với thiết bị nhúng vì ESP32 không cần công khai địa chỉ IP nội bộ. Thiết bị chỉ cần duy trì kết nối tới broker, publish state và subscribe cmd. Backend cũng chỉ cần kết nối broker để nhận dữ liệu và phát lệnh. Cơ chế publish/subscribe giúp mở rộng nhiều node dễ hơn so với HTTP polling, đồng thời LWT giúp backend biết thiết bị mất kết nối đột ngột.

3.4 THIẾT KẾ CƠ SỞ DỮ LIỆU

3.4.1 Mô hình quan hệ và vai trò các bảng dữ liệu

Để đảm bảo tính toàn vẹn dữ liệu, kiểm soát bảo mật đa người dùng và tối ưu truy vấn điện năng, cơ sở dữ liệu được chia thành 13 bảng chính. Mỗi bảng phục vụ một nhóm nghiệp vụ cụ thể thay vì gom tất cả dữ liệu vào một bảng lớn. Cách tách này giúp hệ thống dễ mở rộng, dễ phân quyền và dễ truy vết lỗi.

Bảng 3.12: Vai trò các bảng dữ liệu chính trong hệ thống

Bảng	Vai trò
users	Lưu tài khoản, email, mật khẩu đã băm, Google ID và thông tin khôi phục.
houses	Lưu các ngôi nhà, làm phạm vi quản lý thiết bị và thành viên.
house_members	Lưu quan hệ nhiều-nhiều giữa người dùng và nhà, kèm vai trò/quyền.
house_join_requests	Quản lý yêu cầu xin tham gia nhà và trạng thái phê duyệt.
house_permission_requests	Quản lý yêu cầu thay đổi quyền của thành viên trong nhà.
rooms	Tổ chức thiết bị theo phòng/khu vực vật lý.
nodes	Lưu bo ESP32 vật lý, hardware_id, trạng thái mạng và thông số đo gần nhất.
devices	Lưu relay/thiết bị ngõ ra, pin GPIO và trạng thái gần nhất.
logs	Ghi nhật ký thao tác, cảnh báo, trạng thái trước/sau và nguồn sự kiện.
hourly_energy	Lưu điện năng tiêu thụ theo khung giờ.
daily_energy	Gộp điện năng theo ngày để truy vấn biểu đồ nhanh.
monthly_energy	Gộp điện năng theo tháng để theo dõi dài hạn.

device_schedules	Lưu timer và lịch bật/tắt định kỳ do backend thực thi.
------------------	--

Bảng 3.12 cho thấy cơ sở dữ liệu được thiết kế theo mạch nghiệp vụ. Nhóm users, houses và house_members phục vụ xác thực và phân quyền. Nhóm rooms, nodes và devices ánh xạ thể giới vật lý sang dashboard. Nhóm logs và energy phục vụ truy vết và thống kê. Nhóm device_schedules phục vụ tự động hóa. Nhờ tách như vậy, mỗi loại dữ liệu có thể được cập nhật và truy vấn theo mục tiêu riêng.

3.4.2 Đặc tả chi tiết các bảng dữ liệu quan hệ

Các Bảng 3.13 đến Bảng 3.25 lần lượt đặc tả những trường dữ liệu quan trọng nhất của hệ thống. Phần đặc tả này được đưa vào Chương 3 để cho thấy thiết kế phần mềm có cấu trúc dữ liệu rõ ràng, không chỉ dừng ở mô tả giao thức.

Bảng 3.13: Đặc tả cấu trúc bảng users

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh duy nhất của tài khoản người dùng.
email	VARCHAR	Unique	Địa chỉ email dùng đăng nhập và nhận cảnh báo.
password	VARCHAR		Mật khẩu đã băm, có thể null nếu dùng OAuth.
name	VARCHAR		Tên hiển thị của người dùng.
googleId	VARCHAR		ID người dùng Google phục vụ OAuth.
avatar	TEXT		Đường dẫn ảnh đại diện.
reset_token	VARCHAR		Token đặt lại mật khẩu.
reset_token_expires	TIMESTAMP		Thời điểm hết hạn token đặt lại mật khẩu.

created_at	TIMESTAMP		Thời điểm tạo tài khoản.
------------	-----------	--	--------------------------

Bảng 3.14: Đặc tả cấu trúc bảng houses

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh duy nhất của ngôi nhà.
name	VARCHAR		Tên gọi nhớ của ngôi nhà.
owner_id	UUID	FK	Tham chiếu chủ sở hữu trong bảng users.
created_at	TIMESTAMP		Thời điểm tạo nhà.

Bảng 3.15: Đặc tả cấu trúc bảng house_members

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh quan hệ thành viên.
user_id	UUID	FK	Tham chiếu người dùng.
house_id	UUID	FK	Tham chiếu ngôi nhà.
role	VARCHAR		Vai trò owner hoặc member.
can_manage_rooms	BOOLEAN		Quyền quản lý phòng.
can_manage_nodes	BOOLEAN		Quyền cấu hình node ESP32.
can_manage_devices	BOOLEAN		Quyền quản lý/bật tắt thiết bị.
joined_at	TIMESTAMP		Thời điểm tham gia nhà.

Bảng 3.16: Đặc tả cấu trúc bảng house_join_requests

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh yêu cầu tham gia.
house_id	UUID	FK	Nhà được yêu cầu tham gia.
user_id	UUID	FK	Người dùng gửi yêu cầu.
status	VARCHAR		pending, approved hoặc rejected.
created_at	TIMESTAMP		Thời điểm gửi yêu cầu.
resolved_at	TIMESTAMP		Thời điểm xử lý yêu cầu.
resolved_by	UUID		Người phê duyệt hoặc từ chối.

Bảng 3.17: Đặc tả cấu trúc bảng house_permission_requests

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh yêu cầu cấp quyền.
house_id	UUID	FK	Ngôi nhà liên quan.
user_id	UUID	FK	Thành viên gửi yêu cầu.
permission_type	VARCHAR		Loại quyền cần thay đổi.
status	VARCHAR		pending, approved hoặc rejected.
created_at	TIMESTAMP		Thời điểm gửi yêu cầu.
resolved_at	TIMESTAMP		Thời điểm xử lý.

Bảng 3.18: Đặc tả cấu trúc bảng rooms

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh phòng.

name	VARCHAR		Tên phòng.
type	VARCHAR		Loại phòng dùng cho hiển thị icon.
house_id	UUID	FK	Ngôi nhà chứa phòng.

Bảng 3.19: Đặc tả cấu trúc bảng nodes

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh node phần cứng.
name	VARCHAR		Tên gọi nhớ của node.
hardware_id	VARCHAR	Unique	Mã phần cứng duy nhất của ESP32.
status	VARCHAR		online hoặc offline.
firmware_version	VARCHAR		Phiên bản firmware đang chạy.
last_seen_at	TIMESTAMP		Thời điểm nhận tín hiệu gần nhất.
voltage	DOUBLE		Điện áp gần nhất.
current	DOUBLE		Dòng điện gần nhất.
power	DOUBLE		Công suất gần nhất.
house_id	UUID	FK	Nhà chứa node.
room_id	UUID	FK	Phòng đặt node.

Bảng 3.20: Đặc tả cấu trúc bảng devices

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh thiết bị relay.
name	VARCHAR		Tên thiết bị phụ tải.

type	VARCHAR		Loại thiết bị.
pin	INT		Chân GPIO điều khiển relay trên ESP32.
last_status	VARCHAR		Trạng thái on/off gần nhất.
node_id	UUID	FK	Node ESP32 quản lý thiết bị.
room_id	UUID	FK	Phòng đặt thiết bị.
updated_at	TIMESTAMP		Thời điểm cập nhật trạng thái gần nhất.

Bảng 3.21: Đặc tả cấu trúc bảng logs

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh bản ghi nhật ký.
house_id	UUID	FK	Nhà xảy ra sự kiện.
node_id	UUID	FK	Node liên quan.
device_id	UUID	FK	Thiết bị liên quan.
event_type	VARCHAR		Loại sự kiện như device_toggle, electrical_alert.
action	VARCHAR		Hành động chi tiết.
value	JSONB		Dữ liệu JSON chứa thông số hoặc chi tiết sự cố.
status_before	VARCHAR		Trạng thái trước sự kiện.
status_after	VARCHAR		Trạng thái sau sự kiện.
source	VARCHAR		Nguồn kích hoạt: api, device hoặc scheduler.
actor_id	VARCHAR		Người dùng hoặc tác nhân kích hoạt.

cmd_id	UUID		Mã lệnh để đối chiếu ACK.
created_at	TIMESTAMP		Thời điểm xảy ra sự kiện.

Bảng 3.22: Đặc tả cấu trúc bảng hourly_energy

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh bản ghi năng lượng giờ.
house_id	UUID	FK	Nhà tương ứng.
room_id	UUID	FK	Phòng tương ứng.
device_id	UUID	FK	Thiết bị tiêu thụ điện.
timestamp	TIMESTAMP		Thời điểm bắt đầu khung giờ.
kwh	DOUBLE		Điện năng tiêu thụ trong khung giờ.

Bảng 3.23: Đặc tả cấu trúc bảng daily_energy

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh bản ghi năng lượng ngày.
house_id	UUID	FK	Nhà tương ứng.
room_id	UUID	FK	Phòng tương ứng.
device_id	UUID	FK	Thiết bị tiêu thụ điện.
date	DATE		Ngày ghi nhận dữ liệu.
kwh	DOUBLE		Tổng điện năng trong ngày.

Bảng 3.24: Đặc tả cấu trúc bảng monthly_energy

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh bản ghi năng lượng tháng.
house_id	UUID	FK	Nhà tương ứng.
room_id	UUID	FK	Phòng tương ứng.
device_id	UUID	FK	Thiết bị tiêu thụ điện.
month	DATE		Tháng ghi nhận, thường lấy ngày đầu tháng.
kwh	DOUBLE		Tổng điện năng trong tháng.

Bảng 3.25: Đặc tả cấu trúc bảng *device_schedules*

Tên trường	Kiểu dữ liệu	Khóa	Mô tả
id	UUID	PK	Định danh lịch hẹn.
device_id	UUID	FK	Thiết bị điều khiển.
type	VARCHAR		timer hoặc schedule.
action	VARCHAR		Hành động on hoặc off.
time	VARCHAR		Giờ kích hoạt HH:mm với lịch lặp.
days	VARCHAR		Danh sách thứ trong tuần.
execute_at	TIMESTAMP		Thời điểm thực thi cụ thể với timer.
is_active	BOOLEAN		Trạng thái kích hoạt của lịch.
created_at	TIMESTAMP		Thời điểm tạo lịch.
updated_at	TIMESTAMP		Thời điểm cập nhật lịch.

Các bảng đặc tả từ Bảng 3.13 đến Bảng 3.25 cho thấy nhiều trường dữ liệu gắn trực tiếp với giải thuật. Ví dụ pin trong devices là cơ sở để backend gửi đúng GPIO

xuống ESP32; cmd_id trong logs dùng để đối chiếu ACK; last_seen_at trong nodes dùng để phát hiện offline; hourly_energy, daily_energy và monthly_energy giúp dashboard truy vấn thống kê mà không cần quét toàn bộ nhật ký thô.

3.5 THIẾT KẾ GIAO DIỆN API VÀ GIAO THỨC TRUYỀN THÔNG

3.5.1 Thiết kế các giao diện lập trình REST API

REST API là giao kèo giữa frontend Next.js và backend NestJS cho các tác vụ có yêu cầu xác thực và phản hồi rõ ràng. Bảng 3.26 liệt kê các API cốt lõi được dùng trong hệ thống.

Bảng 3.26: Đặc tả các REST API cốt lõi trong hệ thống

Phương thức	Đường dẫn API	Xác thực	Chức năng
POST	/api/auth/register	Public	Đăng ký tài khoản người dùng mới.
POST	/api/auth/login	Public	Đăng nhập và nhận JWT access/refresh token.
GET	/api/auth/google	Public	Đăng nhập bằng Google OAuth 2.0.
PATCH	/api/auth/profile	JWT	Cập nhật thông tin cá nhân.
POST	/api/houses	JWT	Tạo mới một ngôi nhà.
POST	/api/houses/join/:id/request	JWT	Gửi yêu cầu xin gia nhập nhà.
POST	/api/houses/:houseId/join-requests/:requestId/approve	JWT + owner	Phê duyệt yêu cầu tham gia nhà.
PATCH	/api/devices/:id/toggle	JWT + member	Bật hoặc tắt relay thiết bị.
POST	/api/devices/:id/schedules	JWT + quyền	Tạo lịch hẹn giờ

		quản lý	hoặc lịch tự động.
GET	/api/houses/:houseId/logs	JWT + member	Truy vấn lịch sử nhật ký có phân trang.
GET	/api/houses/:houseId/power-stats	JWT + member	Lấy chỉ số điện năng tiêu thụ đã gộp.

Bảng 3.26 cho thấy REST API đảm nhận các thao tác cần kiểm soát quyền. Đặc biệt, lệnh bật/tắt không được gửi trực tiếp từ frontend xuống MQTT mà phải qua endpoint toggle. Backend kiểm tra người dùng có thuộc nhà đó không, có quyền điều khiển thiết bị không, rồi mới publish cmd tới ESP32. Điều này tránh việc người dùng giả mạo bản tin điều khiển từ trình duyệt.

3.5.2 Thiết kế các sự kiện truyền thông thời gian thực WebSocket

Kênh WebSocket chạy trên namespace realtime và chia room theo houseId. Mỗi client sau khi đăng nhập chỉ được join vào room của nhà mà tài khoản có quyền truy cập. Bảng 3.27 đặc tả các sự kiện chính.

Bảng 3.27: Đặc tả các sự kiện WebSocket trong hệ thống

Tên sự kiện	Namespace/Room	Chiều	Mô tả
join_house	/realtime	Client -> Server	Yêu cầu tham gia room của một nhà.
device_status_changed	house:{houseId}	Server -> Client	Đồng bộ trạng thái bật/tắt mới của relay.
node_status_changed	house:{houseId}	Server -> Client	Cập nhật trạng thái kết nối và thông số đo của node.
devices_provisioned	house:{houseId}	Server -> Client	Thông báo khi thiết bị mới được

			cấu hình.
member_kicked	house:{houseId}	Server -> Client	Thông báo loại thành viên khỏi nhà.
permission_requested	house:{houseId}	Server -> Client	Thông báo có yêu cầu cấp quyền mới.
permission_resolved	house:{houseId}	Server -> Client	Đồng bộ quyền mới sau khi phê duyệt.
new_log	house:{houseId}	Server -> Client	Đẩy log sự kiện mới lên màn hình nhật ký.

Bảng 3.27 được thiết kế để dashboard không cần tải lại trang. Khi node gửi state mới, backend phát node_status_changed. Khi relay đổi trạng thái do dashboard hoặc nút nhấn vật lý, backend phát device_status_changed. Khi có log mới hoặc cảnh báo, backend phát new_log. Cách tổ chức theo room giúp cô lập dữ liệu giữa các nhà khác nhau.

3.5.3 Thiết kế cấu trúc các bản tin giao tiếp MQTT

Giao tiếp giữa backend và ESP32 được thực hiện qua HiveMQ Cloud Broker. Topic được tổ chức phân cấp theo dạng v1/homes/{houseId}/nodes/{hardwareId}/..., giúp backend xác định bản tin thuộc nhà nào và node vật lý nào. Bảng 3.28 trình bày các topic theo firmware cuối.

Bảng 3.28: Đặc tả các topic MQTT trong hệ thống

Topic	Phương thức	Chiều truyền	Mô tả
v1/homes/{houseId}/nodes/{hardwareId}/state	Publish	ESP32 -> Backend	Báo cáo dữ liệu đo, trạng thái relay, heartbeat và

			cảnh báo trong trường alert.
v1/homes/{houseId}/nodes/{hardwareId}/cmd	Subscribe	Backend -> ESP32	ESP32 lắng nghe lệnh điều khiển relay từ backend.
v1/homes/{houseId}/nodes/{hardwareId}/ack	Publish	ESP32 -> Backend	ESP32 phản hồi kết quả thực thi lệnh điều khiển.
Last Will trên topic state	Broker publish	Broker -> Backend	Broker phát {"status":"offline"} khi ESP32 mất kết nối bất thường.

Bản tin state gồm node_id, hardware_id, firmware_version, timestamp, voltage, current, power, energy_kwh, devices[] và alert. Mỗi phần tử devices[] chứa thông tin pin, status, type, name và power_consumption_ma. Trường alert nhận giá trị rỗng khi bình thường hoặc chứa mã sự cố như overcurrent, overvoltage, undervoltage hay no_load tùy tình huống firmware phát hiện. Cách viết này đã chỉnh lại so với mô tả dùng topic alert riêng trong một số bản thảo cũ.

Bản tin cmd do backend gửi xuống chứa cmd_id, action, device_id, pin và value. Trong firmware hiện tại, action chính là toggle và pin giúp ESP32 xác định relay cần điều khiển. Bản tin ack do ESP32 gửi lên chứa cmd_id, device_id, result, actual_value, timestamp và các thông số đo tại thời điểm phản hồi. Backend dùng cmd_id để đối chiếu lệnh và kết thúc bộ đếm timeout.

3.6 THIẾT KẾ CÁC GIẢI THUẬT

3.6.1 Giải thuật kết nối WiFi không chặn

Rủi ro vận hành của thiết bị IoT là mất WiFi hoặc cấu hình sai làm thiết bị không lên cloud. Nếu firmware dùng vòng lặp chờ kết nối quá lâu, thiết bị có thể ngừng đọc

nút nhấn, ngừng đọc PZEM và chậm xử lý sự cố điện. Vì vậy, firmware được thiết kế theo hướng tách cấu hình ban đầu và reconnect khi đang chạy.

Khi chưa có cấu hình WiFi hoặc houseId, ESP32 mở AP SmartSocket-Setup tại địa chỉ 192.168.4.1. Người dùng truy cập trang cấu hình, nhập SSID, mật khẩu và houseId; dữ liệu được lưu vào Preferences. Khi đã có cấu hình, thiết bị kết nối WiFi, đồng bộ thời gian NTP và kết nối MQTT. Trong vòng lặp chính, firmware kiểm tra trạng thái mạng theo thời gian để thử reconnect, đồng thời vẫn duy trì các tác vụ cục bộ. Điểm cần ghi đúng là giai đoạn khởi động ban đầu vẫn có khoảng chờ kết nối giới hạn, nên không mô tả toàn bộ quá trình là non-blocking tuyệt đối.

3.6.2 Giải thuật lưu trữ đệm ngoại tuyến và đồng bộ (LittleFS Local Buffering)

Khi Internet hoặc MQTT Broker bị gián đoạn, dữ liệu đo từ PZEM-004T không thể gửi ngay về backend. Nếu bỏ qua các mẫu đo này, biểu đồ điện năng sẽ bị mất đoạn và thống kê có thể sai. Firmware dùng LittleFS để ghi tạm dữ liệu vào flash nội bộ dưới dạng file /offline.csv.

Thuật toán gồm các bước: ESP32 đọc PZEM theo chu kỳ; nếu MQTT đang kết nối thì publish bình thường; nếu MQTT mất kết nối thì mở file /offline.csv ở chế độ ghi nối tiếp và lưu bản ghi rút gọn gồm thời gian, điện áp, dòng điện, công suất và điện năng. Khi MQTT kết nối lại, ESP32 đọc lần lượt các dòng đã lưu, chuyển thành bản tin JSON và publish lên broker, sau đó xóa file đệm. LittleFS phù hợp hơn RAM vì dữ liệu vẫn còn sau khi reset ngắn, nhưng nó chỉ nên xem là bộ đệm tạm thời do flash có giới hạn dung lượng và chu kỳ ghi [23].

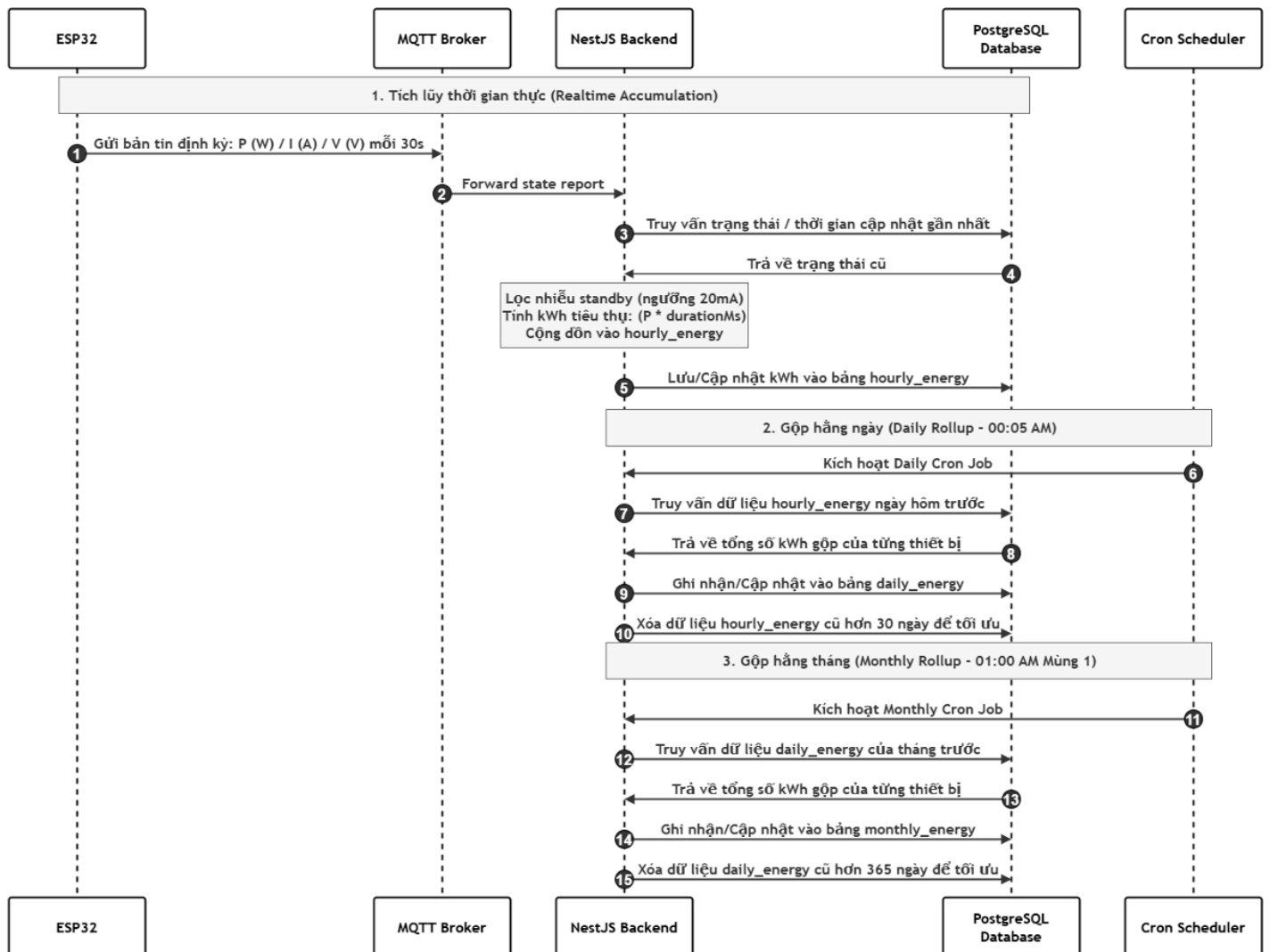
3.6.3 Giải thuật lọc nhiễu và tiết kiệm băng thông (Delta-Threshold Reporting)

Nếu ESP32 publish dữ liệu sau mỗi lần đọc PZEM, hệ thống sẽ tạo rất nhiều bản tin gần giống nhau khi tải ổn định. Điều này làm tăng băng thông MQTT và làm bảng dữ liệu phình nhanh. Vì vậy firmware áp dụng bộ lọc delta-threshold ngay tại ESP32.

Firmware lưu ba giá trị đã publish gần nhất: điện áp, dòng điện và công suất. Ở mỗi chu kỳ đọc, thiết bị tính độ lệch tuyệt đối so với giá trị đã gửi. Bản tin mới chỉ được publish khi $|V_{now} - V_{last}| > 4 \text{ V}$, $|I_{now} - I_{last}| > 0,05 \text{ A}$ hoặc $|P_{now} - P_{last}| > 2 \text{ W}$. Nếu tải ổn định và các ngưỡng không vượt, thiết bị vẫn gửi heartbeat theo chu kỳ để backend biết node còn hoạt động. Thiết kế này giữ được biến động thật của tải nhưng lọc bỏ dao động nhỏ do nhiễu đo hoặc dao động lưới.

3.6.4 Giải thuật tích lũy điện năng và gộp dữ liệu 3 cấp (3-Tier Energy Aggregation)

Giải thuật gộp dữ liệu điện năng thuộc backend. Nếu mọi biểu đồ đều truy vấn trực tiếp từ log thô, số bản ghi sẽ tăng nhanh theo thời gian và truy vấn tháng/năm trở nên nặng. Vì vậy backend tách dữ liệu điện năng thành ba cấp: `hourly_energy`, `daily_energy` và `monthly_energy`. Hình 3.12 mô tả luồng gộp dữ liệu này.



Hình 3.12: Luồng tích lũy và gộp dữ liệu điện năng theo giờ, ngày, tháng

Ở cấp thứ nhất, khi backend nhận bản tin state hoặc sự kiện relay, hệ thống cập nhật điện năng theo khung giờ hiện tại. Trong cấp thứ hai, cron job hằng ngày tính tổng dữ liệu giờ của ngày trước và ghi vào `daily_energy`. Ở cấp thứ ba, cron job hằng tháng tính tổng dữ liệu ngày của tháng trước và ghi vào `monthly_energy`. Cách này giúp dashboard truy vấn biểu đồ nhanh hơn, đồng thời vẫn giữ được dữ liệu tổng hợp dài hạn để theo dõi xu hướng tiêu thụ.

Ở cấp tích lũy theo giờ, khi backend nhận bản tin state từ MQTT hoặc nhận sự kiện relay thay đổi trạng thái, hệ thống không chỉ lưu giá trị đo tức thời mà còn tính phần điện năng phát sinh giữa hai lần cập nhật liên tiếp. Điện năng được tính theo

quan hệ $kWh = P \times dt / 3.600.000$, trong đó P là công suất tại khoảng đo và dt là khoảng thời gian tính bằng mili giây so với lần cập nhật gần nhất. Giá trị này được cộng dồn vào bản ghi `hourly_energy` tương ứng với thiết bị và khung giờ hiện tại, giúp dữ liệu biểu đồ không phụ thuộc vào việc truy vấn lại toàn bộ log thô.

Để giảm sai số tích lũy ở trạng thái không tải, hệ thống bổ sung cơ chế lọc nhiễu standby hai lớp. Ở phía firmware, các giá trị công suất đo được nhỏ hơn 5 W được xem là vùng chết và ép về 0 W trước khi gửi lên broker. Ở phía backend, quá trình tích lũy tiếp tục kiểm tra ngưỡng dòng điện tối thiểu 20 mA; các mẫu có dòng nhỏ hơn ngưỡng này không được dùng để cộng dồn năng lượng. Khi chưa có log trước đó để tính khoảng thời gian dt , backend dùng giá trị fallback bằng 0 để tránh cộng nhầm điện năng cho khoảng thời gian không xác định.

Bên cạnh phần tính toán, hàm lấy dữ liệu thống kê `getPowerStats()` cũng được xử lý lại ở bước định dạng thời gian. Trước khi trả dữ liệu cho dashboard, phút và giây của mốc thời gian hiển thị được làm tròn về 0, nhờ đó trục hoành của biểu đồ ngày hiển thị theo các mốc giờ chẵn. Cách xử lý này không làm thay đổi dữ liệu tích lũy, nhưng giúp biểu đồ dễ đọc hơn và tránh cảm giác các mốc thời gian bị lệch do thời điểm cron hoặc thời điểm nhận MQTT không trùng chính xác đầu giờ.

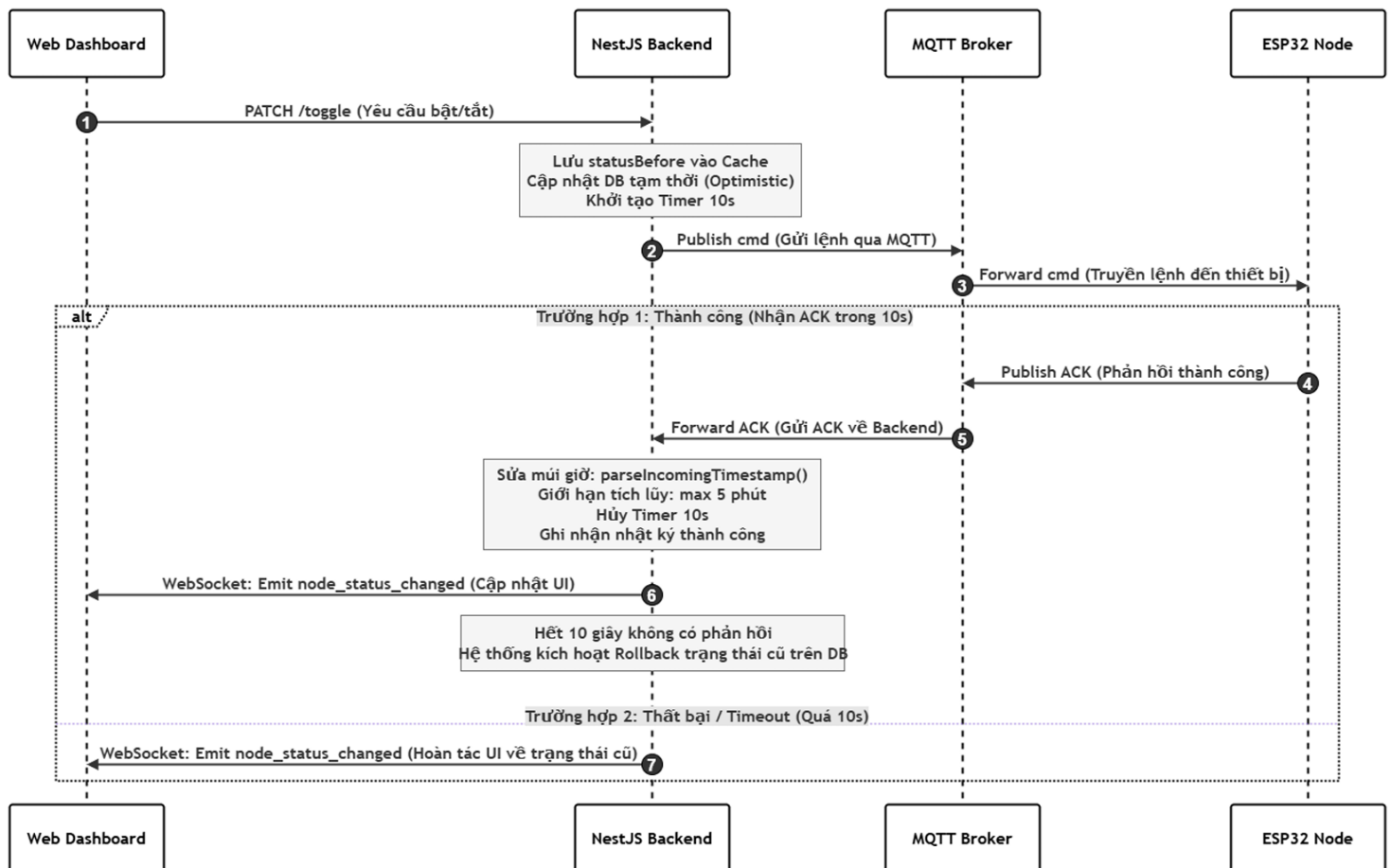
Daily Cron Job được kích hoạt hằng ngày vào khoảng 00:05 để tổng hợp dữ liệu của ngày liền trước. Tiến trình này cộng toàn bộ kWh trong bảng `hourly_energy` theo từng thiết bị, ghi hoặc cập nhật kết quả vào `daily_energy`, sau đó dọn các bản ghi `hourly_energy` cũ hơn 30 ngày. Monthly Cron Job được kích hoạt vào 01:00 ngày đầu mỗi tháng, tổng hợp dữ liệu từ `daily_energy` của tháng trước vào `monthly_energy` và xóa các bản ghi `daily_energy` cũ hơn 365 ngày. Nhờ vậy hệ thống vừa giữ được dữ liệu dài hạn ở cấp tháng, vừa hạn chế dung lượng lưu trữ tăng không kiểm soát.

3.6.5 Giải thuật điều khiển chịu lỗi Command Timeout và Rollback

Điều khiển từ xa có rủi ro: backend đã gửi lệnh nhưng ESP32 mất mạng, broker không chuyển được lệnh hoặc relay không đổi trạng thái. Nếu dashboard cập nhật

trạng thái ngay mà không xác nhận, người dùng có thể thấy thiết bị đã bật trong khi relay vật lý chưa đóng. Vì vậy hệ thống dùng cơ chế cmd_id, ACK, timeout và rollback như Hình 3.13.

Khi người dùng gửi lệnh bật/tắt, backend lưu trạng thái trước đó, tạo cmd_id và publish lệnh xuống topic cmd. ESP32 nhận lệnh, điều khiển relay, sau đó publish ACK với cùng cmd_id và actual_value. Nếu ACK đến trước thời gian chờ, backend ghi nhận thành công, cập nhật log và phát WebSocket. Nếu timeout, backend hoàn nguyên trạng thái trong CSDL/giao diện về statusBefore và ghi log lỗi. Rollback là nghiệp vụ phía backend; firmware ESP32 chỉ thực thi relay và gửi ACK.



Hình 3.13: Sơ đồ luồng thời gian thực thi lệnh và giải thuật khôi phục (Rollback)

Trong Hình 3.13 quá trình xử lý lệnh và tính toán điện năng, backend cần bảo đảm mốc thời gian của bản tin từ ESP32 không gây sai lệch thống kê. Vì vậy hệ thống

bổ sung hàm `parseIncomingTimestamp()` để chuẩn hóa timestamp đầu vào. Hàm này xử lý trường hợp một số bản build firmware cũ gửi thời gian GMT+7 nhưng lại gắn ký tự Z, làm backend hiểu nhầm là thời gian UTC. Khi phát hiện dạng sai lệch này, backend hiệu chỉnh lại mốc thời gian cho phù hợp với múi giờ vận hành. Nếu timestamp nhận được lệch quá xa so với hiện tại, đặc biệt lệch quá 5 phút ở tương lai, hệ thống fallback về thời gian máy chủ để tránh ghi log và tính năng lượng theo mốc sai.

Ngoài việc chuẩn hóa thời gian, backend áp dụng giới hạn `MAX_ENERGY_DURATION_MS = 5` phút cho khoảng thời gian tích lũy. Giới hạn này cần thiết trong trường hợp thiết bị mất mạng lâu, sau đó gửi lại bản tin mới khiến khoảng cách giữa hai mẫu đo trở nên quá lớn. Nếu cộng dồn trực tiếp toàn bộ khoảng thời gian mất liên lạc, hệ thống có thể tạo ra lượng điện năng ảo không phản ánh đúng thực tế. Chặn thời gian tích lũy tối đa giúp bảo vệ số liệu khỏi các khoảng ngắt kết nối bất thường.

Đối với điều khiển relay, backend dùng cơ chế cập nhật lạc quan nhưng luôn kèm bộ đếm timeout. Sau khi nhận yêu cầu từ REST API, trạng thái trước lệnh được lưu vào `statusBefore`, trạng thái mong muốn được cập nhật tạm trong cơ sở dữ liệu và lệnh MQTT chứa `cmd_id` được gửi xuống ESP32. Nếu ESP32 phản hồi ACK trong vòng 10 giây, backend xác nhận lệnh thành công, hủy timer, ghi log và phát WebSocket. Nếu quá thời gian này không có ACK khớp `cmd_id`, backend khôi phục trạng thái về `statusBefore`, ghi nhận lỗi timeout và phát sự kiện WebSocket để dashboard trả nút điều khiển về vị trí cũ.

3.6.6 Giải thuật quét Heartbeat phát hiện Node ngoại tuyến (Offline Detection)

Thiết bị có thể offline do mất nguồn, mất WiFi, broker gián đoạn hoặc thiết bị bị reset. Hệ thống dùng hai lớp phát hiện. Lớp thứ nhất là MQTT Last Will: khi ESP32 kết nối broker, nó đăng ký bản tin offline trên topic `state`; nếu kết nối TCP đứt bất

thường, broker tự publish bản tin này. Lớp thứ hai là backend quét `last_seen_at` trong bảng nodes để phát hiện node không gửi heartbeat/state trong thời gian dài.

Backend định kỳ so sánh thời điểm hiện tại với `last_seen_at`. Nếu node đang online nhưng đã quá ngưỡng mà không có bản tin mới, backend cập nhật status sang offline, ghi log và phát `node_status_changed` xuống dashboard. Khi thiết lập thực tế, ngưỡng offline phải lớn hơn chu kỳ heartbeat của firmware để tránh báo offline sai khi tải ổn định và không có bản tin delta.

Tiến trình phát hiện node ngoại tuyến được triển khai theo chu kỳ tại backend. Cứ mỗi 30 giây, scheduler tính mốc giới hạn $T_{cutoff} = T_{now} - 90$ giây, sau đó quét bảng nodes để tìm các node đang có trạng thái online nhưng `last_seen_at` nhỏ hơn mốc giới hạn này. Điều kiện 90 giây được chọn lớn hơn chu kỳ gửi heartbeat/state của firmware để giảm nguy cơ báo offline nhầm trong giai đoạn tải ổn định hoặc mạng có dao động ngắn.

Với mỗi node thỏa điều kiện quá hạn, backend cập nhật trạng thái trong cơ sở dữ liệu sang offline, ghi một bản ghi log để phục vụ truy vết và phát sự kiện `node_status_changed` qua WebSocket đến các client đang theo dõi cùng ngôi nhà. Cách làm này bổ sung cho MQTT Last Will: Last Will xử lý trường hợp broker phát hiện kết nối TCP bị ngắt bất thường, còn heartbeat scanner xử lý các tình huống thiết bị mất nguồn, treo firmware hoặc không còn gửi bản tin định kỳ nhưng trạng thái trên broker chưa phản ánh đầy đủ.

3.6.7 Giải thuật cảnh báo sự cố điện an toàn và gửi Email báo động khẩn cấp

Các sự cố điện như quá dòng, quá áp hoặc sụt áp cần được xử lý tại thiết bị trước khi phụ thuộc vào cloud. Firmware đọc PZEM và kiểm tra ngưỡng an toàn. Nếu dòng điện vượt 15 A kéo dài hơn 3 giây, ESP32 ngắt cả hai relay để cách ly tải. Nếu điện áp vượt 250 V hoặc thấp hơn 160 V, ESP32 cũng ngắt relay để bảo vệ thiết bị. Nếu relay đang bật nhưng dòng nhỏ hơn 0,02 A trong một khoảng thời gian, firmware phát cảnh báo hờ tải để người dùng kiểm tra phụ tải hoặc kết nối.

Sau khi xử lý tại chỗ, ESP32 gửi mã cảnh báo trong trường alert của bản tin state. Backend nhận bản tin này, ghi log `electrical_alert`, xác định nhà và danh sách thành viên liên quan, sau đó gửi email cảnh báo nếu chức năng email được cấu hình. Thiết kế hai lớp này hợp lý vì thao tác ngắt relay phải nhanh và độc lập cloud, còn gửi email cần thông tin người dùng và địa chỉ nhận trong cơ sở dữ liệu.

Ở phía backend, bản tin cảnh báo từ ESP32 được xử lý thêm một lớp chống spam. Khi nhận alert từ MQTT, hệ thống kiểm tra cache `lastAlertPerNode` theo từng node và loại cảnh báo. Nếu cảnh báo cùng loại đã được gửi trong thời gian cooldown, backend bỏ qua lần gửi mới để tránh tạo quá nhiều email và log trùng lặp. Chu kỳ cooldown được đặt là `ALERT_COOLDOWN_MS = 5` phút, đủ để người dùng nhận biết sự cố nhưng không làm nghẽn hộp thư khi cảm biến liên tục phát cùng một lỗi.

Riêng cảnh báo hở tải hoặc undercurrent có khả năng xuất hiện lặp lại trong một số tình huống như chưa cắm phụ tải, tải công suất quá nhỏ hoặc nhiều dòng gần ngưỡng. Vì vậy firmware cho phép tắt nhóm cảnh báo này bằng cờ cấu hình `ENABLE_UNDERCURRENT_ALERT` trong `dashboard_config.h`. Khi cờ được tắt, thiết bị vẫn duy trì các lớp bảo vệ nghiêm trọng như quá dòng, quá áp và sụt áp, nhưng không phát cảnh báo hở tải liên tục gây nhiều trải nghiệm vận hành.

Sau khi xác nhận cảnh báo hợp lệ, backend truy vấn danh sách thành viên thuộc ngôi nhà chứa node xảy ra sự cố, lấy địa chỉ email tương ứng và gọi `EmailService` để gửi thư cảnh báo dạng HTML. Nội dung thư cần thể hiện rõ tên nhà, tên thiết bị hoặc node, loại sự cố và các trị số đo tại thời điểm phát hiện như điện áp, dòng điện, công suất. Nhờ tách nhiệm vụ như vậy, ESP32 tập trung xử lý ngắt relay nhanh tại chỗ, còn backend đảm nhận phần thông báo người dùng và lưu vết sự kiện.

3.6.8 Giải thuật hẹn giờ và lên lịch tự động

Hẹn giờ và lên lịch tự động phục vụ các nhu cầu như tự tắt sạc sau một khoảng thời gian, bật thiết bị trước một thời điểm hoặc lặp lại lịch theo ngày trong tuần. Trong hệ thống này, lịch được lưu và xử lý ở backend thay vì lưu trực tiếp trong ESP32. Khi

người dùng tạo lịch, backend ghi bản ghi vào device_schedules với type, action, time, days, execute_at và is_active.

Cron job của backend chạy định kỳ, lấy thời gian hiện tại theo múi giờ UTC+7 và so khớp với các lịch đang active. Với type timer, backend kiểm tra execute_at đã tới hạn hay chưa; nếu tới hạn thì gửi lệnh MQTT xuống ESP32, vô hiệu hóa lịch và ghi log nguồn scheduler. Với type schedule, backend kiểm tra time và days để phát lệnh on/off theo lịch lặp. Mọi lệnh do lịch tạo ra vẫn đi qua cùng luồng cmd/ack/rollback như thao tác thủ công, nhờ đó trạng thái relay, CSDL và dashboard luôn được đồng bộ.

Cron job hẹn giờ và lập lịch được kích hoạt mỗi 1 phút, tại giây thứ 00 của phút hiện tại. Khi chạy, backend lấy thời gian hệ thống, quy đổi theo múi giờ UTC+7 và tách thành hai thành phần chính: chuỗi giờ phút ở dạng HH:mm và chỉ mục ngày trong tuần. Chỉ mục ngày dùng quy ước JavaScript từ 0 đến 6, trong đó 0 là Chủ Nhật, 1 là Thứ Hai và tiếp tục đến 6 là Thứ Bảy. Quy ước này giúp tránh lỗi lệch lịch khi frontend, backend và cơ sở dữ liệu cùng xử lý mảng days.

Với lịch loại timer, backend truy vấn các bản ghi có type = 'timer', is_active = true và execute_at nhỏ hơn hoặc bằng thời điểm hiện tại. Mỗi lịch thỏa điều kiện sẽ tạo một lệnh MQTT bật hoặc tắt relay tương ứng, sau đó lịch được cập nhật is_active = false để bảo đảm chỉ chạy một lần. Hệ thống đồng thời ghi log với nguồn tác nhân scheduler và phát WebSocket để dashboard cập nhật trạng thái mới.

Với lịch loại schedule, backend truy vấn các bản ghi có type = 'schedule', is_active = true và time trùng với HH:mm hiện tại. Nếu mảng days rỗng, lịch được hiểu là lịch một lần theo thời điểm đã đặt: backend thực thi lệnh rồi tắt is_active. Nếu days chứa chỉ mục ngày hiện tại, backend thực thi lệnh nhưng giữ is_active = true để lịch tiếp tục lặp lại vào tuần sau. Nếu ngày hiện tại không nằm trong days, lịch bị bỏ qua trong chu kỳ đó. Tất cả lệnh sinh ra từ timer hoặc schedule vẫn đi qua cơ chế cmd_id, ACK và rollback giống lệnh điều khiển thủ công.

3.6.9 Công cụ dọn dẹp dữ liệu lỗi (Database Reset Tooling)

Trong quá trình thử nghiệm và hiệu chỉnh hệ thống, cơ sở dữ liệu có thể phát sinh các bản ghi không còn phản ánh đúng trạng thái vận hành thực tế. Các lỗi thường gặp gồm log cảnh báo hở tải lặp lại do ngưỡng đo chưa ổn định, dữ liệu điện năng bị cộng sai do lệch múi giờ hoặc các bản ghi thử nghiệm được tạo trước khi thuật toán lọc nhiễu hoàn thiện. Nếu các dữ liệu này tiếp tục được giữ lại, biểu đồ thống kê điện năng và phần ước tính chi phí trên dashboard có thể bị sai lệch.

Để xử lý tình huống trên, hệ thống bổ sung công cụ dọn dẹp dữ liệu lỗi dưới dạng một kịch bản Node.js chạy độc lập với backend chính. Công cụ này không phục vụ người dùng cuối mà dùng trong giai đoạn bảo trì, kiểm thử hoặc trước khi bàn giao hệ thống. Mục tiêu của công cụ là đưa các bảng tích lũy điện năng và nhóm log spam về trạng thái sạch, đồng thời vẫn giữ nguyên các bảng nghiệp vụ quan trọng như users, houses, rooms, nodes và devices.

Thuật toán của công cụ gồm ba bước. Bước thứ nhất, chương trình đọc chuỗi kết nối cơ sở dữ liệu từ tệp môi trường .env hoặc biến DATABASE_URL. Bước thứ hai, công cụ phân tích chuỗi kết nối; nếu phát hiện cơ sở dữ liệu được triển khai trên các máy chủ ngoại vi như Render, Supabase hoặc Neon, chương trình tự động bật cấu hình kết nối SSL với tùy chọn rejectUnauthorized: false để phù hợp với môi trường cloud và tránh lỗi reset kết nối. Bước thứ ba, công cụ thực thi các truy vấn SQL dọn dẹp dữ liệu trong hourly_energy, daily_energy, monthly_energy và xóa các log cảnh báo hở tải undercurrent trong bảng logs.

Tách công cụ này khỏi luồng vận hành chính giúp giảm rủi ro thao tác nhầm từ giao diện người dùng. Khi cần làm sạch dữ liệu, người vận hành chủ động chạy kịch bản trong môi trường quản trị, kiểm tra log thực thi và chỉ áp dụng cho các nhóm bảng đã được xác định trước. Cách tiếp cận này phù hợp với hệ thống đang phát triển vì cho phép sửa dữ liệu thử nghiệm nhanh, nhưng vẫn giữ được ranh giới an toàn giữa chức năng vận hành hằng ngày và thao tác bảo trì cơ sở dữ liệu.

CHƯƠNG 4

THI CÔNG HỆ THỐNG

4.1 GIỚI THIỆU

Tiếp nối phần thiết kế phần cứng đã trình bày ở Chương 3, chương này trình bày quá trình thi công thực tế của hệ thống, bao gồm việc thiết kế và chế tạo bo mạch in, hàn lắp linh kiện cùng các module và đóng gói sản phẩm vào vỏ hộp hoàn chỉnh. Mục tiêu của khâu thi công là hiện thực hóa sơ đồ nguyên lý thành một thiết bị hoạt động được, gọn gàng và bảo đảm an toàn điện cho người sử dụng.

4.2 THI CÔNG PHẦN CỨNG

Quá trình thi công phần cứng được thực hiện qua ba bước chính: thiết kế và chế tạo bo mạch in từ sơ đồ nguyên lý, hàn lắp linh kiện và đấu nối các module, cuối cùng là đóng gói toàn bộ vào vỏ hộp. Các linh kiện và module chính sử dụng trong mạch được liệt kê ở Bảng 4.1.

Bảng 4.1: Danh sách linh kiện và module chính của mạch

Ký hiệu	Linh kiện / Module	Chức năng
U2	ESP32-WROOM-32	Vi điều khiển trung tâm, tích hợp WiFi
U3	Module PZEM-004T (100A, biến dòng CT)	Đo điện áp, dòng điện, công suất, điện năng
U4	Module relay 2 kênh (SRD-05VDC-SL-C)	Đóng/ngắt nguồn 220VAC cho hai ổ cắm
U5	Module nguồn HLK-PM01B	Chuyển 220VAC sang 5VDC
AMS1	IC AMS1117-3.3V	Hạ 5V xuống 3,3V cấp cho ESP32

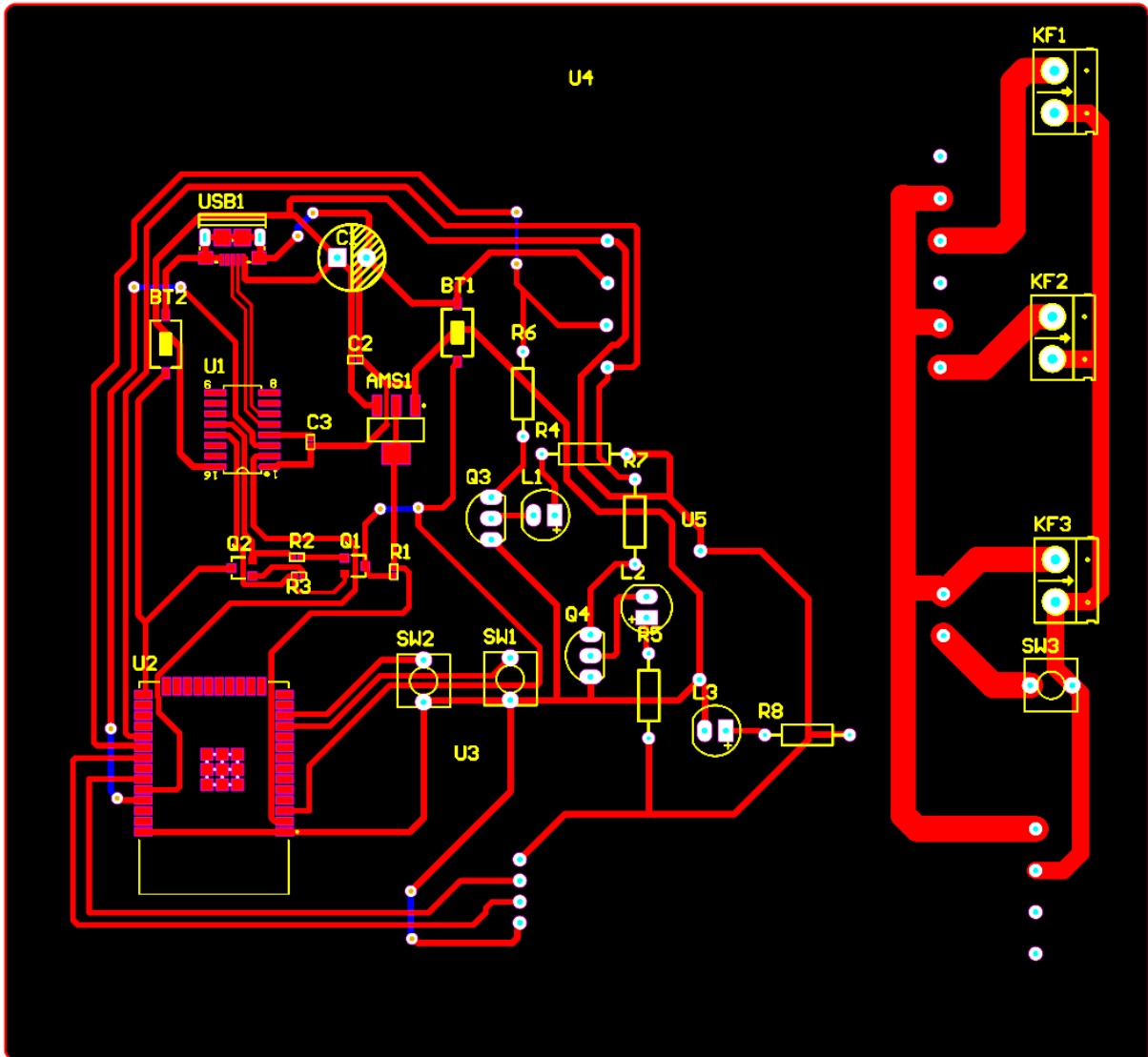
U1	IC CH340C	Chuyển USB sang UART để nạp firmware
Q1, Q2	Transistor 2N2222	Mạch tự động reset/nạp
Q3, Q4	Transistor C1815	Kích đèn LED báo theo tín hiệu relay
SW1, SW2	Nút nhấn	Điều khiển bật/tắt ổ điện tại chỗ
SW3	Công tắc nguồn tổng 220VAC	Đóng/ngắt nguồn 220VAC cấp cho toàn thiết bị
L1, L2, L3	LED báo	L1/L2 báo trạng thái relay, L3 báo nguồn tổng
KF1, KF2, KF3	Đầu nối (terminal)	Nối điện lưới vào và ổ cắm ra

4.2.1 Thiết kế và chế tạo bo mạch in

Từ sơ đồ nguyên lý đã phân tích ở Chương 3, bo mạch in được triển khai trên Altium Designer theo một quy trình khép kín thay vì vẽ thủ công rời rạc. Trước hết, sơ đồ nguyên lý được kiểm tra lại tên linh kiện, chân kết nối và nhãn mạng; sau đó nhóm dùng chức năng đồng bộ từ schematic sang PCB để đưa footprint, danh sách linh kiện và toàn bộ kết nối sang môi trường thiết kế mạch in. Bước tiếp theo là sắp xếp linh kiện theo dòng năng lượng và không gian cơ khí, đi dây các đường mạch, đặt luật thiết kế về bề rộng đường mạch và khoảng cách cách điện, đổ đồng cho các vùng nguồn/mass, kiểm tra mô hình 3D, xuất bộ file Gerber và Drill, cuối cùng gửi đặt gia công PCB. Bố cục bo mạch hoàn chỉnh sau quá trình này được thể hiện ở Hình 4.1.

Ưu điểm của việc đồng bộ trực tiếp từ sơ đồ nguyên lý sang PCB là hạn chế sai sót nối nhầm chân. Nếu một chân ESP32, relay, PZEM hoặc CH340C đã được định nghĩa trong schematic thì sang PCB Altium sẽ tạo đường nối tham chiếu tương ứng, người thiết kế chỉ quyết định vị trí linh kiện và cách đi dây. Nhờ vậy, quá trình thi

công bám sát thiết kế điện ở Chương 3, đồng thời để kiểm tra lại bằng DRC trước khi xuất file sản xuất.



Hình 4.1: Sơ đồ bố trí mạch in (PCB layout) thiết kế trên Altium

Một nguyên tắc quan trọng khi bố trí mạch, như thể hiện ở Hình 4.1, là tách rõ vùng điện áp cao xoay chiều và vùng điều khiển điện áp thấp. Các đầu nối điện lưới và ổ cắm KF1, KF2, KF3 cùng nút/công tắc tổng SW3 được đặt về phía phải của bo mạch, nơi các đường mạch 220VAC đi ngắn và rộng. Ngược lại, ESP32, CH340C, AMS1117, transistor, nút nhấn tín hiệu SW1/SW2 và các LED báo được đặt ở phía trái hoặc giữa bo mạch, nơi chỉ có mức 3,3V/5V. Cách phân vùng này giúp giảm nguy

cơ phóng điện, giảm nhiều từ phần công suất sang phần vi điều khiển và giúp người thi công dễ kiểm tra đường điện nguy hiểm khi lắp ráp.

Linh kiện được sắp xếp không chỉ dựa trên sơ đồ điện mà còn dựa trên kích thước thật của linh kiện khi đóng hộp. Module relay và PZEM-004T có chiều cao lớn, module nguồn HLK-PM01B cũng chiếm không gian đáng kể, trong khi ổ cắm và nút/công tắc tổng có phần thân nhô vào bên trong vỏ. Vì vậy, các linh kiện cao được gom về vùng ít vướng cơ khí, còn khu vực gần ổ cắm và các lỗ thao tác được ưu tiên cho nút nhấn, LED và mạch nạp có chiều cao thấp hơn. Nếu đặt các module cao sát vị trí ổ cắm hoặc nắp hộp, khi siết nắp có thể làm cong bo, chạm linh kiện hoặc tạo lực kéo lên dây 220VAC.

Đối với SW1 và SW2, đây là hai nút nhấn tín hiệu nối về chân GPIO của ESP32 và dùng chế độ INPUT_PULLUP, nên dòng qua nút rất nhỏ. Vì vậy đường mạch của SW1/SW2 chỉ cần đi theo chuẩn tín hiệu thấp áp, ưu tiên gọn, ngắn và tránh chạy song song sát vùng 220VAC. Riêng SW3 nằm ở vùng đầu nối công suất và đóng vai trò nút/công tắc tổng đóng/ngắt 220VAC cho toàn thiết bị, vì vậy đường mạch đi qua SW3 phải được xem như đường chịu tải, không thiết kế như đường tín hiệu GPIO. Đây là lý do SW3 được đặt cùng phía với KF1, KF2, KF3 và dùng đường đồng rộng hơn, khoảng cách tới vùng điều khiển cũng lớn hơn.

Về bề rộng đường mạch, bo được thiết kế hai lớp và chia thành hai nhóm chính. Nhóm thứ nhất là các đường công suất 220VAC cấp vào module nguồn, đi qua nút/công tắc tổng SW3, tiếp điểm relay và ra các đầu nối ổ cắm. Do relay sử dụng tiếp điểm định mức 10A/250VAC, các đường công suất được thiết kế theo dòng tải tối đa 10A nên được đi rộng khoảng 2 mm, kết hợp khoảng cách cách điện lớn giữa hai vùng điện áp. Bề rộng lớn giúp giảm điện trở đường mạch, hạn chế phát nhiệt tại các đoạn đồng và tăng độ bền cơ khí phải hàn dây công suất. Nhóm thứ hai là các đường GPIO, UART, USB, LED và nút nhấn SW1/SW2; các đường này chỉ mang dòng mức mA hoặc nhỏ hơn nên dùng bề rộng khoảng 0,7 mm để dễ đi dây, tiết kiệm diện tích và tránh chen lấn giữa các chân linh kiện nhỏ như CH340C, ESP32 và transistor.

Các đường LED báo cũng được xếp vào nhóm tín hiệu thấp áp. L1 và L2 báo trạng thái relay, còn L3 báo nguồn hệ thống; cả ba LED đều lấy điện từ nguồn 5V và dòng qua LED đã bị giới hạn bởi điện trở nối tiếp nên chỉ ở mức vài mA. Do đó đường LED không cần rộng như đường 220VAC. Tách riêng cách xử lý cho LED báo, nút nhấn tín hiệu và công tắc tổng 220VAC giúp bo mạch vừa gọn vừa đúng bản chất tải của từng đường: đường nào chịu dòng công suất thì ưu tiên bề rộng và khoảng cách, đường nào chỉ mang tín hiệu thì ưu tiên ngắn, rõ và ít nhiễu.

Các linh kiện phụ như điện trở, tụ điện và transistor không phải phân dư thừa mà là điều kiện để mạch hoạt động ổn định. Điện trở dùng để giới hạn dòng LED, giới hạn dòng vào chân base transistor, tạo điều kiện phân cực an toàn và tránh để tín hiệu điều khiển bị kéo quá dòng. Tụ điện đặt gần nguồn và IC giúp lọc nhiễu, giảm sụt áp tức thời khi ESP32 phát WiFi hoặc khi relay đóng/ngắt. Transistor C1815 được dùng để LED sáng theo tín hiệu điều khiển nhưng không bắt chân GPIO cấp trực tiếp toàn bộ dòng LED; transistor 2N2222 trong mạch nạp giúp CH340C điều khiển EN và IO0 của ESP32 tự động khi nạp chương trình.

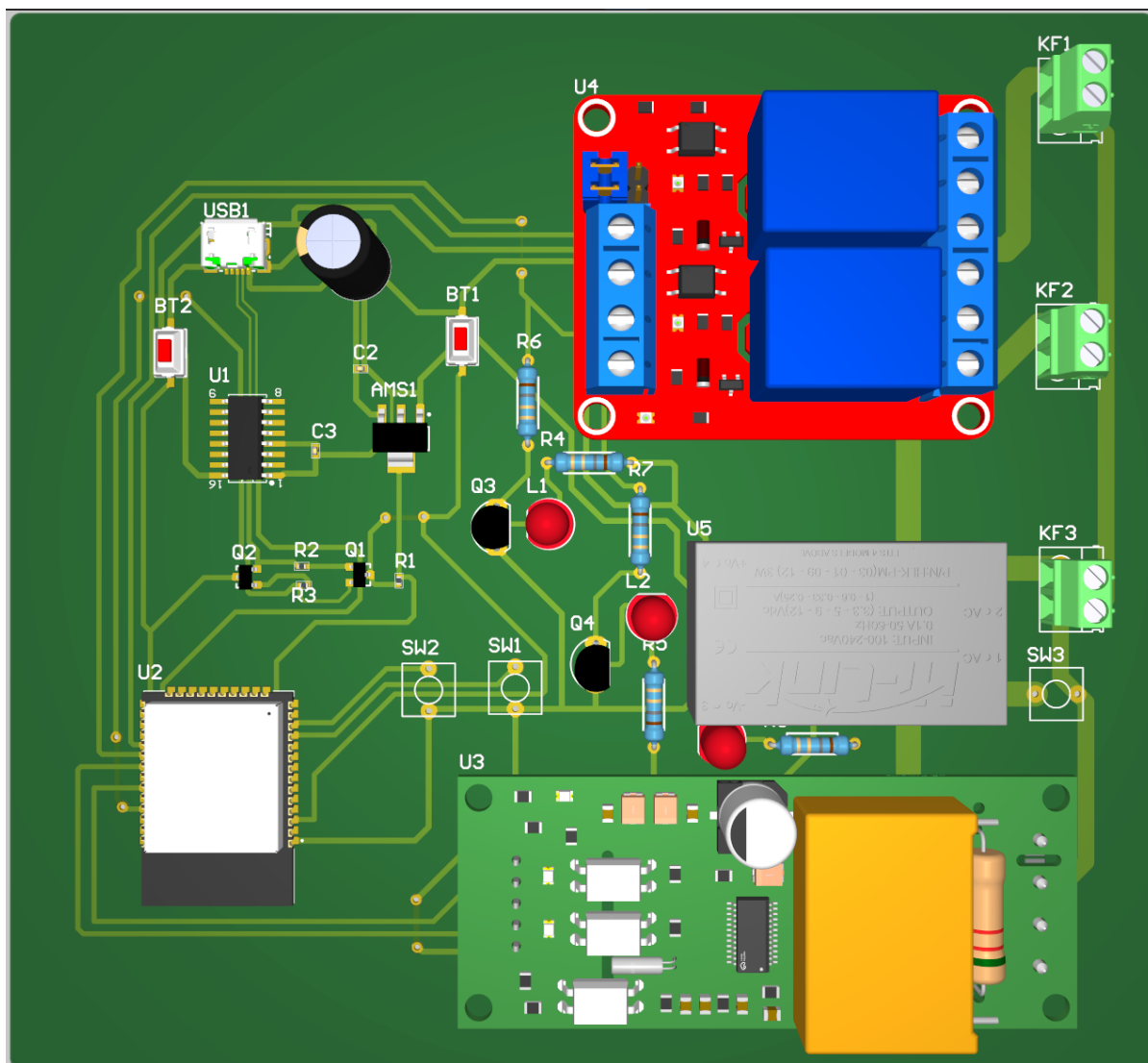
Ở mạch nạp, các điện trở R2 và R3 chọn giá trị 10 k Ω vì hai đường DTR/RTS của CH340C chỉ cần tạo tín hiệu điều khiển logic cho transistor 2N2222, không phải cấp dòng tải lớn. Giá trị 10 k Ω đủ để kích transistor kéo các chân EN/IO0 theo đúng trình tự nạp, đồng thời không làm nặng ngõ ra của CH340C và hạn chế dòng không cần thiết khi cắm USB lâu dài. Nếu chọn điện trở quá nhỏ, dòng base tăng nhưng không mang lại lợi ích rõ rệt; nếu chọn quá lớn, transistor có thể chuyển trạng thái kém chắc chắn trong một số thời điểm nạp.

Đối với mạch LED, điện trở 1 k Ω được chọn cho cả nhánh hạn dòng LED và nhánh kích base transistor. Với nguồn 5V và LED đỏ có điện áp thuận xấp xỉ 2V, điện trở 1 k Ω cho dòng LED khoảng $(5 - 2) / 1000 = 3 \text{ mA}$, đủ sáng để quan sát trạng thái nhưng không gây chói và không làm tăng tải nguồn. Ở nhánh base, điện trở 1 k Ω giới hạn dòng từ tín hiệu điều khiển khoảng vài mA, đủ đưa transistor C1815 vào trạng thái

dẫn bão hòa cho tải LED nhỏ. Vì vậy cùng giá trị $1\text{ k}\Omega$ vừa dễ thi công, dễ kiểm tra, vừa phù hợp với dòng làm việc thực tế.

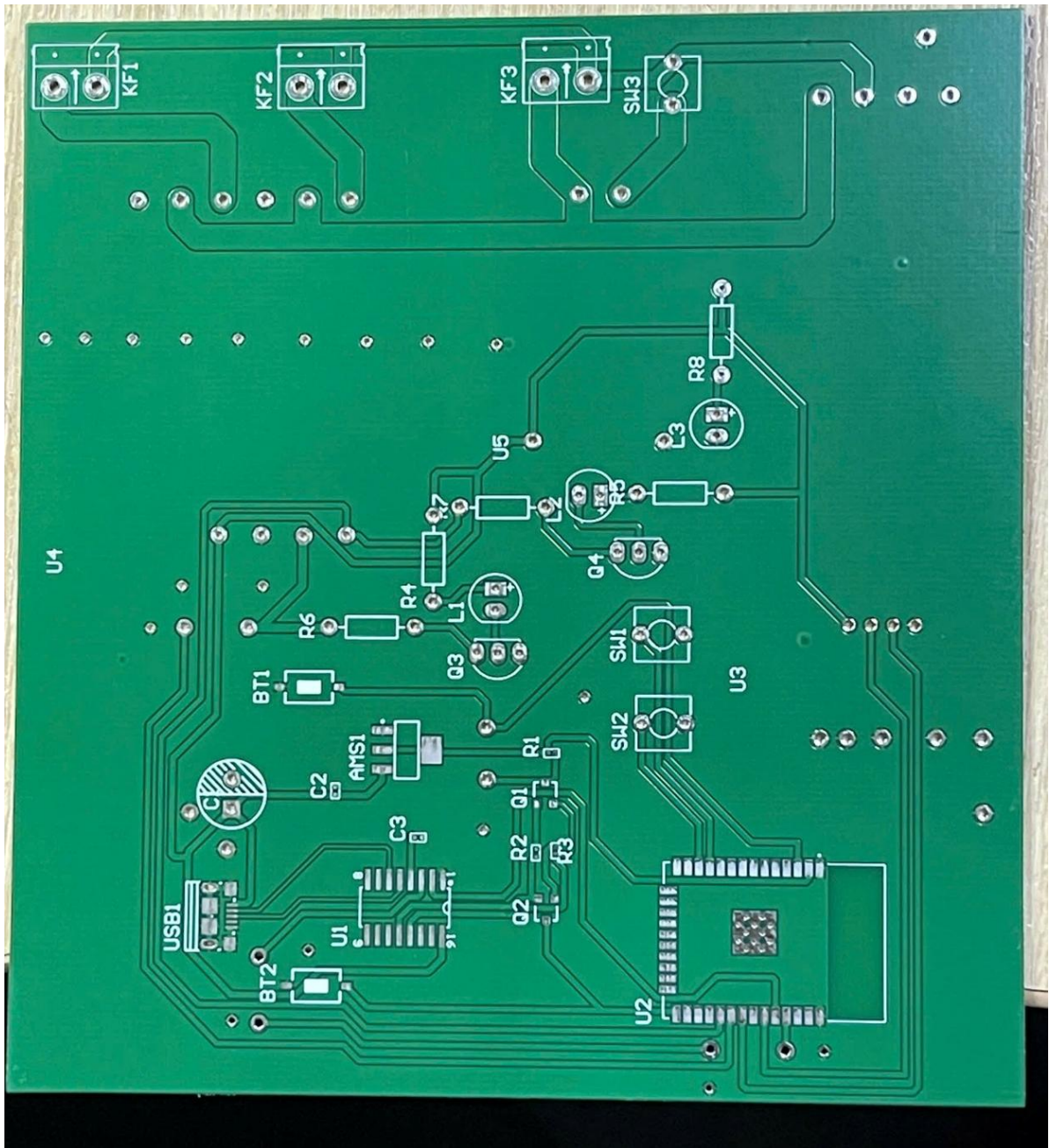
Trên bo mạch có cả điện trở dán và điện trở xuyên lỗ. Điện trở dán được ưu tiên ở các vùng mạch nạp, transistor và tín hiệu nhỏ vì kích thước gọn, đường nối ngắn và phù hợp với footprint của các IC chân nhỏ. Trong khi đó một số điện trở xuyên lỗ được dùng ở vùng LED/nút hoặc vùng còn nhiều khoảng trống để dễ hàn tay, dễ thay đổi giá trị khi thử độ sáng LED và thuận tiện quan sát trong quá trình kiểm tra. Về chức năng điện, cả hai loại đều là điện trở; sự khác nhau chủ yếu nằm ở cách lắp ráp, kích thước cơ khí, khả năng sửa chữa và mức công suất chịu được.

Sau khi hoàn tất đi dây, nhóm thực hiện đổ đồng cho các vùng nguồn và mass để giảm trở kháng đường hồi dòng, tăng diện tích tản nhiệt và giúp mạch gọn hơn. Tiếp đó, mô hình ba chiều của bo mạch trong Hình 4.2 được dùng để kiểm tra lại vị trí linh kiện theo chiều cao thực tế, đặc biệt là relay, PZEM, HLK-PM01B, cổng USB, nút nhấn và các đầu nối dây. Khi mô hình 3D không còn xung đột với vỏ hộp, bộ file Gerber và Drill được xuất từ Altium để gửi nhà sản xuất gia công bo FR-4 hai lớp, dày 1,6 mm, lớp đồng 1 oz và phủ xanh. Đặt gia công giúp đường mạch đều, lỗ khoan chính xác, lớp phủ bảo vệ tốt hơn và độ bền cơ học cao hơn so với phương án tự ăn mòn thủ công.



Hình 4.2: Mô hình ba chiều của bo mạch dựng trên Altium

Sau khi hoàn tất kiểm tra thiết kế trên Altium và xuất file gia công, nhóm nhận được bo mạch in hai lớp ở trạng thái chưa lắp linh kiện. Mặt trước của PCB sau gia công được thể hiện ở Hình 4.3, cho thấy các ký hiệu linh kiện, vị trí đầu nối và đường mạch công suất đã được phủ xanh theo đúng thiết kế.



Hình 4.3: Mặt trước bo mạch sau khi gia công, chưa lắp linh kiện

Quan sát Hình 4.3 có thể thấy các vùng đầu nối KF1, KF2, KF3, nút/công tắc tổng SW3 và các đường mạch 220VAC được đặt ở phía trên bo, tách khỏi vùng điều khiển ESP32, CH340C, transistor, LED và nút nhấn ở phía dưới. Ký hiệu linh kiện và đường bao footprint rõ ràng giúp quá trình hàn lắp sau đó ít nhầm vị trí hơn, nhất là với các linh kiện có cực tính như LED, tụ điện và transistor.

Mặt sau của PCB chưa lắp linh kiện được trình bày ở Hình 4.4. Ở mặt này, lớp phủ xanh che hầu hết bề mặt đồng, chỉ để hở các pad hàn và lỗ xuyên linh kiện; nhờ vậy khi hàn tay, người thi công dễ kiểm tra mỗi hàn và hạn chế chạm ngoài ý muốn giữa các vùng dẫn điện.



Hình 4.4: Mặt sau bo mạch sau khi gia công, chưa lắp linh kiện

Hình 4.4 cũng cho thấy các lỗ khoan và pad hàn đã được gia công tương đối đồng đều, phù hợp với việc lắp linh kiện xuyên lỗ, terminal, nút nhấn và các module

rời. Trước khi hàn linh kiện, nhóm cần kiểm tra nhanh bề mặt bo, các pad quan trọng và vùng đường công suất để phát hiện lỗi sản xuất nếu có, sau đó mới chuyển sang bước hàn lắp ở mục 4.2.2.

4.2.2 Hàn lắp linh kiện và các module

Sau khi có bo mạch, quá trình hàn lắp được thực hiện theo nguyên tắc đi từ linh kiện thấp, nhỏ và khó thao tác đến linh kiện cao hoặc module rời. Các linh kiện dạng nhỏ như điện trở, tụ điện, transistor, IC CH340C, AMS1117 và cổng USB được ưu tiên hàn trước để dễ kiểm soát mỗi hàn. Sau đó mới lắp ESP32, nút nhấn, LED, terminal và các dây nối ra module. Cách làm này phù hợp với bo có cả linh kiện dán, linh kiện xuyên lỗ và module rời; nếu lắp module cao trước thì đầu hàn sẽ khó tiếp cận các vị trí nhỏ, dễ làm xấu mỗi hàn hoặc gây chạm giữa các pad gần nhau.

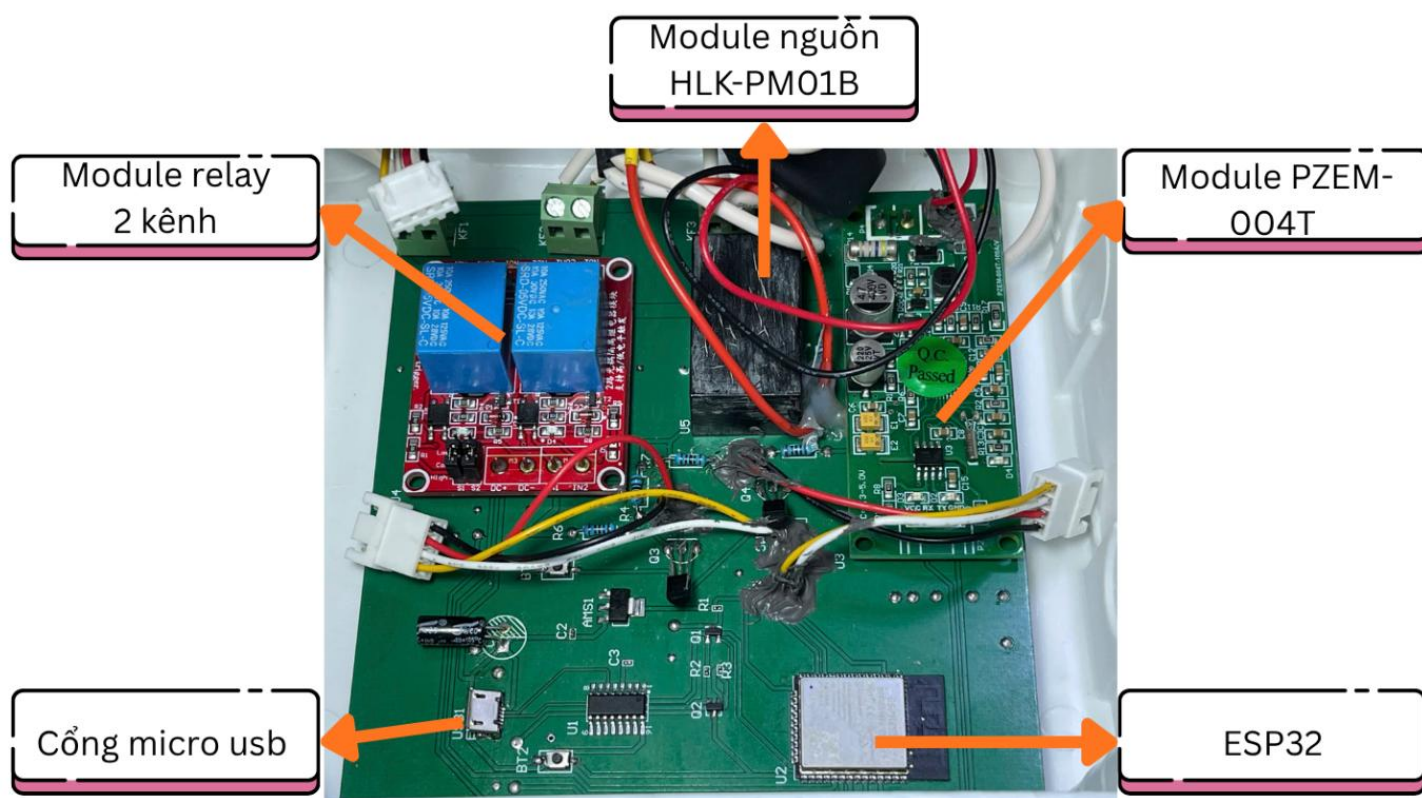
Trên bo chính, các linh kiện hàn trực tiếp gồm ESP32-WROOM-32, IC nạp CH340C, IC ổn áp AMS1117-3.3V, transistor 2N2222 cho mạch tự nạp, transistor C1815 cho LED báo, các điện trở, tụ điện, nút nhấn, LED và cổng USB. Đây là các linh kiện thuộc phần điều khiển điện áp thấp nên được bố trí ở vùng tách khỏi đường công suất. Các linh kiện này cần mỗi hàn gọn vì nhiều chân nằm gần nhau, đặc biệt là CH340C, ESP32 và cổng USB.

Ba khối được đấu như module rời là HLK-PM01B, module relay hai kênh và PZEM-004T. Cách dùng module rời giúp tận dụng mạch cách ly, mạch điều khiển relay và mạch đo đã được nhà sản xuất tích hợp sẵn, giảm rủi ro khi tự thiết kế lại phần có liên quan đến 220VAC. Ngoài ra, các module này có kích thước và chiều cao lớn hơn linh kiện rời; đấu nối bằng dây giúp dễ bố trí trong hộp, dễ thay thế khi hư hỏng và không bắt bo chính phải chịu toàn bộ lực cơ từ dây công suất.

Đối với phần đo điện năng, PZEM-004T nhận tín hiệu UART từ ESP32 và dùng biến dòng CT để đo dòng tải. Dây pha 220VAC được đưa qua vòng CT của PZEM trước khi đi qua nhánh đóng/ngắt tải, nhờ vậy module đo được dòng tiêu thụ mà không

cần cắt dòng tải chạy qua bo điều khiển thấp áp. Cách đo bằng CT cũng giúp phân tín hiệu của PZEM tách biệt hơn với dòng tải lớn, phù hợp với yêu cầu an toàn của ổ cắm.

Sau khi hàn lắp, bước kiểm tra được thực hiện theo hướng không cấp 220VAC ngay lập tức. Trước tiên cần quan sát mối hàn, kiểm tra các điểm dễ chạm như chân IC, chân USB, vùng AMS1117, vùng transistor và các terminal. Tiếp theo kiểm tra thông mạch và kiểm tra chập giữa các đường nguồn chính như 5V, 3,3V và GND. Chỉ khi phân thấp áp không có dấu hiệu chập mới cấp nguồn theo từng bước để kiểm tra mức 5V, 3,3V và phản ứng đóng/ngắt relay. Cách kiểm tra này giúp cô lập lỗi, tránh trường hợp đưa điện lưới vào khi mạch điều khiển vẫn còn lỗi hàn.



Hình 4.5: Bo mạch sau khi hàn lắp linh kiện và đấu nối các module

Hình 4.5 là bo mạch sau khi hoàn thiện hàn lắp và đấu nối các module. Có thể nhận ra vi điều khiển ESP32, module đo PZEM-004T, module nguồn HLK-PM01B và module relay hai kênh. Các dây tín hiệu được đi bằng cáp nhiều lõi để gom gọn, còn

dây công suất 220VAC được tách khỏi nhóm tín hiệu và đưa qua đường đo, nút/công tắc tổng SW3, tiếp điểm relay trước khi ra ổ cắm.

Một điểm cần chú ý trong Hình 4.5 là các module rời và dây công suất không chỉ được đấu đúng về điện mà còn phải được giữ ổn định về cơ. Điều này đặc biệt quan trọng tại terminal, module nguồn, module relay và dây đi ra ổ cắm, vì các vị trí này chịu lực kéo nhiều hơn so với dây tín hiệu. Nếu lực cơ truyền trực tiếp vào mối hàn, tiếp xúc có thể lỏng sau một thời gian sử dụng.

Phân dây 220VAC và phân dây tín hiệu được xử lý theo hai yêu cầu khác nhau. Dây 220VAC dùng dây silicone lõi đồng nhiều sợi loại 20AWG chịu nhiệt cao, bọc cách điện, được chọn phù hợp với phạm vi tải thử nghiệm của mô hình, đi theo đường ngắn, ít uốn gấp và tách khỏi phần ESP32. Dây tín hiệu như UART PZEM, tín hiệu relay, nút nhấn và LED chỉ mang dòng nhỏ nên dùng cáp nhiều sợi nhỏ có giắc cắm JST để dễ gom và không chiếm nhiều không gian trong hộp. Hai loại dây sử dụng được thể hiện ở Hình 4.6. Sự phân biệt này thống nhất với nguyên tắc thiết kế PCB ở Hình 4.1: đường công suất ưu tiên dòng và cách điện, đường tín hiệu ưu tiên gọn và ít nhiễu.



Hình 4.6: Hai loại dây sử dụng trong thi công (nguồn: nhà sản xuất)

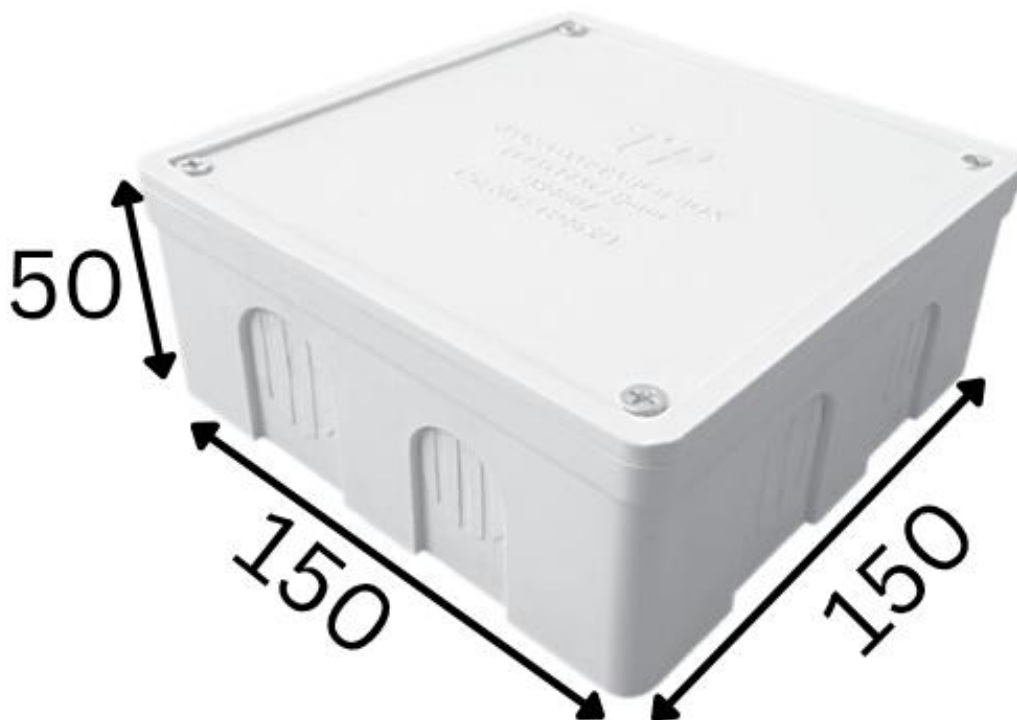
Trong quá trình thi công thực tế, các vị trí cần kiểm tra kỹ nhất là vùng cấp nguồn 5V/3,3V, vùng USB-CH340C để bảo đảm nạp chương trình ổn định, vùng relay để tránh nhầm COM/NO/NC, và vùng PZEM để bảo đảm dây pha đi đúng qua CT.

Các lỗi ở những vị trí này thường gây hậu quả khác nhau: lỗi nguồn làm ESP32 khởi động không ổn định, lỗi mạch nạp làm không nạp được chương trình, lỗi relay làm ổ cắm không đóng/ngắt đúng, còn lỗi CT khiến số đo dòng/công suất sai hoặc bằng không.

Sau khi cấp nguồn và chạy thử, khối nguồn HLK-PM01B 5V/0,6A không ghi nhận hiện tượng ESP32 tự reset hoặc sụt áp làm relay hoạt động sai, cho thấy nguồn đáp ứng được bản mẫu hiện tại.

4.2.3 Đóng gói sản phẩm

Sau khi hàn lắp và kiểm tra, toàn bộ mạch được đóng gói vào vỏ hộp nhựa tự chế kích thước 150 mm x 150 mm x 50 mm để bảo vệ cơ học, hạn chế người dùng chạm vào phần 220VAC và tạo không gian lắp đặt cho bo mạch, module relay, PZEM-004T, HLK-PM01B cùng dây đấu nối. Vỏ hộp được chọn là loại hộp nhựa có nắp bắt vít và các lỗ chờ ở thành hộp như Hình 4.7. Kích thước này đủ cho bản mẫu vì chiều cao 50 mm vẫn bố trí được các module có chiều cao lớn, đồng thời diện tích đáy 150 mm x 150 mm cho phép tách tương đối rõ vùng công suất và vùng điều khiển.



Hình 4.7: Vỏ hộp nhựa kích thước 150 mm x 150 mm x 50 mm

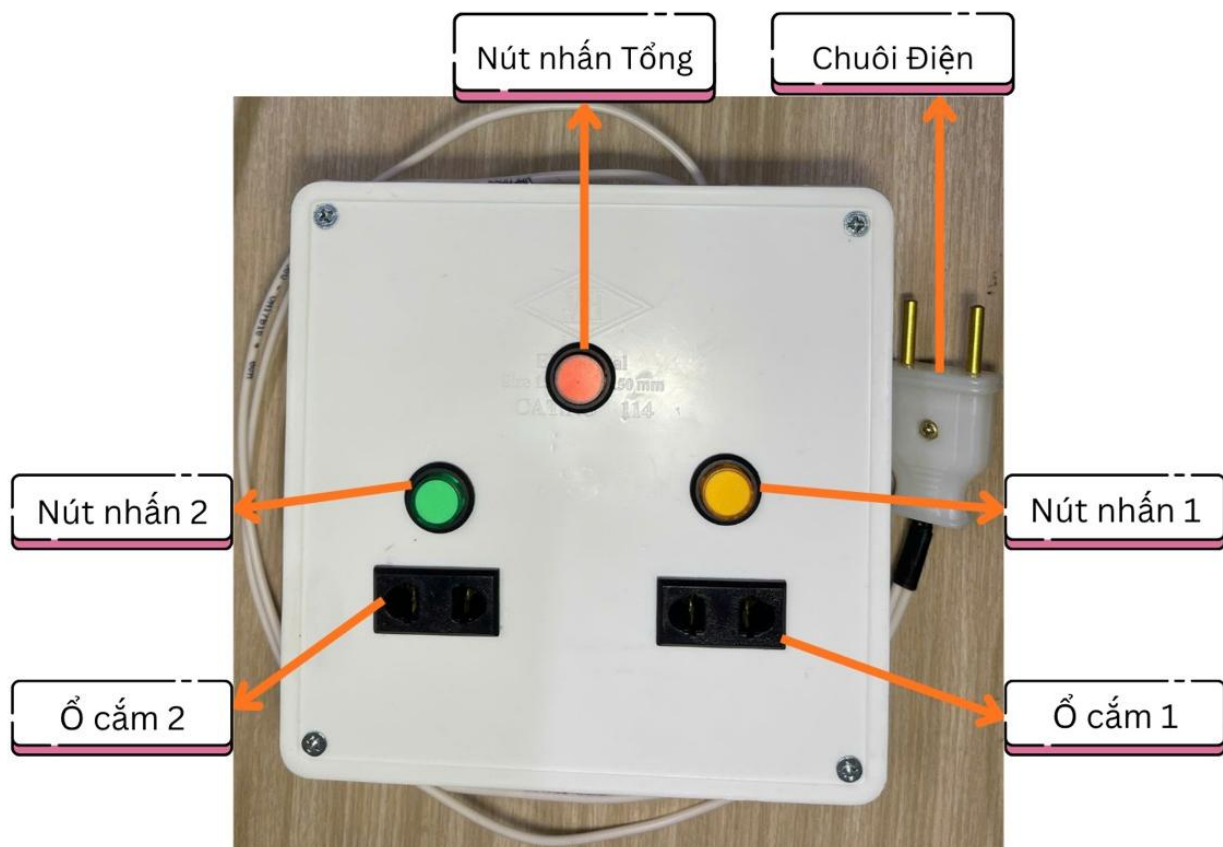
Khi đưa mạch vào hộp, phần công suất 220VAC được ưu tiên bố trí sát khu vực dây vào/ra để đường dây ngắn và dễ kiểm tra. Dây pha đi qua nút/công tắc tổng SW3, CT đo dòng của PZEM và tiếp điểm relay trước khi ra ổ cắm; dây trung tính và các dây cấp nguồn được đi theo sơ đồ đấu nối đã thiết kế. Tại đầu vào nguồn và đầu ra của hai ổ cắm, nhóm sử dụng domino/đầu nối bắt vít để siết dây, giúp việc đấu nối chắc hơn so với xoắn dây trực tiếp và thuận tiện khi cần tháo kiểm tra.

Trong phần đóng gói, yêu cầu cố định được hiểu là giữ cho dây và đầu nối không bị xô dịch gây kéo căng mỗi hàn hoặc chạm sang vùng điện áp khác. Vì vậy, các đầu dây công suất được siết chắc tại domino và chuỗi ổ cắm, bó dây được đi theo hướng ít bị ép khi đóng nắp, còn dây tín hiệu được gom gọn để không vướng vào nắp hộp hoặc chạm sang vùng 220VAC. Cách xử lý này phù hợp với bản mẫu dùng vỏ hộp nhựa tự chế, nơi không có cơ cấu gá riêng như một sản phẩm thương mại hoàn chỉnh.

Các phần hiển thị và thao tác trên mặt hộp được bố trí ở vùng thấp áp, không lấy trực tiếp từ 220VAC. Hai nút nhấn điều khiển từng ổ cắm được đặt gần khu vực ổ cắm tương ứng để người dùng dễ thao tác, còn nút nhấn tổng được đặt phía trên để phân biệt với hai nút kênh. Cách bố trí này giúp mặt trước thiết bị dễ nhận biết chức năng, đồng thời tránh đưa các chi tiết thao tác của người dùng vào gần vùng dây công suất bên trong hộp.

Thiết bị không lắp thêm cầu chì hoặc aptomat riêng bên trong hộp, do đó phần bảo vệ quá dòng ngắn mạch phụ thuộc vào hệ thống điện bên ngoài và giới hạn sử dụng của bản mẫu. Khi trình bày trong báo cáo, điểm này cần được nêu đúng thực tế: mạch đã có công tắc nguồn tổng, relay đóng/ngắt từng kênh và domino đấu nối dây, nhưng chưa tích hợp tầng bảo vệ điện chuyên dụng như cầu chì, aptomat mini hoặc chống sét lan truyền. Đây cũng là một hướng cần cải tiến nếu phát triển sản phẩm theo hướng sử dụng lâu dài.

Dùng vỏ hộp nhựa có sẵn giúp rút ngắn thời gian thi công so với việc thiết kế và in 3D vỏ riêng. Với đồ án này, mục tiêu chính là chứng minh mạch đo, điều khiển và truyền dữ liệu hoạt động ổn định; do đó vỏ tự chế là lựa chọn thực tế vì dễ mua, dễ gia công, đủ cứng và đủ cách điện cho bản mẫu. Tuy nhiên, phương án này vẫn có hạn chế về thẩm mỹ, độ vừa khít của các lỗ khoan/cắt và khả năng tối ưu vị trí linh kiện so với một vỏ được thiết kế riêng theo kích thước PCB. Sản phẩm hoàn chỉnh sau khi lắp ổ cắm, nút nhấn và dây nguồn được thể hiện ở Hình 4.8.



Hình 4.8: Sản phẩm hoàn chỉnh sau khi đóng gói

Hình 4.8 cho thấy sản phẩm sau khi hoàn thiện đóng gói. Mặt trước hộp bố trí hai ổ cắm đầu ra tương ứng với hai kênh relay, cho phép điều khiển hai tải độc lập. Hai nút nhấn màu vàng và xanh được đặt gần từng ổ cắm để người dùng thao tác trực tiếp với ổ cắm 1 và ổ cắm 2, trong khi nút nhấn tổng màu cam đặt ở phía trên dùng cho thao tác điều khiển chung của thiết bị. Chuôi điện được đưa ra bên hông hộp để cấp nguồn 220VAC cho toàn bộ hệ thống. Cách bố trí này giúp người dùng dễ nhận biết từng chức năng khi thao tác, đồng thời giữ phần điều khiển và phần ổ cắm ở các vị trí tách biệt, thuận tiện cho quá trình sử dụng và kiểm tra. Do đây là bản mẫu thi công phục vụ đồ án, hình thức vỏ hộp ưu tiên tính gọn, dễ gia công và dễ kiểm tra hơn so với tiêu chuẩn thẩm mỹ của sản phẩm thương mại.

4.3 PHẦN MỀM

Hệ thống phần mềm của đề tài được xây dựng gồm bốn thành phần chính phối hợp chặt chẽ với nhau, bao gồm máy chủ backend trên nền NestJS, cơ sở dữ liệu quan hệ PostgreSQL, lớp truyền thông nhúng thời gian thực giữa thiết bị và máy chủ, và giao diện điều khiển Web Dashboard xây dựng bằng Next.js. Phần này trình bày chi tiết cách cài đặt môi trường và hiện thực hóa từng thành phần đó.

4.3.1 Cài đặt và cấu hình môi trường

Toàn bộ phần mềm phía máy chủ được xây dựng trên môi trường Node.js. Cơ sở dữ liệu PostgreSQL được triển khai dưới dạng dịch vụ Managed Database trên nền tảng đám mây Render, kết hợp cấu hình tường lửa kiểm soát truy cập để bảo đảm an toàn. MQTT Broker HiveMQ Cloud được khởi tạo dưới dạng một cụm broker bảo mật, yêu cầu xác thực bằng tài khoản và mật khẩu nhúng, đồng thời bắt buộc sử dụng chuẩn mã hóa SSL/TLS trên cổng bảo mật 8883. Tập cấu hình môi trường chứa toàn bộ các tham số nhạy cảm, bao gồm chuỗi kết nối tới PostgreSQL, các khóa bảo mật JWT, thông số Google Client ID và Client Secret phục vụ đăng nhập bằng tài khoản Google, cùng địa chỉ của HiveMQ Broker, để từ đó khởi chạy máy chủ NestJS tại cổng dịch vụ 3001.

4.3.2 Hiện thực hóa các API backend và cơ sở dữ liệu

Máy chủ backend được xây dựng trên khung chương trình NestJS, tuân thủ chặt chẽ mô hình kiến trúc hướng mô-đun. Toàn bộ nghiệp vụ được chia nhỏ thành các mô-đun chức năng biệt lập gồm AuthModule phụ trách xác thực, HousesModule quản lý nhà, RoomsModule quản lý phòng, NodesModule quản lý các bộ điều khiển, DevicesModule quản lý thiết bị, MqttModule phụ trách truyền thông, RealtimeModule phụ trách thời gian thực và LogsModule phụ trách nhật ký, tất cả được liên kết với nhau bằng cơ chế tiêm phụ thuộc thông qua mô-đun gốc AppModule. Cách tách mô-đun như vậy giúp mã nguồn rõ ràng, dễ bảo trì và mở rộng từng phần một cách độc lập.

Để tương tác với cơ sở dữ liệu PostgreSQL, hệ thống sử dụng thư viện TypeORM để ánh xạ các bảng dữ liệu vật lý thành các thực thể định kiểu mạnh trong TypeScript. Thực thể User quản lý thông tin định danh người dùng, mật khẩu đã được mã hóa bằng thuật toán bcrypt và mã đặt lại mật khẩu. Thực thể House định nghĩa không gian ngôi nhà và tham chiếu tới chủ sở hữu, còn thực thể HouseMember là bảng quan hệ liên kết giữa người dùng và ngôi nhà, quy định vai trò là chủ nhà hay thành viên cùng các cờ phân quyền cụ thể như quyền quản lý phòng, quyền quản lý bộ điều khiển và quyền quản lý thiết bị. Thực thể Node quản lý các vi điều khiển ESP32 bao gồm mã địa chỉ MAC phần cứng, trạng thái kết nối trực tuyến hay ngoại tuyến, phiên bản firmware và các thông số đo điện tức thời, trong khi thực thể Device quản lý chi tiết từng ro-le chấp hành vật lý và ánh xạ tới mã định danh cổng GPIO trên vi xử lý. Ngoài ra, các thực thể Log, HourlyEnergy, DailyEnergy và MonthlyEnergy đảm nhận việc lưu trữ nhật ký sự kiện, lỗi vận hành và dữ liệu năng lượng tích lũy theo ba cấp giờ, ngày và tháng. Cấu trúc cơ sở dữ liệu được đồng bộ hóa và nâng cấp an toàn qua cơ chế Migrations của TypeORM thông qua các tệp kịch bản lịch sử, bảo đảm tính toàn vẹn dữ liệu khi chuyển giao giữa các môi trường phát triển khác nhau.

Quy trình xác thực và kiểm soát phân quyền được thực thi nghiêm ngặt thông qua hai lớp bảo vệ nối tiếp nhau. Ở tầng ngoài, lớp bảo vệ JwtAuthGuard kế thừa từ AuthGuard của thư viện Passport, kết hợp với chiến lược JwtStrategy để giải mã chữ ký số trong Bearer Token gửi kèm trong phần Header của yêu cầu, kiểm tra thời gian hết hạn và so khớp định danh người dùng trong cơ sở dữ liệu; nếu token hợp lệ, thông tin người dùng được gắn trực tiếp vào đối tượng yêu cầu để chuyển tiếp tới các Controller. Ở tầng trong, lớp dịch vụ HouseMembershipService được nhúng vào các phương thức nghiệp vụ để kiểm tra quyền hạn thực tế của người dùng đối với ngôi nhà cần truy cập; hàm assertMember truy vấn bảng HouseMember để xác nhận tư cách thành viên, nếu không tìm thấy sẽ lập tức ném ra lỗi từ chối truy cập với mã HTTP 403, còn các hàm chuyên biệt như assertManageRooms, assertManageNodes và assertManageDevices kiểm tra chi tiết từng quyền tương ứng trước khi cho phép thay

đôi cấu trúc hệ thống. Cách bảo vệ hai lớp này vừa xác thực được danh tính, vừa giới hạn chính xác phạm vi thao tác của từng người dùng trên từng ngôi nhà.

4.3.3 Hiện thực hóa cơ chế truyền thông thời gian thực

Cơ chế truyền thông thời gian thực giữa thiết bị nhúng ESP32, máy chủ NestJS và giao diện Web Dashboard được xây dựng đồng thời trên cả hai giao thức MQTT và WebSocket. Lớp dịch vụ MqttService đảm nhận vai trò một MQTT Client chạy ngầm, quản lý kết nối an toàn tới broker HiveMQ Cloud. Bằng cách thực thi các cổng giao tiếp vòng đời của NestJS, dịch vụ tự động kết nối qua cổng bảo mật TLS 8883 khi máy chủ khởi động và đóng kết nối gọn gàng khi máy chủ tắt. Lớp này đăng ký lắng nghe hai nhóm chủ đề chính, gồm chủ đề trạng thái dạng `v1/homes/+/nodes/+/state` để tiếp nhận liên tục dữ liệu trạng thái và số đo điện năng từ cảm biến PZEM-004T gửi lên, và chủ đề phản hồi dạng `v1/homes/+/nodes/+/ack` để lắng nghe bản tin xác nhận lệnh từ vi điều khiển gửi về.

Giải thuật điều khiển chịu lỗi theo cơ chế hết thời gian chờ và tự động hoàn tác được hiện thực hóa ngay trong lớp này thông qua một cấu trúc bản đồ `pendingCommands` dùng để quản lý các lệnh đang chờ phản hồi. Khi có yêu cầu thay đổi trạng thái rơ-le từ người dùng, hệ thống trước hết lưu lại trạng thái rơ-le hiện hành của thiết bị trước khi thay đổi, sau đó thực hiện cập nhật tạm thời trạng thái mới lên cơ sở dữ liệu theo kiểu cập nhật lạc quan để tối ưu trải nghiệm giao diện, rồi sinh một mã lệnh ngẫu nhiên dạng UUID, thiết lập một bộ định thời đếm ngược mười giây gắn liền với mã lệnh đó và lưu vào bộ nhớ đệm. Tiếp theo, hệ thống đóng gói và phát bản tin lệnh dưới dạng chuỗi JSON chứa mã lệnh, hành động toggle, mã thiết bị và chỉ số chân rơ-le lên chủ đề lệnh dạng `v1/homes/{houseId}/nodes/{hardwareId}/cmd`. Trong trường hợp thành công, thiết bị nhúng nhận tin, kích hoạt chân GPIO vật lý và trả ngay bản tin xác nhận về chủ đề phản hồi; hàm xử lý phản hồi trên máy chủ bắt được tin, giải phóng bộ đếm thời gian, xóa bản ghi khỏi bộ nhớ đệm, ghi nhật ký thành công và phát tín hiệu đồng bộ giao diện qua WebSocket. Trong trường hợp thất bại hoặc bộ đếm mười giây trôi qua mà chưa nhận được phản hồi, hàm xử lý hết thời gian chờ sẽ

tự động ghi đè cơ sở dữ liệu để hoàn tác thiết bị về trạng thái cũ, lưu nhật ký sự cố và phát sự kiện WebSocket yêu cầu trình duyệt trả nút bấm về vị trí ban đầu để cảnh báo người dùng.

Tầng giao tiếp WebSocket với trình duyệt được thiết lập bởi lớp cổng RealtimeGateway sử dụng thư viện Socket.io tích hợp sẵn trong NestJS, và được bảo vệ chặt chẽ bằng cách yêu cầu xác thực JWT ngay trong quá trình thiết lập kết nối thông qua thông tin token đính kèm. Để ngăn ngừa rò rỉ dữ liệu giữa các ngôi nhà khác nhau, cổng giao tiếp áp dụng cơ chế truyền phát theo phòng. Khi client gửi thông điệp tham gia nhà, máy chủ tiến hành kiểm tra bảo mật trong cơ sở dữ liệu, nếu hợp lệ mới cho phép kết nối vào nhóm tương ứng của ngôi nhà đó. Mọi cập nhật thông số điện năng hay trạng thái thiết bị sau đó chỉ được phát nội bộ bên trong phòng tương ứng qua các hàm phát sự kiện thay đổi trạng thái thiết bị và thay đổi trạng thái node, đảm bảo tính riêng tư tuyệt đối cho dữ liệu của từng ngôi nhà.

4.3.4 Hiện thực hóa giao diện Web Dashboard

Giao diện người dùng Web Dashboard được phát triển bằng framework Next.js 15 sử dụng cấu trúc định tuyến mới App Router, tập trung tối ưu hóa hiệu năng kết xuất động và đồng bộ hóa trạng thái thời gian thực. Trạng thái toàn cục của ứng dụng được quản lý qua một lớp ngữ cảnh React, duy trì danh sách các ngôi nhà mà người dùng tham gia và ngôi nhà đang được chọn hiển thị trên bảng điều khiển; định danh ngôi nhà đang hiển thị được lưu lâu dài dưới trình duyệt thông qua bộ nhớ cục bộ, giúp giữ nguyên ngữ cảnh hoạt động của người dùng kể cả khi tải lại trang. Cầu nối thời gian thực phía máy khách là một hook tùy biến, khi bảng điều khiển được kết xuất sẽ tự động gọi kết nối Socket.io tới không gian tên realtime của máy chủ, gửi thông điệp tham gia nhà kèm theo định danh của ngôi nhà hiện tại để đăng ký nhận thông tin của phòng. Hook này đăng ký lắng nghe các sự kiện từ máy chủ, bao gồm sự kiện thay đổi trạng thái thiết bị để đổi màu biểu tượng bật tắt, sự kiện thay đổi trạng thái node để cập nhật trạng thái trực tuyến cùng các chỉ số công suất, dòng và áp lên biểu đồ động, sự kiện phát hiện thiết bị mới được cắm vào để tự động cập nhật danh sách thiết bị, và sự

kiện người dùng bị xóa khỏi nhà để tự động đẩy ra khỏi giao diện điều khiển. Đặc biệt, trong phần dọn dẹp của hook, toàn bộ các lắng nghe sự kiện được hủy đăng ký để ngăn chặn hiện tượng rò rỉ bộ nhớ khi người dùng chuyển trang hoặc đổi nhà liên tục. Ngoài ra, ứng dụng được trang bị hệ thống dịch đa ngôn ngữ động bằng thư viện i18next, trong đó các nhãn hiển thị, tiêu đề và thông điệp cảnh báo được phân tách thành các tệp bản dịch dạng khóa và giá trị, giúp giao diện chuyển đổi tức thời giữa tiếng Việt và tiếng Anh mà không cần tải lại trang.

4.3.5 Hiện thực hóa cơ chế hẹn giờ và lập lịch tự động

Động cơ tự động hóa hẹn giờ đếm ngược và lập lịch lặp lại được xây dựng dựa trên bảng `device_schedules` trong cơ sở dữ liệu PostgreSQL và một tiến trình chạy ngầm theo lịch được cấu hình chạy mỗi phút một lần trên máy chủ NestJS. Bảng `device_schedules` lưu trữ mốc thời điểm thực thi cho bộ hẹn giờ, hoặc chuỗi giờ kích hoạt theo định dạng giờ phút và một mảng các thứ trong tuần cần lặp lại cho bộ lập lịch. Khi được kích hoạt, tiến trình lấy giờ hiện hành của máy chủ, quy đổi sang múi giờ Việt Nam rồi thực hiện so khớp; đối với bộ hẹn giờ, nó quét và thực thi các bản ghi đã quá hạn thời điểm thực thi rồi chuyển cờ kích hoạt về trạng thái ngừng, còn đối với bộ lập lịch, nó so khớp chuỗi giờ phút và thứ trong tuần hiện tại. Các lệnh bật tắt sinh ra được gửi trực tiếp tới thiết bị nhúng qua MQTT và đồng thời truyền phát về Dashboard qua WebSocket để đồng bộ nút bấm trên giao diện. Phía giao diện được hiện thực hóa qua một hộp thoại tùy biến tích hợp bộ nhập thời gian và bộ chọn nhiều thứ trong tuần, còn trạng thái đếm ngược được cập nhật liên tục trên thẻ thiết bị bằng một bộ định thời phía máy khách, giúp người dùng dễ dàng theo dõi thời gian thực thi còn lại.

4.3.6 Hiện thực hóa công cụ dọn dẹp dữ liệu lỗi

Để giải quyết triệt để tình trạng sai số dữ liệu tích lũy và phình to cơ sở dữ liệu do các bản ghi thử nghiệm bị lỗi, hệ thống tích hợp một tệp kịch bản Node.js độc lập đặt trong thư mục `scripts` của dự án. Kịch bản sử dụng trực tiếp trình khách của thư

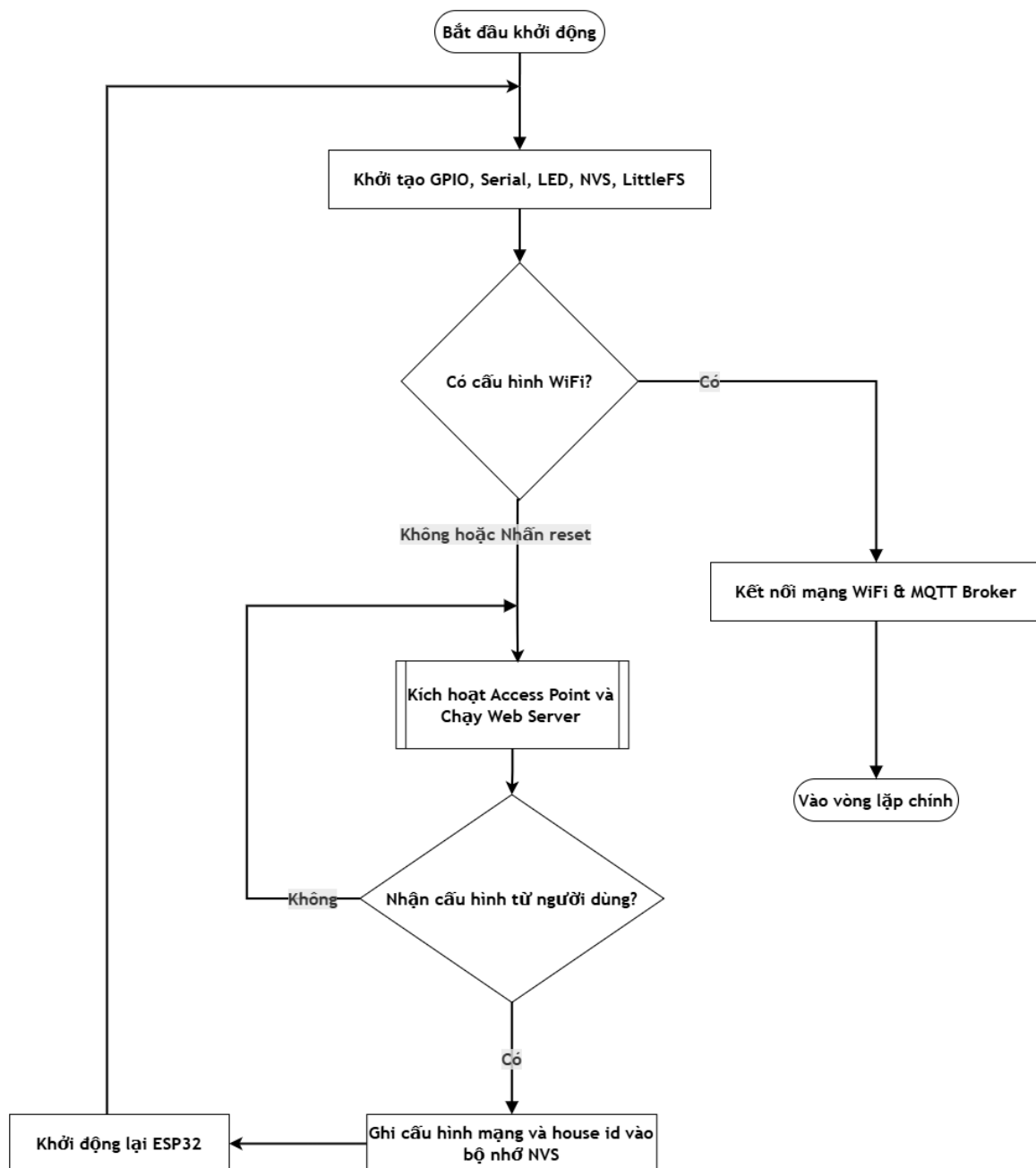
viện pg để tối ưu hóa hiệu năng kết nối tới PostgreSQL, tự động đọc các biến cấu hình từ tệp môi trường hoặc tham số chuỗi kết nối cơ sở dữ liệu. Đặc biệt, khi phát hiện kết nối tới máy chủ đám mây từ xa, kịch bản tự động kích hoạt kết nối mã hóa SSL với tùy chọn không bắt buộc kiểm chứng chứng chỉ để tránh các lỗi ngắt kết nối đột ngột từ phía nhà cung cấp dịch vụ. Sau khi kết nối thành công, kịch bản gửi các câu lệnh xóa toàn bộ dữ liệu trong các bảng năng lượng theo giờ, theo ngày và theo tháng, đồng thời làm sạch các bản ghi nhật ký cảnh báo hỏng tải bị lặp, trả lại cơ sở dữ liệu sạch và chính xác.

4.4 LƯU ĐỒ GIẢI THUẬT

Để điều khiển và mô tả rõ các tiến trình hoạt động bất đồng bộ trong hệ thống, phần này trình bày các lưu đồ giải thuật cốt lõi của cả thiết bị nhúng và máy chủ. Mỗi lưu đồ được vẽ theo chuẩn ISO 5807, trong đó hình elip biểu diễn điểm bắt đầu hoặc kết thúc, hình chữ nhật biểu diễn bước xử lý, hình thoi biểu diễn điều kiện phân nhánh và hình bình hành biểu diễn thao tác nhập xuất dữ liệu.

4.4.1 Lưu đồ giải thuật khởi tạo và cấu hình mạng

Lưu đồ giải thuật khởi tạo và cấu hình mạng của thiết bị nhúng được thể hiện ở Hình 4.9.



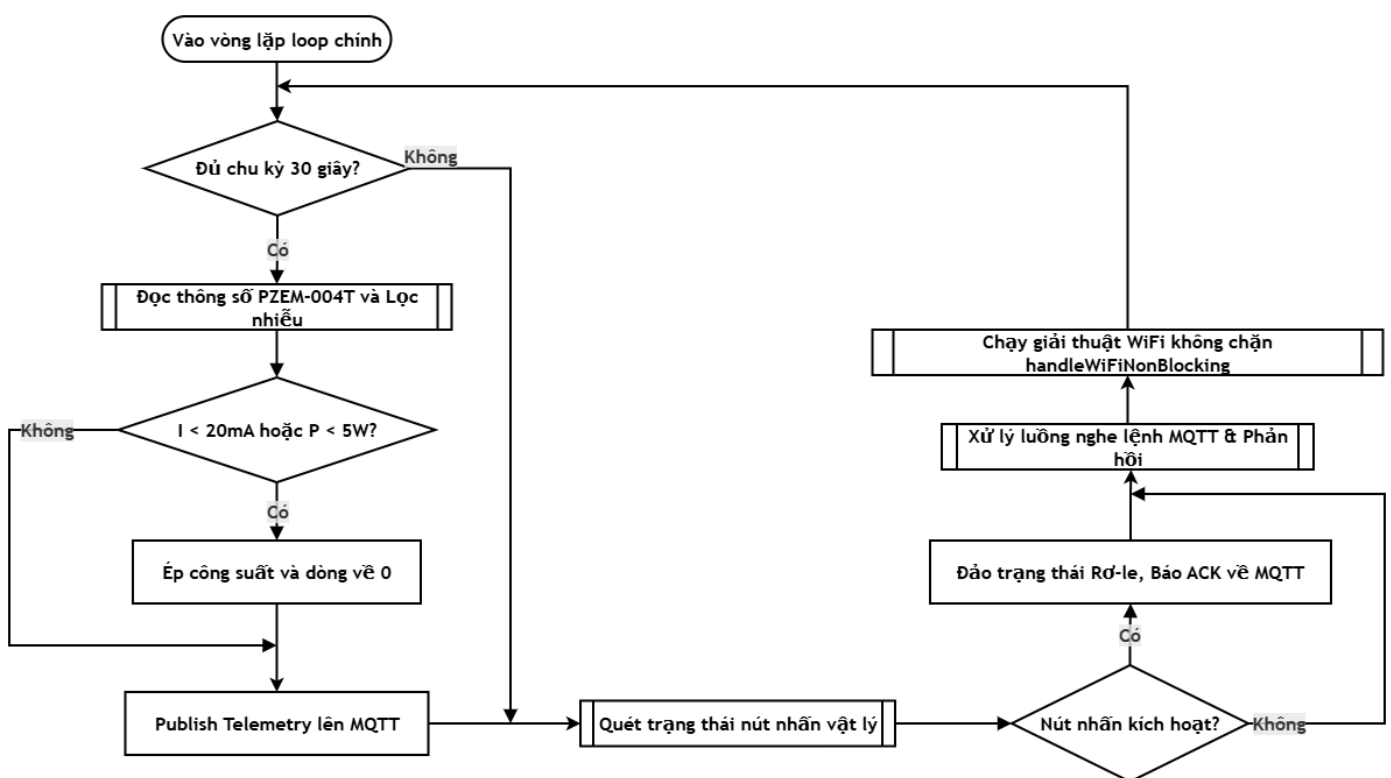
Hình 4.9: Lưu đồ giải thuật khởi tạo và cấu hình mạng của thiết bị nhúng

Lưu đồ thể hiện luồng khởi động ban đầu và tiến trình cấu hình mạng thông qua một điểm truy cập nội bộ khi thiết bị ESP32 chưa có thông tin WiFi hoặc khi người dùng yêu cầu đặt lại cấu hình. Sau khi khởi tạo các cổng ngoại vi GPIO, đèn LED, bộ nhớ NVS và hệ thống tệp LittleFS, vi điều khiển kiểm tra xem trong bộ nhớ Flash đã lưu cấu hình WiFi hay chưa. Nếu chưa có hoặc người dùng nhấn giữ nút đặt lại,

ESP32 chuyển sang chế độ điểm truy cập, phát một mạng WiFi cấu hình tại địa chỉ 192.168.4.1 để nhận tên mạng, mật khẩu và mã nhà từ một trang web cấu hình nội bộ. Khi nhận được cấu hình hợp lệ, thông tin được lưu vào NVS và ESP32 tự động khởi động lại để vào chế độ hoạt động bình thường.

4.4.2 Lưu đồ giải thuật vòng lặp chính

Lưu đồ giải thuật vòng lặp chính của thiết bị nhúng được thể hiện ở Hình 4.10.



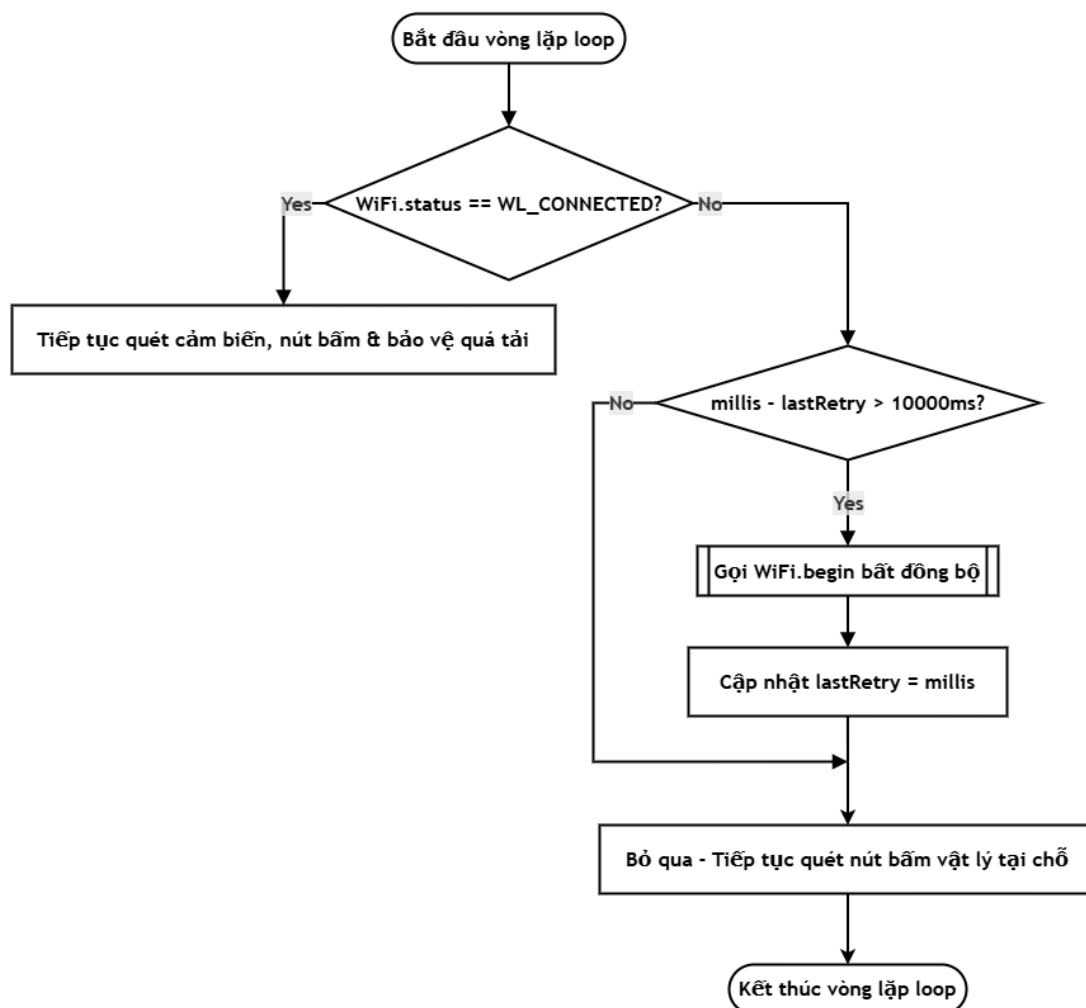
Hình 4.10: Lưu đồ giải thuật vòng lặp chính của thiết bị nhúng

Lưu đồ mô tả logic hoạt động thời gian thực theo cơ chế không chặn trong vòng lặp chính của chương trình nhúng sau khi đã kết nối mạng thành công. Vì xử lý liên tục thực hiện song song nhiều tác vụ bất đồng bộ. Theo định kỳ, thiết bị đọc dữ liệu cảm biến PZEM-004T rồi áp dụng giải thuật lọc nhiễu tiêu thụ nền và giải thuật ngưỡng chênh lệch Delta-Threshold, theo đó chỉ đóng gói và gửi bản tin JSON lên broker khi điện áp lệch quá 4,0V, dòng điện lệch quá 0,05A hoặc công suất lệch quá 2,0W, hoặc khi đã quá hai phút kể từ lần gửi trước theo cơ chế nhịp tim, để tiết kiệm

băng thông truyền tải. Song song đó, thiết bị quét trạng thái nút nhấn vật lý tại chỗ để đóng ngắt ro-le tức thời, lắng nghe và xử lý các lệnh điều khiển nhận được từ broker, và định kỳ gọi tiến trình duy trì kết nối WiFi không chặn.

4.4.3 Lưu đồ giải thuật kết nối WiFi không chặn

Lưu đồ giải thuật kết nối WiFi không chặn được thể hiện ở Hình 4.11.



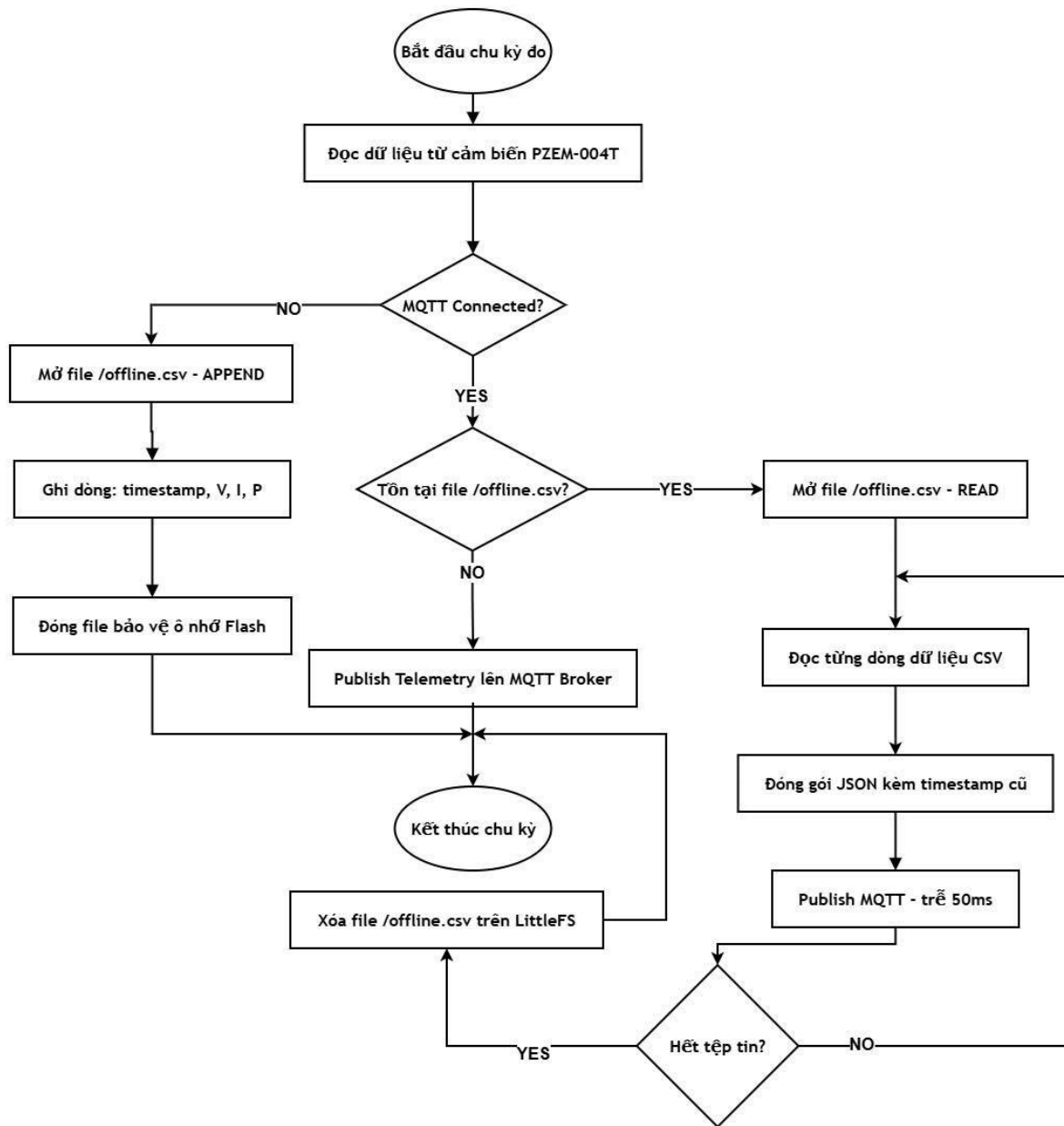
Hình 4.11: Lưu đồ giải thuật kết nối WiFi không chặn

Mỗi lần được gọi trong vòng lặp chính, hàm kiểm tra trạng thái WiFi hiện tại. Nếu đã kết nối, hệ thống tiến hành đồng bộ dữ liệu đệm ngoại tuyến trên LittleFS lên broker rồi tiếp tục chu trình hoạt động bình thường. Nếu mất kết nối, hệ thống kiểm tra khoảng thời gian kể từ lần thử lại gần nhất; nếu chưa quá mười giây thì tiến trình bỏ

qua và tiếp tục thực hiện các tác vụ quét nút nhấn vật lý cùng đọc cảm biến tại chỗ để tránh gây nghẽn chương trình, còn nếu đã quá mười giây thì thiết bị gọi lệnh thiết lập lại kết nối theo kiểu bất đồng bộ và cập nhật mốc thời gian thử lại gần nhất. Nhờ cơ chế này, thiết bị không bị treo trong lúc chờ kết nối và các chức năng điều khiển cục bộ vẫn hoạt động bình thường khi mất mạng.

4.4.4 Lưu đồ giải thuật lưu trữ đệm ngoại tuyến LittleFS

Lưu đồ giải thuật lưu trữ đệm ngoại tuyến trên LittleFS được thể hiện ở Hình 4.12.



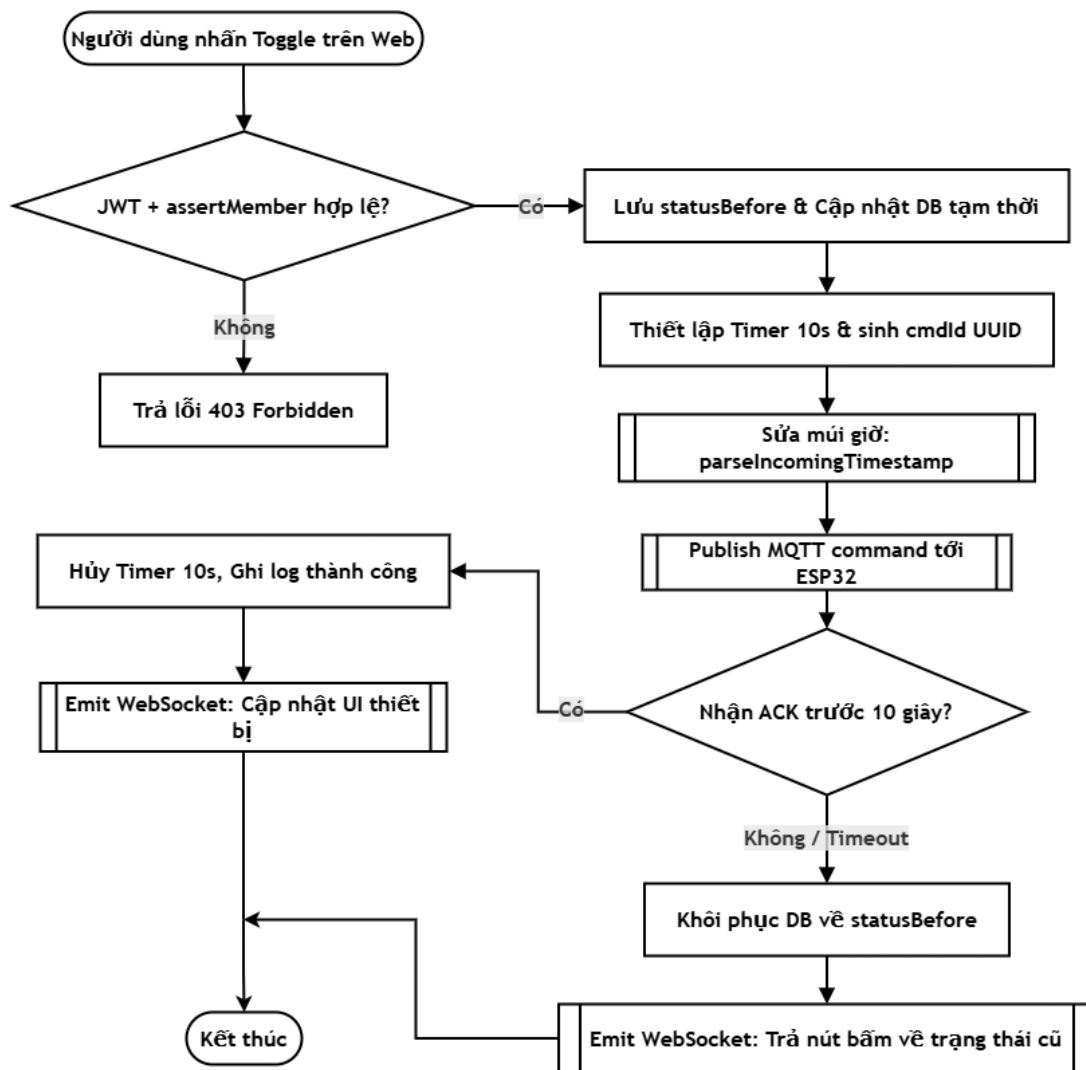
Hình 4.12: Lưu đồ giải thuật lưu trữ đệm ngoại tuyến LittleFS

Sau khi đọc dữ liệu từ cảm biến PZEM-004T, hệ thống kiểm tra trạng thái kết nối MQTT. Nếu đã kết nối, dữ liệu được phát trực tiếp lên broker dưới dạng JSON, đồng thời hệ thống kiểm tra xem có tệp đệm ngoại tuyến tồn tại hay không để đồng bộ dữ liệu lịch sử lên trước khi tiếp tục. Nếu mất kết nối, dữ liệu được ghi nối đuôi vào một tệp đệm trên hệ thống tệp LittleFS theo định dạng rút gọn kèm mốc thời gian nội bộ, bảo đảm không mất mát dữ liệu đo lường trong thời gian gián đoạn. Cơ chế này chính

là giải pháp trực tiếp cho hiện tượng thiếu hụt dữ liệu năng lượng khi thiết bị tạm thời mất kết nối với máy chủ.

4.4.5 Lưu đồ điều khiển Toggle thiết bị và Rollback khi Timeout

Lưu đồ điều khiển bật tắt thiết bị kèm cơ chế tự hoàn tác khi hết thời gian chờ được thể hiện ở Hình 4.13.



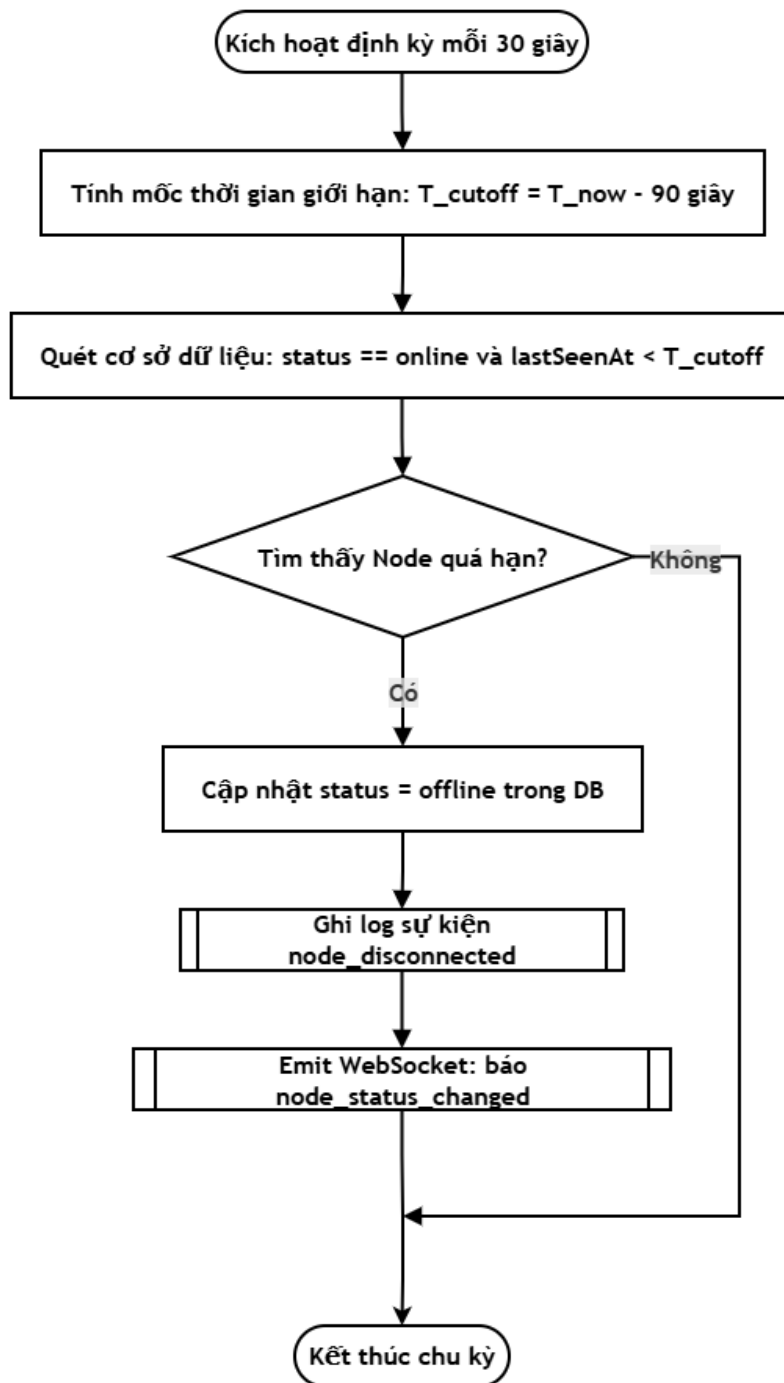
Hình 4.13: Lưu đồ điều khiển Toggle thiết bị và Rollback khi Timeout

Khi người dùng nhấn nút bật tắt từ phía Dashboard, máy chủ NestJS trước hết xác thực token JWT và kiểm tra quyền thành viên đối với ngôi nhà. Nếu hợp lệ, máy chủ ghi nhận trạng thái gốc trước điều khiển, cập nhật tạm thời cơ sở dữ liệu sang giá

trị mới, sinh mã lệnh dạng UUID và thiết lập bộ đếm ngược mười giây, rồi phát lệnh xuống ESP32 qua broker. Nếu nhận được phản hồi xác nhận trước mười giây, bộ đếm bị hủy và WebSocket phát sự kiện cập nhật chính thức giao diện. Nếu hết mười giây mà chưa có phản hồi, máy chủ tự động hoàn tác trạng thái trong cơ sở dữ liệu về giá trị gốc và phát sự kiện WebSocket trả nút bấm trên Dashboard về vị trí cũ. Để tăng độ tin cậy về thời gian, máy chủ triển khai một hàm tự động hiệu chỉnh lệch bảy giờ đối với các bản tin từ firmware bị lệch múi giờ Việt Nam, đồng thời thiết lập hằng số giới hạn khoảng thời gian tích lũy năng lượng tối đa là năm phút để tránh sai lệch dữ liệu không khi thiết bị mất kết nối kéo dài.

4.4.6 Lưu đồ giải thuật quét Heartbeat phát hiện node offline

Lưu đồ giải thuật quét nhịp tim để phát hiện node ngoại tuyến được thể hiện ở Hình 4.14.



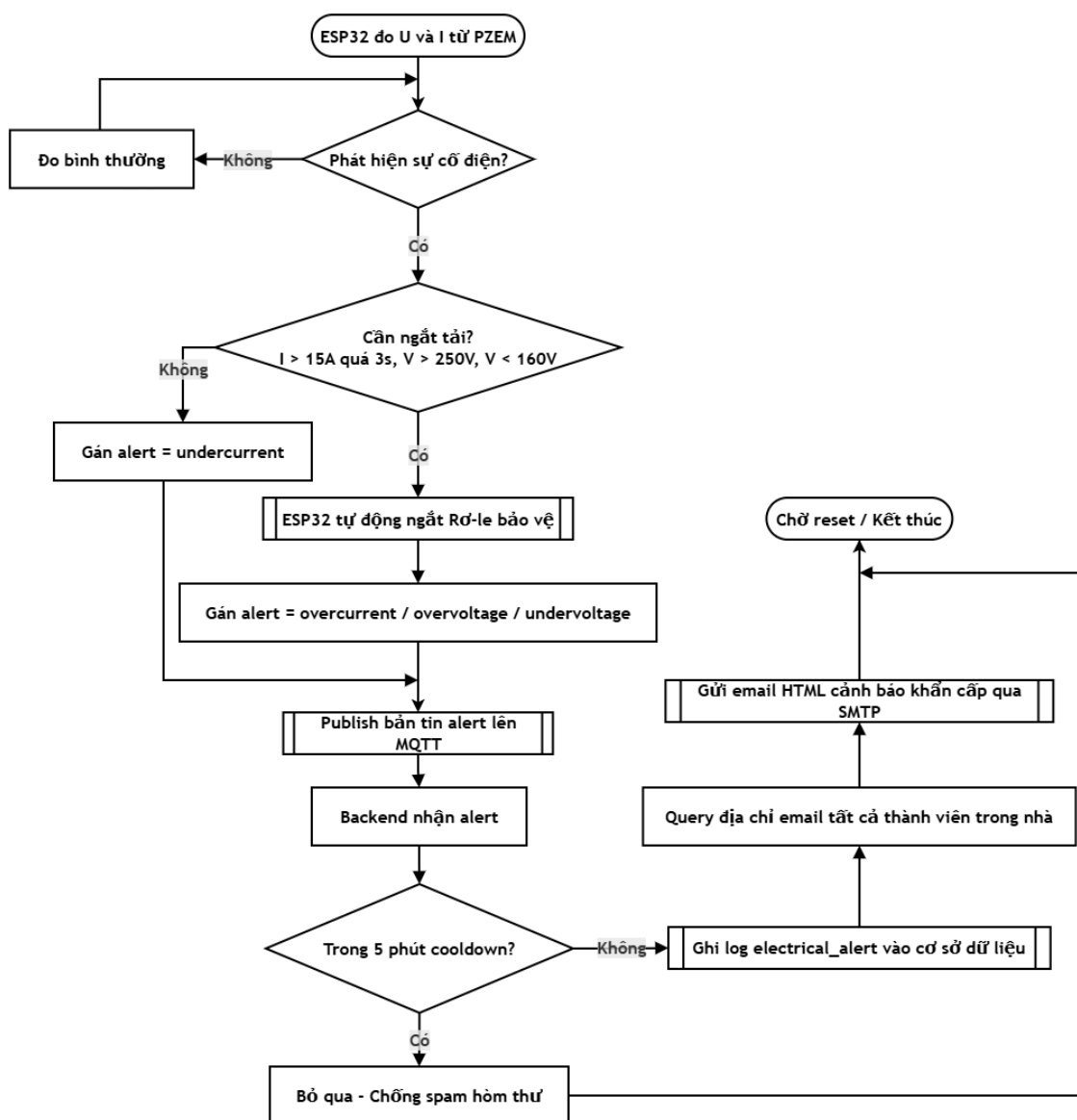
Hình 4.14: Lưu đồ giải thuật quét Heartbeat phát hiện node ngoại tuyến

Máy chủ NestJS thiết lập một chu kỳ quét ngấm định kỳ, mỗi lần tính mốc thời gian giới hạn ngoại tuyến bằng thời điểm hiện tại trừ đi khoảng quá hạn cho phép, sau đó truy vấn cơ sở dữ liệu tìm tất cả các node đang ghi nhận trạng thái trực tuyến nhưng

có thời điểm xuất hiện gần nhất đã quá hạn. Với mỗi node quá hạn tìm thấy, hệ thống cập nhật trạng thái sang ngoại tuyến, ghi nhật ký sự kiện mất kết nối và phát sự kiện WebSocket để đổi biểu tượng hiển thị trên giao diện sang màu báo ngất. Nhờ vậy giao diện luôn phản ánh đúng những thiết bị đang thực sự trực tuyến.

4.4.7 Lưu đồ giải thuật cảnh báo sự cố điện và gửi email tự động

Lưu đồ giải thuật cảnh báo sự cố điện và gửi email tự động được thể hiện ở Hình 4.15.

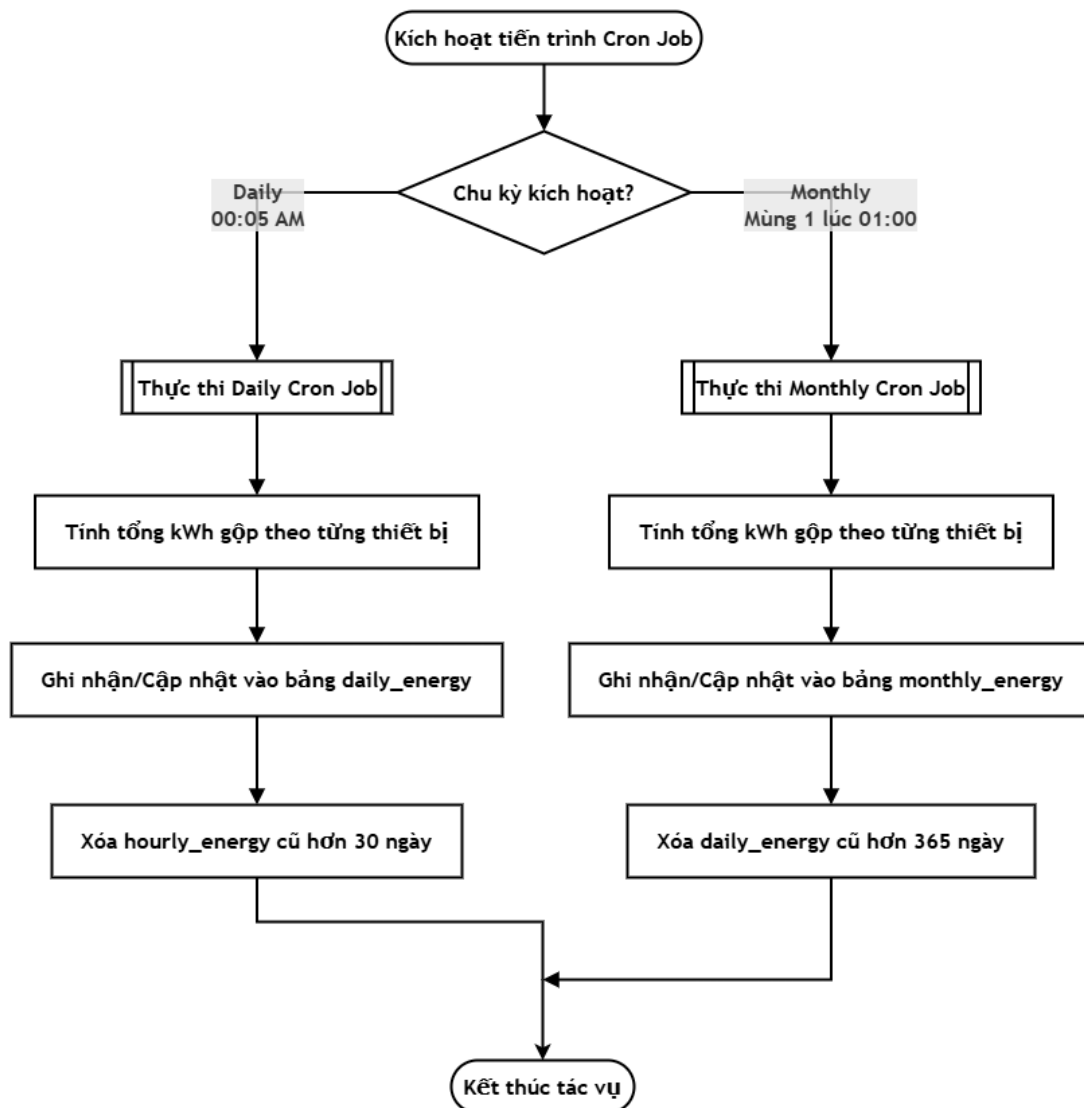


Hình 4.15: Lưu đồ giải thuật cảnh báo sự cố điện và gửi email tự động

Hệ thống nhúng liên tục cập nhật các thông số điện từ cảm biến PZEM-004T và thực hiện bảo vệ phụ tải nhiều cấp ngay tại phần cứng, gồm quá dòng khi dòng tải vượt mười lăm ampe kéo dài quá ba giây để tránh ngắt nhầm do dòng đỉnh lúc khởi động thiết bị, quá áp khi điện áp vượt hai trăm năm mươi vôn, và sụt áp khi điện áp xuống dưới một trăm sáu mươi vôn. Khi bất kỳ sự cố nào xảy ra, ESP32 chủ động xuất tín hiệu ngắt rơ-le ngay lập tức tại phần cứng mà không cần chờ lệnh từ máy chủ, đồng thời gửi bản tin MQTT chứa mã cảnh báo tương ứng cho từng loại sự cố lên broker. Máy chủ NestJS nhận bản tin khẩn cấp, ghi nhật ký sự kiện vào cơ sở dữ liệu, truy vấn danh sách người dùng thuộc ngôi nhà và gọi dịch vụ gửi email cảnh báo định dạng HTML chi tiết, chỉ rõ vị trí, thiết bị và thông số đo thực tế tại thời điểm xảy ra sự cố. Nhằm tránh hiện tượng làm tràn cơ sở dữ liệu và hòm thư khi sự cố kéo dài, máy chủ áp dụng cơ chế chờ tối thiểu năm phút giữa hai lần cảnh báo cho mỗi node, đồng thời người dùng có thể cấu hình tắt hoàn toàn cảnh báo hở tải từ phía phần cứng thông qua một cờ cấu hình trong tệp `dashboard_config.h`.

4.4.8 Lưu đồ giải thuật gom cụm và tích lũy điện năng ba cấp

Lưu đồ giải thuật luân chuyển và dọn dẹp dữ liệu điện năng ba cấp được thể hiện ở Hình 4.16.



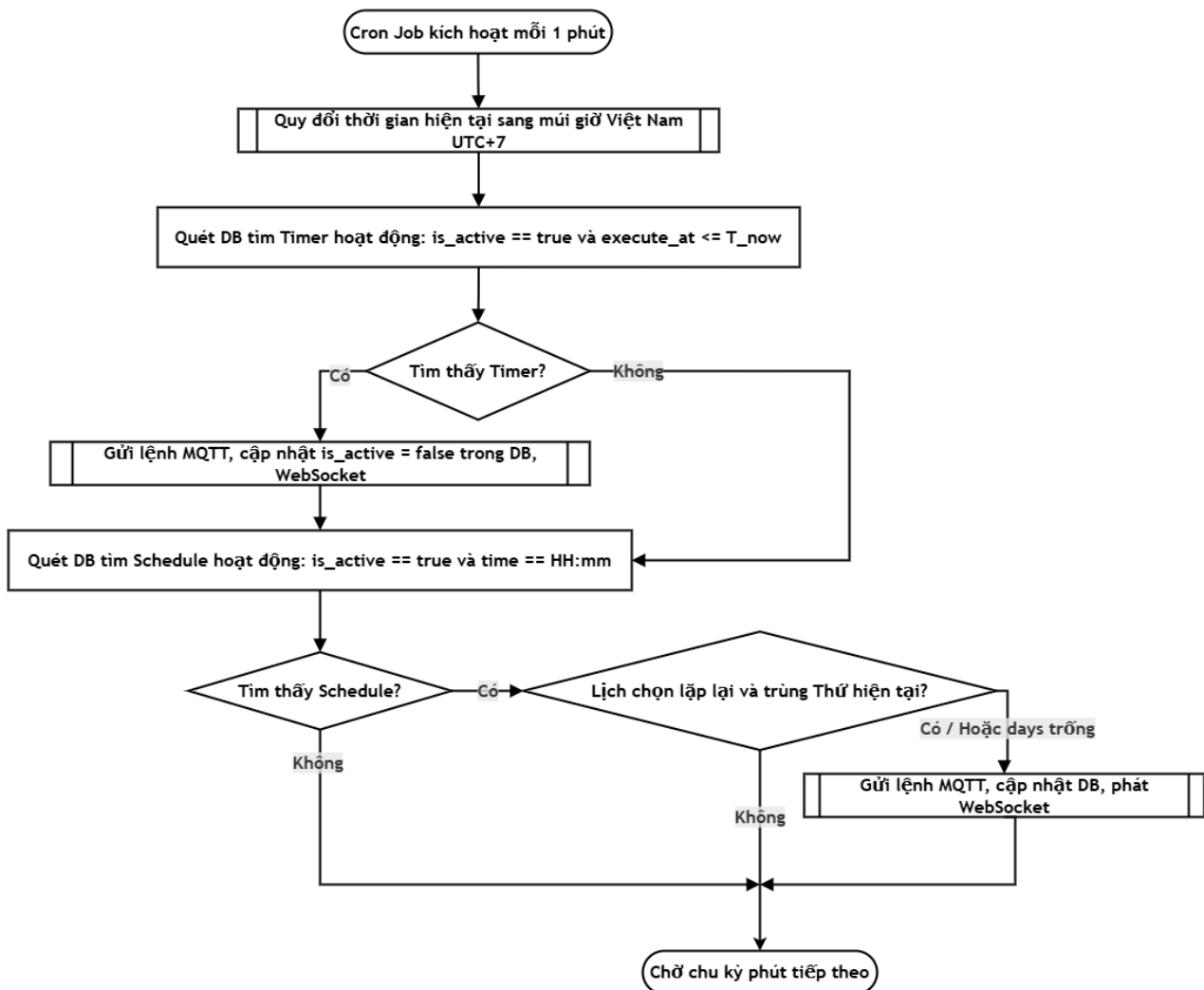
Hình 4.16: Lưu đồ giải thuật luân chuyển và dọn dẹp dữ liệu điện năng ba cấp

Lưu đồ mô tả hoạt động của hai tiến trình chạy ngầm theo lịch trong máy chủ NestJS để định kỳ gom dữ liệu điện năng và dọn dẹp các phân vùng cũ để tránh lãng phí dung lượng lưu trữ. Tiến trình hằng ngày được kích hoạt tự động vào lúc không giờ năm phút mỗi sáng, truy vấn toàn bộ dữ liệu tiêu thụ từ bảng năng lượng theo giờ của ngày hôm trước, tính tổng lượng điện năng gộp theo từng thiết bị và lưu vào bảng năng lượng theo ngày, sau đó xóa cuốn chiếu các bản ghi theo giờ cũ hơn ba mươi ngày. Tiến trình hằng tháng được kích hoạt vào lúc một giờ sáng ngày mùng một mỗi tháng, truy vấn dữ liệu theo ngày của tháng trước, tính tổng cộng dồn theo từng thiết bị

và cập nhật vào bảng năng lượng theo tháng lưu trữ vĩnh viễn, rồi xóa các bản ghi theo ngày cũ hơn ba trăm sáu mươi lăm ngày. Cơ chế gộp ba cấp này bảo đảm dữ liệu năng lượng nhất quán lâu dài trong khi dung lượng cơ sở dữ liệu luôn được giới hạn ổn định. Trong quá trình tích lũy, để lọc nhiễu công suất tiêu thụ nền của mạch và rơ-le khi không cảm tải, máy chủ áp dụng ngưỡng dòng tối thiểu hai mươi miliampe, còn firmware áp dụng vùng chết ép các giá trị đo dưới năm oát về không oát.

4.4.9 Lưu đồ giải thuật hẹn giờ và lên lịch tự động

Lưu đồ giải thuật xử lý tác vụ hẹn giờ và lên lịch tự động được thể hiện ở Hình 4.17.



Hình 4.17: Lưu đồ giải thuật xử lý tác vụ hẹn giờ và lên lịch tự động

Tiến trình chạy ngầm theo chu kỳ mỗi phút trên máy chủ lấy thời gian hiện hành, quy đổi về múi giờ Việt Nam rồi lần lượt xét hai loại tác vụ. Đối với bộ hẹn giờ đếm ngược, tiến trình tìm các bản ghi có mốc thực thi đã tới hoặc quá hạn để phát lệnh bật tắt rồi đánh dấu ngừng kích hoạt. Đối với bộ lập lịch lặp lại, tiến trình so khớp giờ phút và thứ trong tuần hiện tại với cấu hình đã lưu để phát lệnh đúng thời điểm. Mọi lệnh sinh ra đều được gửi tới thiết bị nhúng qua MQTT và đồng bộ về giao diện qua WebSocket để cập nhật nút bấm.

CHƯƠNG 5

NHẬN XÉT, ĐÁNH GIÁ KẾT QUẢ

Chương này trình bày kết quả kiểm thử thực tế của ổ cắm điện sau khi hoàn thiện phần cứng, lắp đặt trong hộp và kết nối với giao diện giám sát. Các ảnh được trình bày riêng từng hình để thể hiện rõ từng trạng thái, từng thiết bị đo và từng chức năng đã kiểm thử.

Trước khi trình bày kết quả đo và kiểm thử, Bảng 5.1 tổng hợp các chức năng chính mà hệ thống đã hiện thực, phân theo phần thực hiện trên firmware và trên web.

Bảng 5.1: Tổng kê các chức năng chính của hệ thống

STT	Chức năng	Phân hệ thực hiện
1	Đo 6 thông số điện (U, I, P, kWh, tần số, hệ số công suất) qua PZEM-004T	Firmware (ESP32)
2	Đóng/ngắt độc lập hai ổ cắm bằng relay hai kênh	Firmware (ESP32)
3	Điều khiển tại chỗ bằng hai nút nhấn vật lý, LED báo trạng thái	Firmware (ESP32)
4	Cấu hình WiFi qua chế độ Access Point; giữ hai nút trên 3 giây để đặt lại	Firmware (ESP32)
5	Kết nối MQTT (TLS) tới HiveMQ Cloud: gửi dữ liệu JSON, nhận lệnh và phản hồi ACK	Firmware (ESP32)
6	Tự bảo vệ an toàn điện: tự ngắt khi quá dòng, quá áp, sụt áp	Firmware (ESP32)
7	Đệm dữ liệu offline (LittleFS) khi mất mạng, đồng bộ lại khi có kết nối	Firmware (ESP32)
8	Tự kết nối lại WiFi/MQTT; đồng bộ thời gian theo NTP	Firmware (ESP32)
9	Giám sát thông số điện theo thời gian thực trên	Web/Cloud

	dashboard	
10	Bật/tắt thiết bị từ xa qua Internet	Web/Cloud
11	Quản lý đa người dùng, đa nhà; phân quyền theo vai trò (RBAC)	Web/Cloud
12	Đăng nhập bằng JWT và Google OAuth	Web/Cloud
13	Hẹn giờ bật/tắt và hẹn giờ đếm ngược	Web/Cloud
14	Lập lịch bật/tắt theo thứ trong tuần	Web/Cloud
15	Cảnh báo khi công suất vượt ngưỡng và gửi email	Web/Cloud
16	Thống kê điện năng theo giờ, ngày, tháng; biểu đồ tiêu thụ	Web/Cloud
17	Ước tính tiền điện theo biểu giá bậc thang của EVN	Web/Cloud
18	Quản lý cấu trúc: thêm nhà, thêm phòng, thêm thiết bị	Web/Cloud

5.1 BỐ TRÍ THÍ NGHIỆM VÀ TẢI KIỂM THỬ

Bố trí đo thực tế gồm ổ cắm, tải kiểm thử, công tơ điện tử mini, ampe kìm và đồng hồ vạn năng. Cách bố trí đo được thể hiện ở Hình 5.1.



Hình 5.1: Bố trí đo thực tế với thiết bị, tải và công tơ đối chứng

5.2 KIỂM THỬ TẢI ĐÈN, QUẠT VÀ TRẠNG THÁI Ổ CẮM

Nhóm kiểm thử các trạng thái thực tế của hai ổ cắm với đèn và quạt. Mục đích của phần này là chứng minh hai ổ cắm đều cấp được điện cho tải thật, relay đóng/ngắt đúng, nút nhấn vật lý hoạt động và LED báo đúng trạng thái. Các trạng thái kiểm thử được minh họa lần lượt ở Hình 5.2 đến Hình 5.5.



Hình 5.2: Trạng thái hai tải đều tắt



Hình 5.3: Trạng thái chỉ tải đèn hoạt động

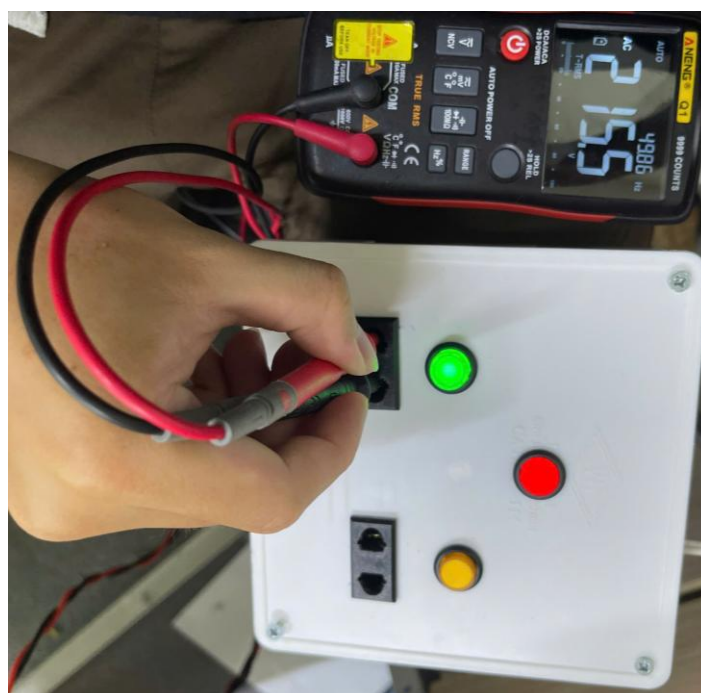


Hình 5.4: Trạng thái đèn và quạt cùng hoạt động

5.3 PHƯƠNG PHÁP ĐO VÀ XỬ LÝ SỐ LIỆU

Mỗi tải định lượng được đo 10 lần. Công tơ điện tử được dùng để đo chứng điện áp và công suất, ampe kìm được dùng để đo dòng điện, còn đồng hồ vạn năng True RMS được dùng để kiểm tra trực tiếp điện áp AC tại đầu ra ổ cắm của thiết bị và tại ổ điện lưới. Giá trị ampe kìm đã trừ offset khoảng 0,04 A theo ảnh đo nền trước khi kẹp tải. Sai số tương đối được tính bằng sai số tuyệt đối chia cho giá trị đo trung bình.

Ngoài công tơ và ampe kìm, nhóm dùng đồng hồ vạn năng True RMS để kiểm tra điện áp xoay chiều ở đầu ra của ổ cắm. Phép kiểm tra này để chứng minh khi relay đóng, ổ cắm trên thiết bị tạo ra điện áp tương đương ổ điện lưới thông thường. Kết quả được thể hiện ở Hình 5.6 và Hình 5.7.



Hình 5.6: Đo điện áp đầu ra tại ổ cắm của thiết bị bằng đồng hồ vạn năng



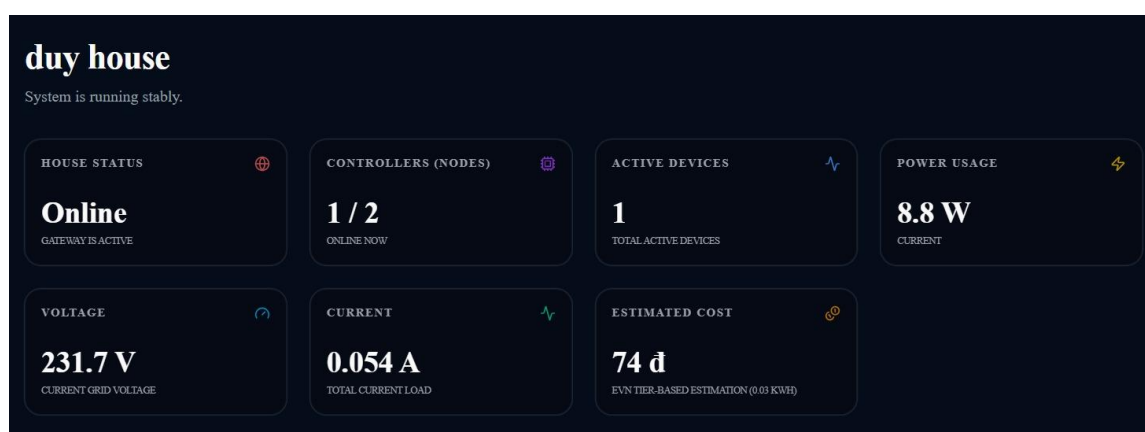
Hình 5.7: Đo điện áp tại ổ điện lưới để đối chiếu với đầu ra thiết bị

Từ hai ảnh đo, điện áp đầu ra của thiết bị khoảng 215,6 V, trong khi ổ điện lưới đo được khoảng 215,4 V. Sai lệch khoảng 0,2 V, tương đương xấp xỉ 0,09%. Kết quả

này cho thấy đường cáp AC qua relay và domino của thiết bị không gây sụt áp đáng kể trong điều kiện kiểm tra, nên tải cắm vào ổ của thiết bị nhận được điện áp tương đương ổ điện thường.

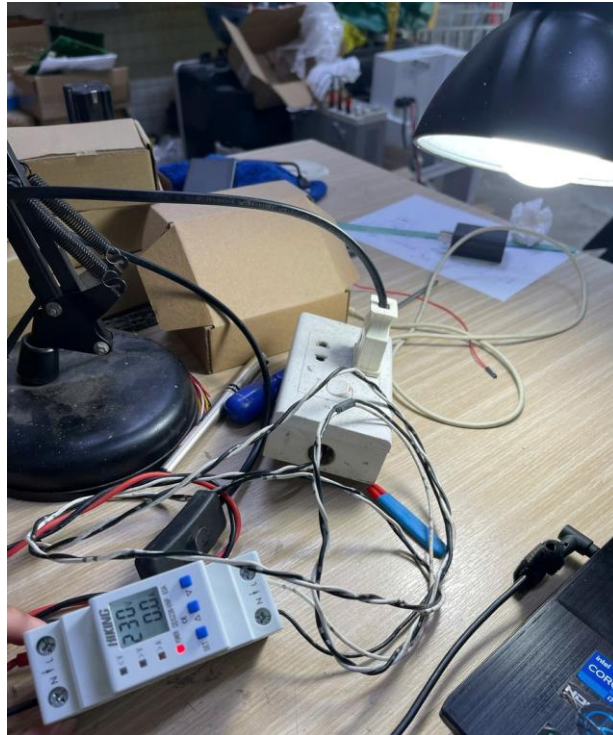
5.4 KẾT QUẢ ĐO TẢI 1 - ĐÈN BÀN

Khi cấp tải 1, dashboard nhận dữ liệu từ PZEM và hiển thị điện áp 231,7 V, dòng điện 0,054 A và công suất 8,8 W tại thời điểm chụp như Hình 5.8.

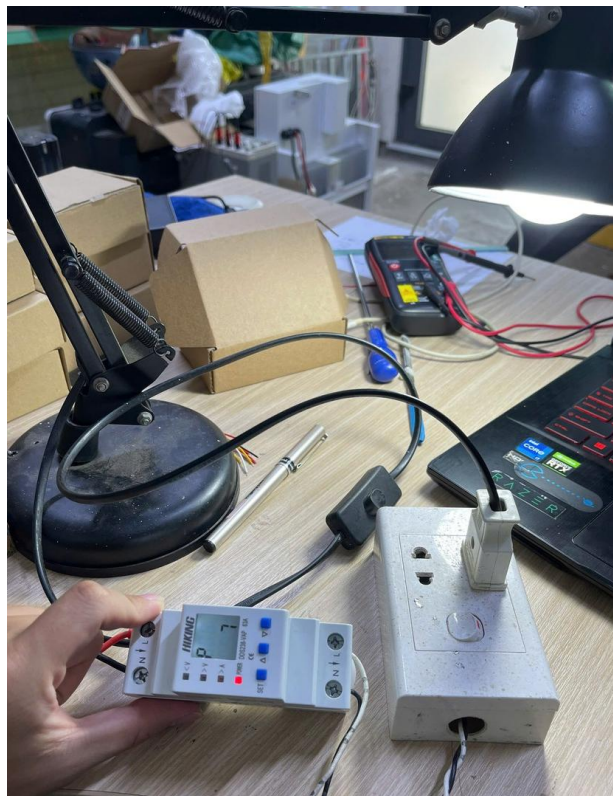


Hình 5.8: Dashboard hiển thị thông số PZEM khi đo tải 1

Các thiết bị đối chứng cho tải 1 được trình bày riêng ở Hình 5.9, Hình 5.10, Hình 5.11 và Hình 5.12 để thể hiện rõ từng phép đo điện áp, công suất, dòng điện và giá trị offset của ampe kìm.



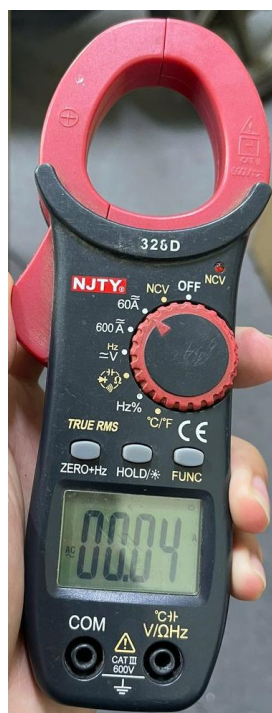
Hình 5.9: Công tơ đối chứng điện áp khi đo tải 1



Hình 5.10: Công tơ đối chứng công suất khi đo tải 1



Hình 5.11: Ampe kìm đối chứng dòng điện khi đo tải 1



Hình 5.12: Giá trị nền của ampe kìm trước khi đo tải

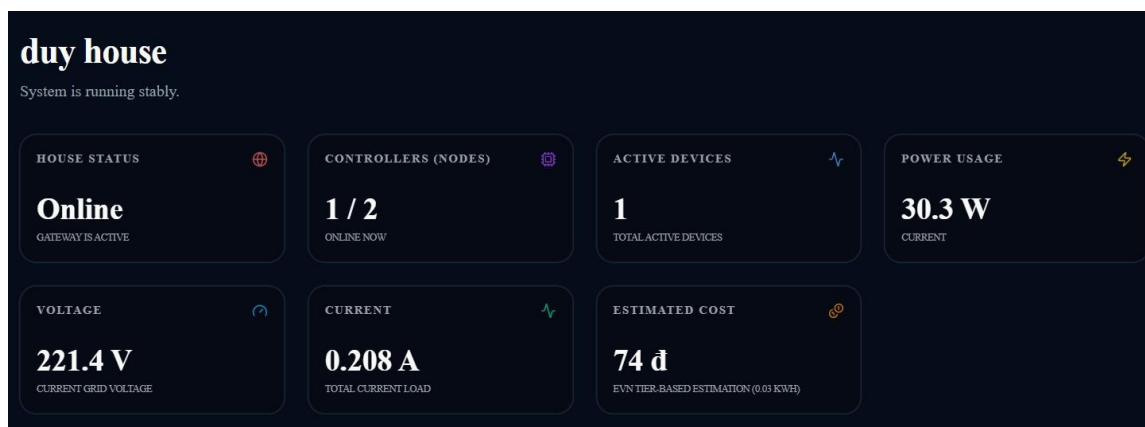
Do ampe kìm có giá trị nền là 0.04A nên các phép đo sử dụng ampe kìm sẽ phải giảm bớt đi 0.04A trước khi sử dụng số liệu, số liệu đo lặp của tải 1 được tổng hợp ở Bảng 5.2.

Bảng 5.2: Kết quả 10 lần đo điện áp, dòng điện và công suất với tải 1

Lần	PZEM V (V)	PZEM I (A)	PZEM P (W)	Công tơ V (V)	Công tơ P (W)	Ampe kìm I (A)
1	231.1	0.055	9	231	8	0.06
2	231.1	0.055	9	230	7	0.056
3	230	0.055	8.9	230	7	0.051
4	229.8	0.055	8.9	231	7	0.06
5	231.6	0.054	8.8	231	7	0.059
6	230.8	0.053	8.6	231	8	0.056
7	231.5	0.054	8.8	230	7	0.054
8	230.6	0.054	8.8	231	8	0.053
9	230.3	0.054	8.8	230	7	0.056
10	231.7	0.054	8.8	230	7	0.056

5.5 KẾT QUẢ ĐO TẢI 2 - ĐÈN CÔNG SUẤT LỚN

Với tải 2, dashboard hiển thị điện áp 221,4 V, dòng điện 0,208 A và công suất 30,3 W tại thời điểm chụp như Hình 5.13.

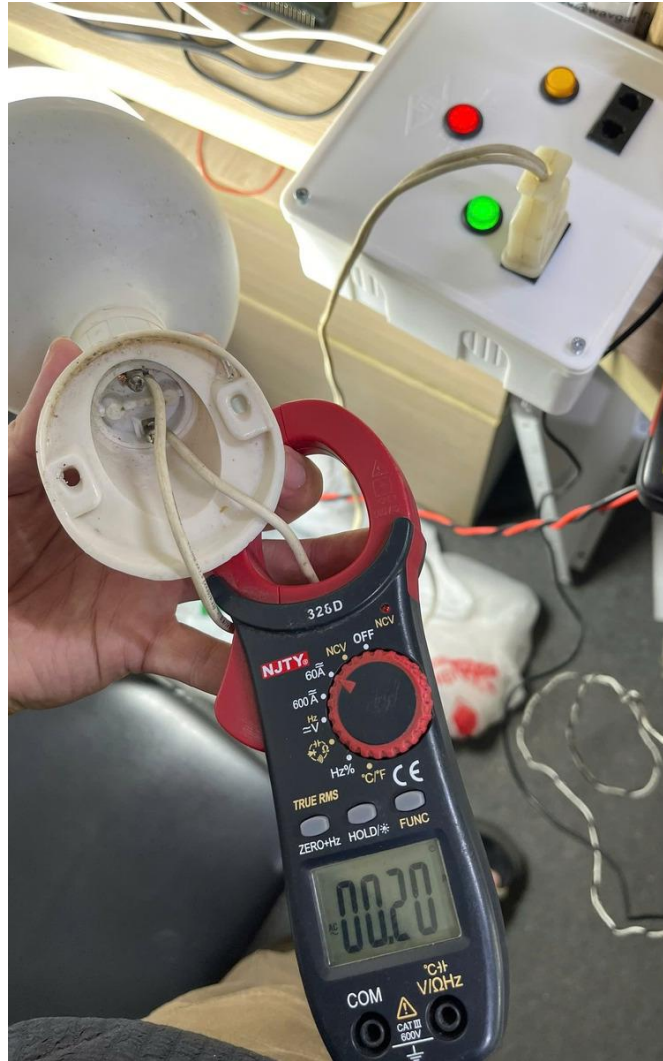


Hình 5.13: Dashboard hiển thị thông số PZEM khi đo tải 2

Hình 5.14 đến Hình 5.15 trình bày riêng từng ảnh đối chứng của tải 2, gồm công tơ đo công suất và ampe kìm đo dòng điện.



Hình 5.14: Công tơ đối chứng công suất khi đo tải 2



Hình 5.15: Ampe kìm đối chứng dòng điện khi đo tải 2

Số liệu đo lặp của tải 2 được tổng hợp ở Bảng 5.3.

Bảng 5.3: Kết quả 10 lần đo điện áp, dòng điện và công suất với tải 2

Lần	PZEM V (V)	PZEM I (A)	PZEM P (W)	Công tơ V (V)	Công tơ P (W)	Ampe kìm I (A)
1	220.3	0.21	30.4	221	30	0.197
2	220.6	0.211	30.6	220	29	0.193
3	220.3	0.211	30.5	220	29	0.198

4	219.9	0.208	30.1	221	30	0.205
5	221.4	0.21	30.5	220	30	0.215
6	221.6	0.211	30.7	221	29	0.212
7	221.9	0.209	30.5	221	29	0.193
8	219.4	0.212	30.6	220	30	0.212
9	221	0.209	30.3	221	29	0.205
10	221.4	0.208	30.3	221	29	0.206

5.6 KIỂM CHỨNG ĐIỆN NĂNG TIÊU THỤ CỦA TẢI 2 TRONG 1 GIỜ

Sau khi đo công suất tức thời của tải 2, nhóm tiếp tục cắm tải 2 trong khoảng 1 giờ và theo dõi biểu đồ điện năng trên web. Mục đích của phép kiểm này là kiểm tra sơ bộ xem giá trị kWh trên dashboard có cùng bậc với giá trị tính từ công suất đo được hay không. Kết quả hiển thị trên web được thể hiện ở Hình 5.16.



Hình 5.16: Biểu đồ web hiển thị điện năng tiêu thụ của tải 2 sau khoảng 1 giờ

Từ Bảng 5.3, công suất trung bình của tải 2 do PZEM đo được là 30,45 W. Nếu tải hoạt động liên tục trong 1 giờ, điện năng lý thuyết xấp xỉ 30,45 Wh, tương đương 0,03045 kWh. Trên web, giá trị tại thời điểm kiểm tra là 0,0335 kWh. Chênh lệch tuyệt đối khoảng 0,00305 kWh, tương đương 10,02% so với giá trị tính từ công suất trung bình.

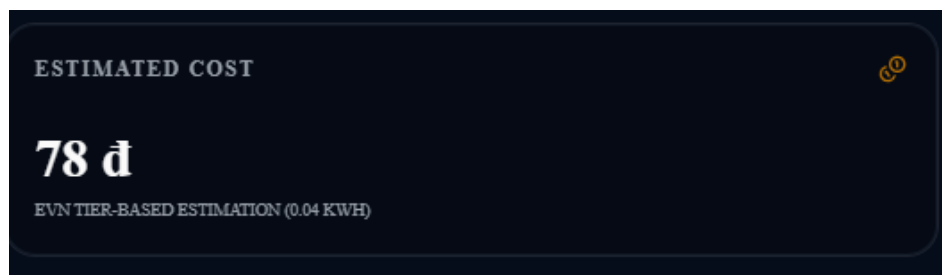
Bảng 5.4: Kiểm chứng điện năng tiêu thụ tải 2 trong khoảng 1 giờ

Đại lượng	Giá trị	Cách xác định	Nhận xét
Công suất trung bình tải 2	30,45 W	Trung bình 10 lần đo PZEM ở Bảng 5.3	Dùng làm mốc tính toán
Thời gian kiểm tra	1 giờ	Quan sát theo thời gian cắm tải	Tải 2 hoạt động liên tục
Điện năng tính theo công suất	0,03045 kWh	$30,45 \text{ W} \times 1 \text{ h} / 1000$	Giá trị lý thuyết
Điện năng hiển thị trên web	0,0335 kWh	Hình 5.16	Giá trị ghi nhận từ dashboard
Sai lệch	0,00305 kWh (10,02%)	$ 0,0335 - 0,03045 $	Chấp nhận được cho kiểm chứng sơ bộ

Sai lệch trong Bảng 5.4 có thể đến từ việc công suất tải không hoàn toàn cố định trong suốt 1 giờ, thời điểm đọc trên web không trùng tuyệt đối với thời điểm bắt đầu/kết thúc đo, và giá trị 30,45 W là trung bình của một loạt đo ngắn. Vì vậy, phép kiểm này được dùng để chứng minh chức năng ghi nhận kWh trên web hoạt động hợp lý, còn đánh giá chính xác hơn cần đo bằng công tơ chuẩn trong cùng khoảng thời gian.

Ngoài điện năng tiêu thụ, dashboard còn hiển thị chi phí điện ước tính từ lượng kWh ghi nhận. Sau khi chạy tải 2 khoảng 1 giờ, web hiển thị chi phí ước tính 78 đ như

Hình 5.17. Dòng mô tả trên giao diện cho biết phép tính đang dựa trên khoảng 0,04 kWh.



Hình 5.17: Giá tiền điện ước tính trên web sau khi chạy tải 2 khoảng 1 giờ

Bảng 5.5: Kiểm tra giá tiền điện ước tính sau khi chạy tải 2

Đại lượng	Giá trị	Cách xác định	Nhận xét
Điện năng dùng để tính tiền	0,04 kWh	Hiển thị dưới thẻ Estimated Cost	Giá trị đã làm tròn trên web
Chi phí web hiển thị	78 đ	Hình 5.17	Kết quả ước tính của dashboard
Đơn giá suy ra	1.950 đ/kWh	$78 / 0,04$	Phù hợp mức giá bậc thấp dùng để ước tính
So với kWh ở Hình 5.16	0,0335 kWh	Tooltip biểu đồ	Khác biệt do dashboard làm tròn khi tính/hiển thị chi phí

Kết quả ở Bảng 5.5 cho thấy chức năng tính tiền điện đã hoạt động cùng với dữ liệu kWh. Giá trị tiền điện chỉ có ý nghĩa ước tính, vì hệ thống đang dùng lượng điện năng nhỏ trong thời gian ngắn và hiển thị kWh theo dạng làm tròn. Khi chạy tải lâu hơn hoặc dùng tổng điện năng theo tháng, sai số do làm tròn sẽ giảm.

Đơn giá suy ra khoảng 1.950 đ/kWh nằm trong khoảng các bậc thấp của biểu giá bán lẻ điện sinh hoạt do Tập đoàn Điện lực Việt Nam ban hành. Vì đây là mức giá ở bậc thấp với lượng điện nhỏ trong thời gian ngắn nên chi phí ước tính chỉ mang tính

minh họa chức năng tính tiền; để chặt chẽ, nên trích dẫn quyết định biểu giá điện của EVN mà phần web dùng làm căn cứ tính toán.

5.7 SO SÁNH SAI SỐ ĐO LƯỜNG

Bảng 5.6: Giá trị trung bình và sai số của hai tải thử nghiệm

Thông số	Tải 1 - đèn bàn	Tải 2 - đèn công suất lớn
PZEM V trung bình	230.85 V	220.78 V
Công tơ V trung bình	230.5 V	220.6 V
Sai số điện áp	0.35 V (0.15%)	0.18 V (0.08%)
PZEM I trung bình	0.0543 A	0.2099 A
Ampe kìm I trung bình	0.0561 A	0.2036 A
Sai số dòng điện	0.0018 A (3.21%)	0.0063 A (3.09%)
PZEM P trung bình	8.84 W	30.45 W
Công tơ P trung bình	7.3 W	29.4 W
Sai số công suất	1.54 W (21.1%)	1.05 W (3.57%)

Bảng 5.6 cho thấy sai số điện áp rất nhỏ, khoảng 0,15% với tải 1 và 0,08% với tải 2, chứng tỏ PZEM đo điện áp rất sát công tơ. Về công suất, cần lưu ý rằng PZEM đo trên toàn bộ đường dây nên bao gồm cả phần điện tự tiêu thụ của thiết bị, trong khi công tơ chỉ đo riêng phần tải cắm vào ổ ra. Phần tự tiêu thụ này đã được đo riêng, gồm khoảng 0,7W ở chế độ chờ và thêm khoảng 0,5W cho mỗi relay đang đóng, tức khoảng 1,2W khi một ổ hoạt động. Do đó chênh lệch công suất giữa PZEM và công tơ là 1,54W ở tải 1 và 1,05W ở tải 2 chính là phần điện tự tiêu thụ đó, không phải sai số đo của PZEM.

Sau khi trừ phần tự tiêu thụ khoảng 1,2W, công suất tải do PZEM đo được ở tải 2 vào khoảng 29,3W, rất sát giá trị 29,4W của công tơ, tương ứng sai số chỉ khoảng

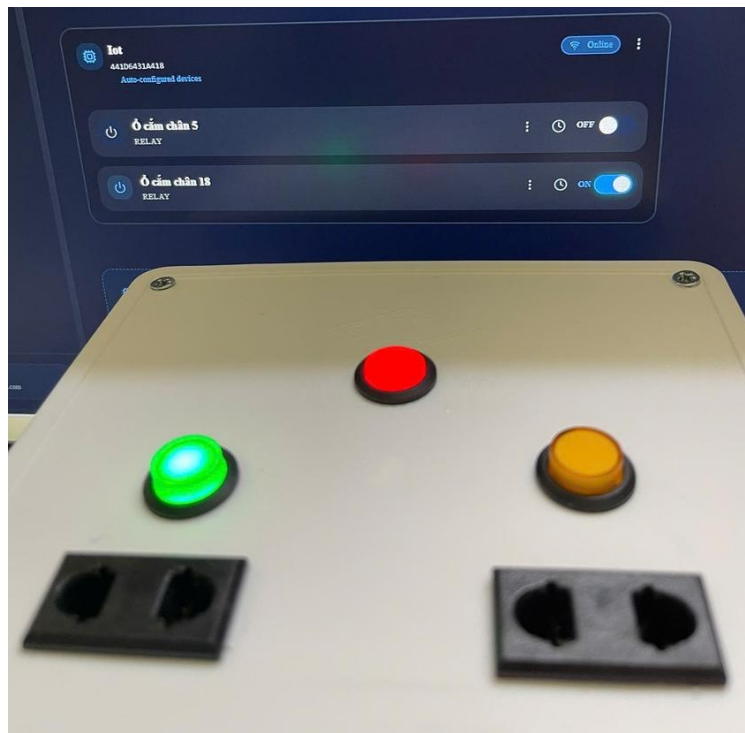
0,5%; ở tải 1 vào khoảng 7,6W so với 7,3W của công tơ, sai số vài phần trăm. Phần lệch còn lại ở tải 1 một phần đến từ việc công tơ chỉ hiển thị công suất theo bước 1W, nên trong vùng vài watt bản thân công tơ cũng có sai số lượng tử đáng kể. Như vậy độ chính xác đo công suất của PZEM là tốt, và thể hiện rõ nhất ở tải lớn nơi phần tự tiêu thụ chiếm tỉ lệ nhỏ.

Về dòng điện, ampe kìm được dùng để đối chứng, nhưng ở dòng rất nhỏ như tải 1 chỉ khoảng 0,054A thì giá trị này nằm gần đáy thang đo của ampe kìm nên dao động mạnh và phải trừ một lượng offset khá lớn so với chính giá trị đo. Vì vậy ở tải nhỏ, độ tin cậy được đặt chủ yếu vào sự khớp giữa PZEM và công tơ hơn là vào ampe kìm. Ngoài ra, hai tải thử nghiệm là đèn LED có mạch điện tử nên không thuần trở, hệ số công suất chỉ vào khoảng 0,66 đến 0,70; điều này giải thích vì sao tích điện áp nhân dòng điện lớn hơn công suất đo được, đồng thời cho thấy việc PZEM đo được công suất thực của tải phi tuyến là có ý nghĩa.

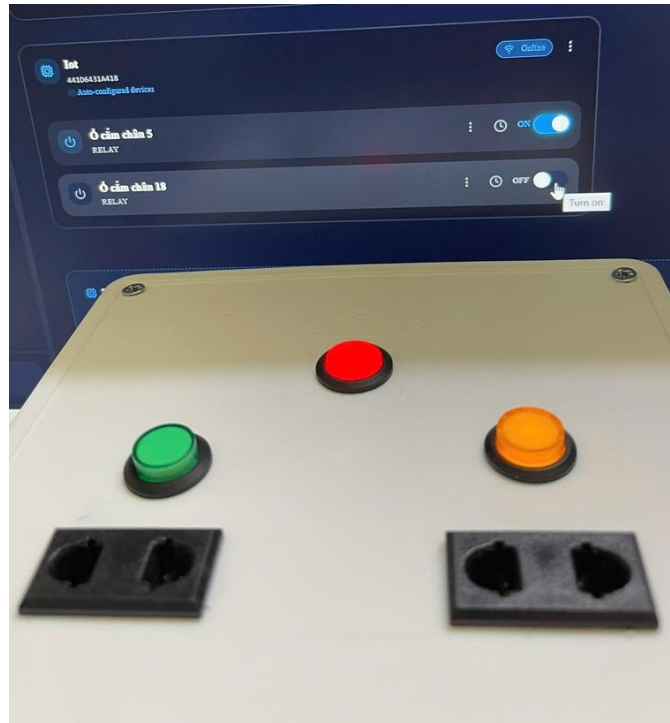
Xét tính lặp lại của phép đo, độ lệch chuẩn công suất PZEM qua mười lần đo chỉ khoảng 0,12W với tải 1 và 0,18W với tải 2, còn độ lệch chuẩn điện áp khoảng 0,67V và 0,81V, cho thấy sai số ngẫu nhiên nhỏ và số liệu ổn định qua các lần đo.

5.8 KIỂM THỬ BẬT/TẮT TỪ DASHBOARD, NÚT NHẤN VÀ RELAY

Chuỗi ảnh từ Hình 5.18 đến Hình 5.21 thể hiện quá trình bật/tắt trên dashboard, trạng thái LED/relay trên thiết bị và sự đồng bộ trạng thái sau thao tác điều khiển.



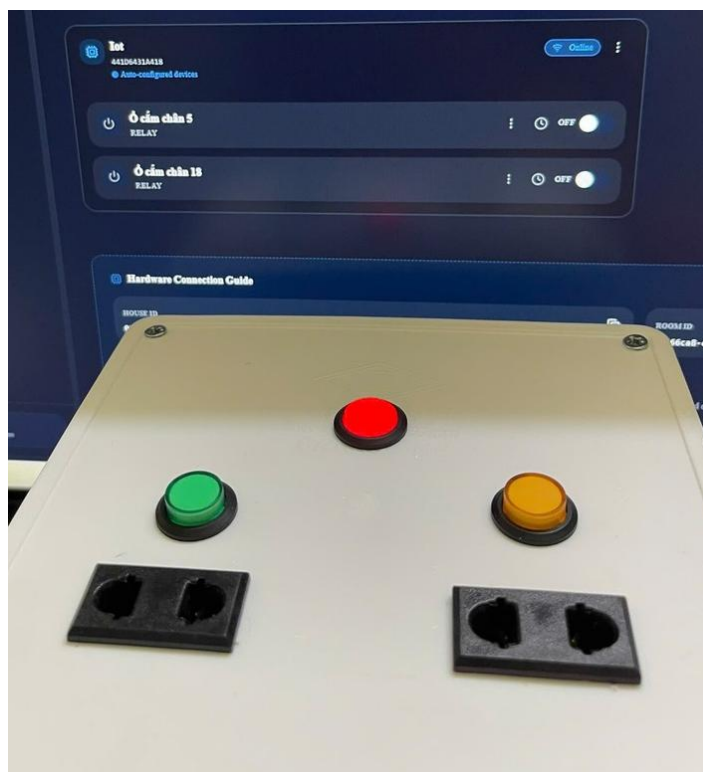
Hình 5.18: Dashboard hiển thị trạng thái ổ cắm trước khi thao tác



Hình 5.19: Thao tác đổi trạng thái thiết bị trên dashboard



Hình 5.20: Dashboard cập nhật trạng thái sau khi điều khiển



Hình 5.21: Thiết bị hoạt động đồng thời với giao diện giám sát

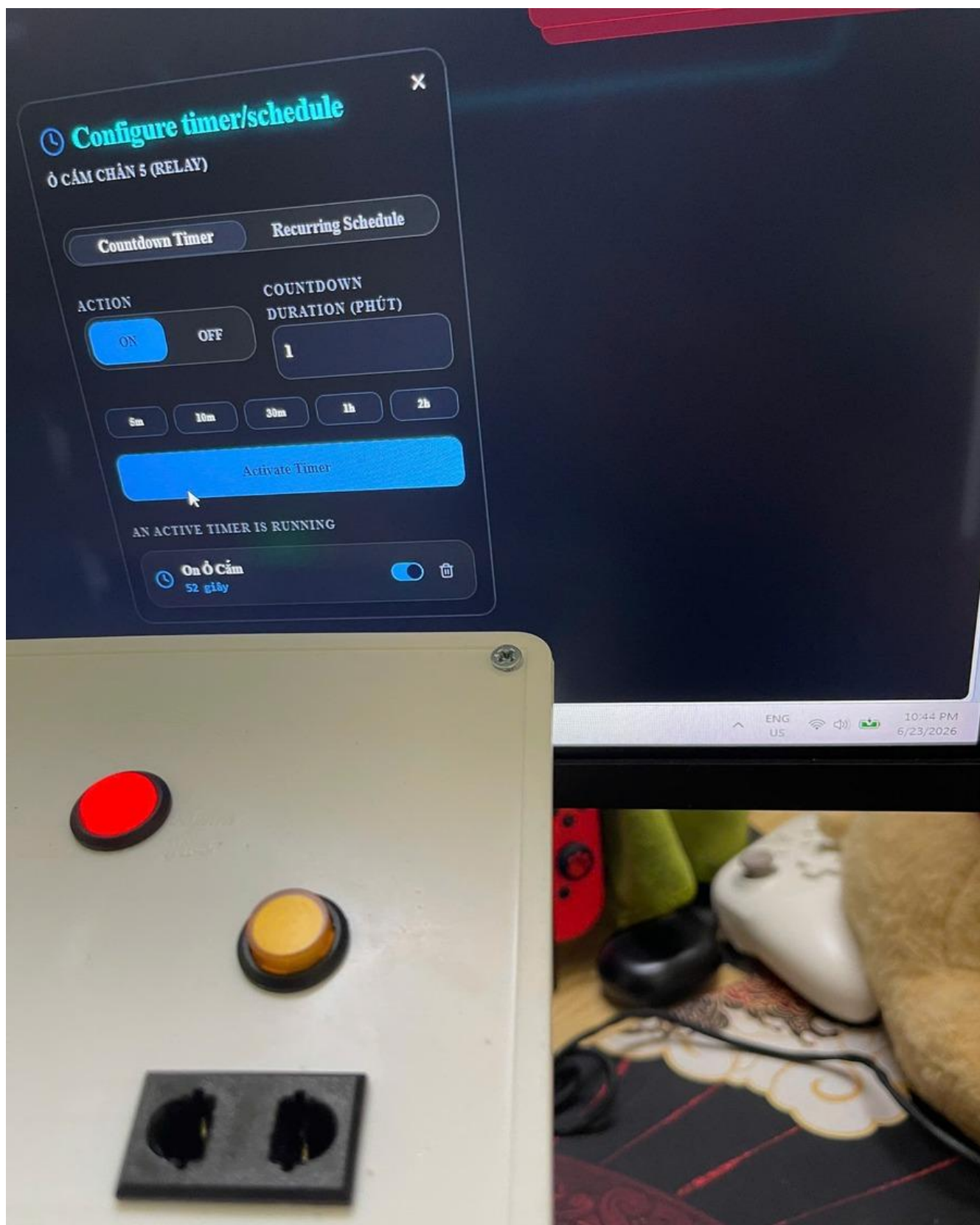
Kết quả kiểm thử bật/tắt và đồng bộ trạng thái được tổng hợp ở Bảng 5.7.

Bảng 5.7: Kết quả kiểm thử bật/tắt và đồng bộ trạng thái

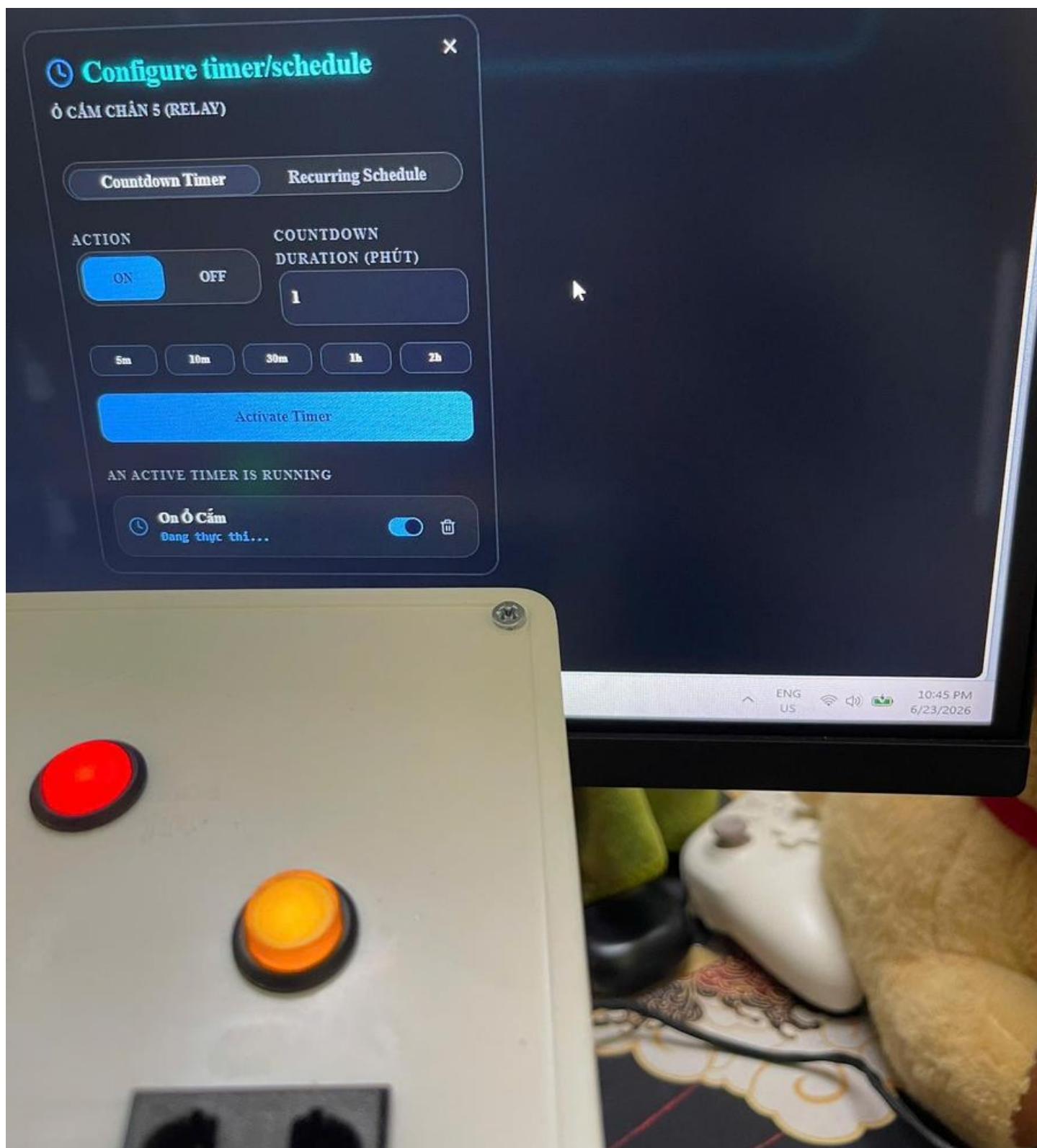
Chức năng	Số lần thử	Dấu hiệu trên thiết bị	Phản hồi dashboard	Kết luận
Bật/tắt ổ 1 từ web	10	Relay click, LED 1 đổi trạng thái	0,7-1,8 s khi gần WiFi	Đạt
Bật/tắt ổ 2 từ web	10	Relay click, LED 2 đổi trạng thái	0,7-1,8 s khi gần WiFi	Đạt
Bấm nút vật lý ổ 1	10	Relay click, LED 1 đổi trạng thái	Cập nhật lên web	Đạt
Bấm nút vật lý ổ 2	10	Relay click, LED 2 đổi trạng thái	Cập nhật lên web	Đạt
Bật/tắt tải đèn-quạt	Quan sát thực tế	Tải sáng/tắt đúng kênh	Thông số tải thay đổi theo	Đạt

5.9 KIỂM THỬ HẸN GIỜ VÀ LỊCH BẬT/TẮT

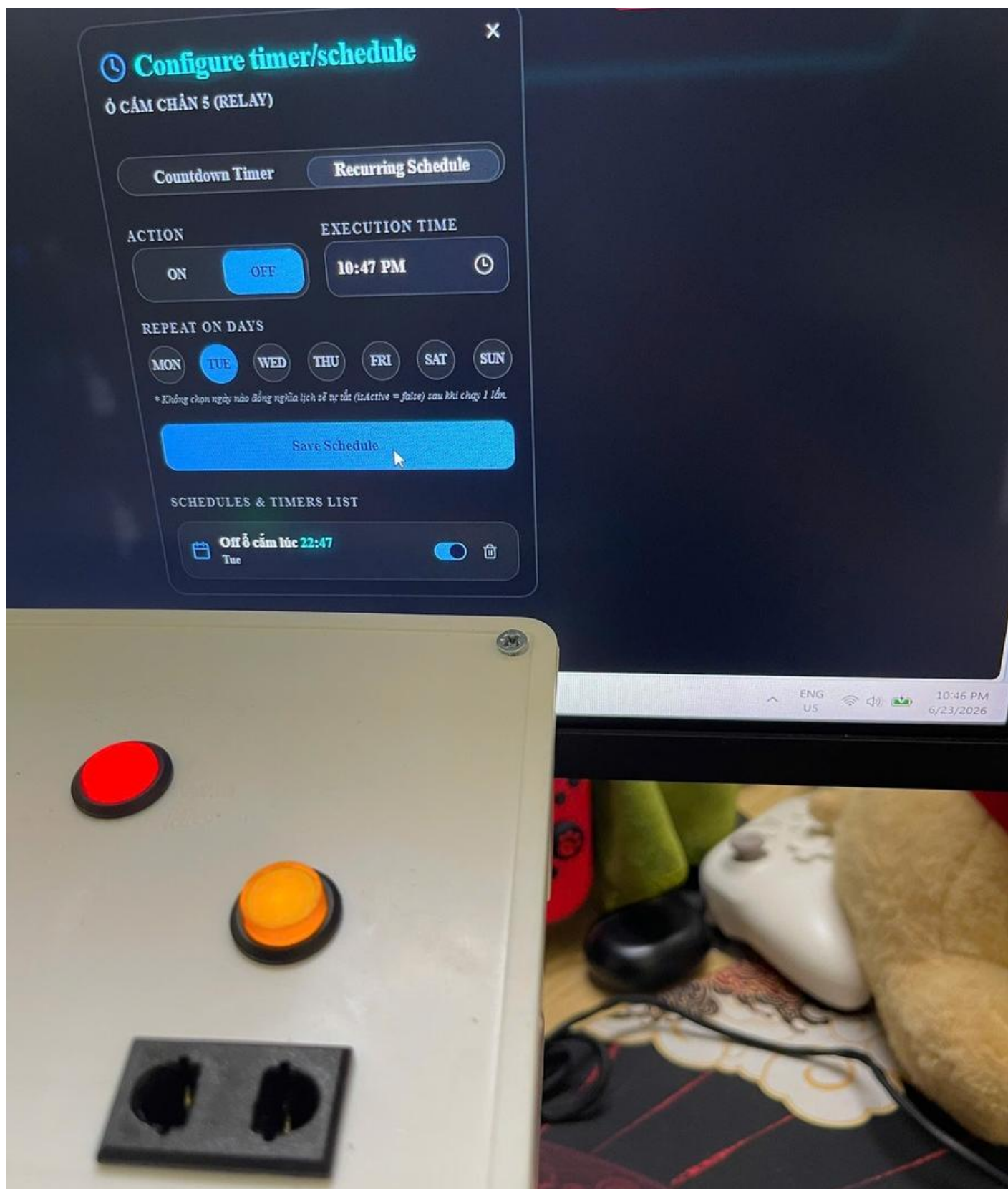
Hình 5.22 và Hình 5.23 thể hiện quá trình kiểm thử hẹn giờ bật thiết bị. Hình 5.24 và Hình 5.25 thể hiện kiểm thử lịch tắt. Các ảnh này chứng minh chức năng đã được chạy trên thiết bị thật; nếu cần đánh giá sai lệch thời gian chính xác thì nên bổ sung video.



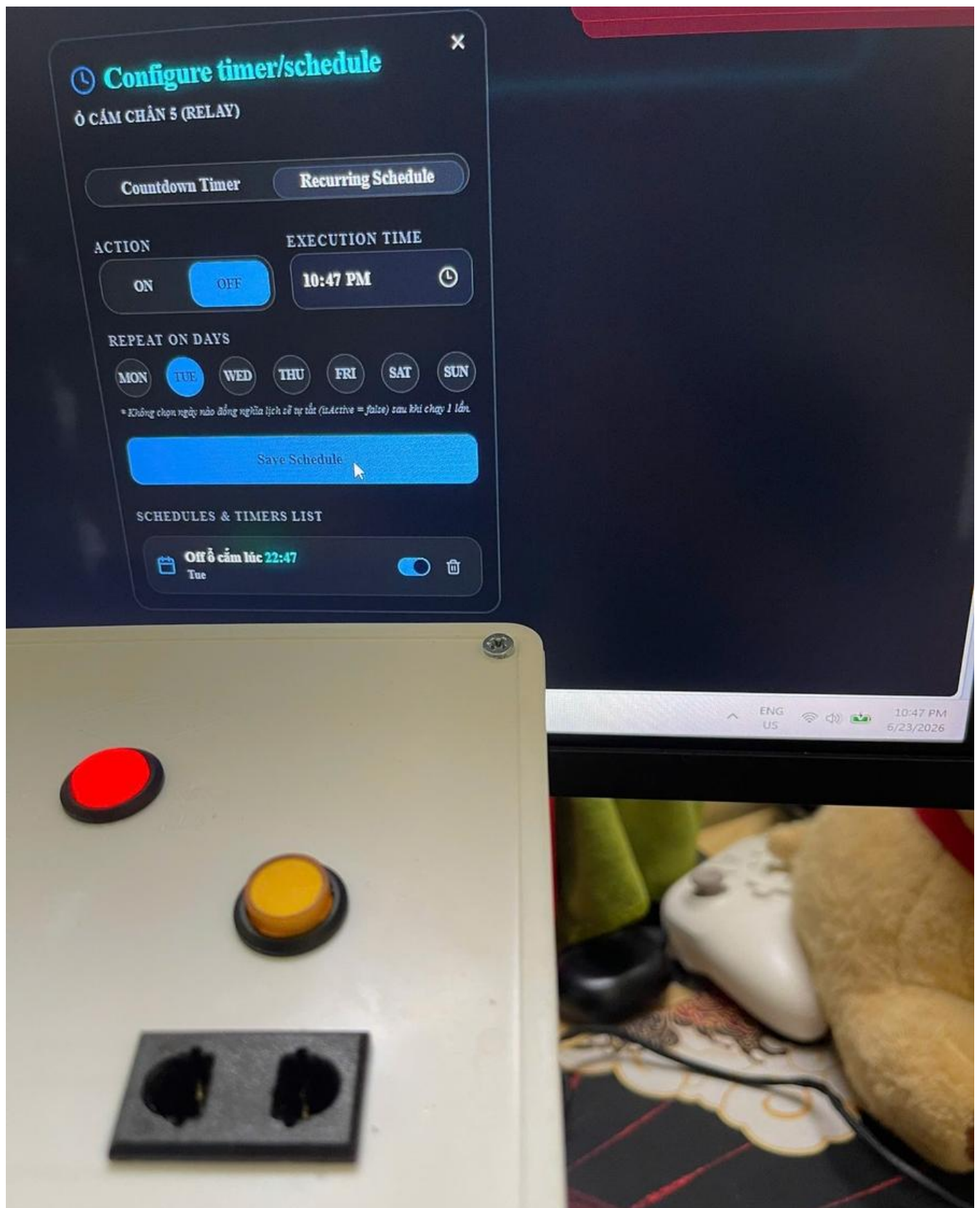
Hình 5.22: Thiết lập hẹn giờ bật trên dashboard



Hình 5.23: Trạng thái thiết bị sau khi kiểm thử hẹn giờ bật



Hình 5.24: Thiết lập lịch tắt trên dashboard



Hình 5.25: Trạng thái thiết bị sau khi kiểm thử lịch tắt

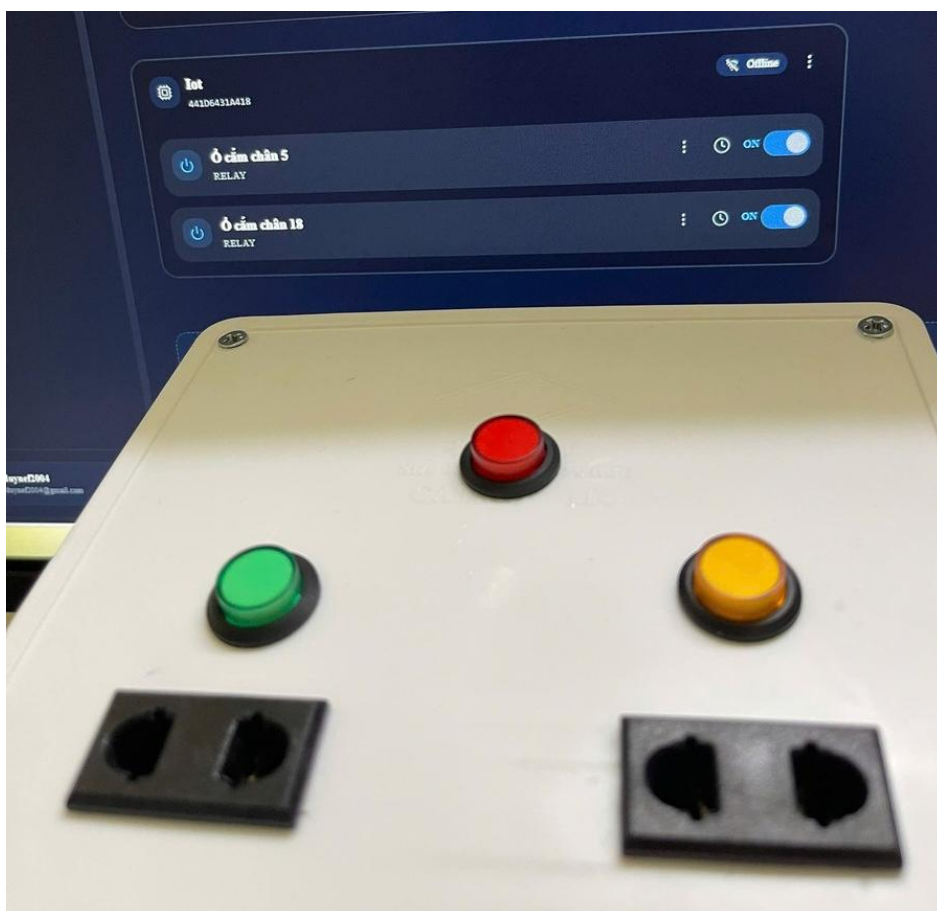
Kết quả kiểm thử hẹn giờ và lịch bật/tắt được tổng hợp ở Bảng 5.8.

Bảng 5.8: Kết quả kiểm thử hẹn giờ và lịch bật/tắt

Chức năng	Kịch bản thử	Minh chứng	Kết quả	Ghi chú
Hẹn giờ bật	Thiết lập thời điểm bật cho một kênh	Hình 5.22, Hình 5.23	Thiết bị đổi trạng thái	Cần video nếu muốn đo sai lệch thời gian chính xác
Lịch tắt	Thiết lập lịch tắt theo thời điểm	Hình 5.24, Hình 5.25	Thiết bị đổi trạng thái	Dashboard cập nhật trạng thái sau khi thực thi
Đồng bộ trạng thái	Theo dõi LED, relay và dashboard	Hình 5.22- Hình 5.25	Trạng thái khớp	Ảnh minh chứng chức năng đã được chạy thử

5.10 ĐỘ TRỄ ĐIỀU KHIỂN VÀ KHẢ NĂNG KẾT NỐI LẠI

Độ trễ điều khiển được đánh giá bằng quan sát thời điểm relay/LED đổi trạng thái so với thao tác trên dashboard hoặc nút vật lý. Hình 5.26 là ảnh trong quá trình kiểm thử độ trễ và khả năng kết nối lại của thiết bị. Kết quả quan sát độ trễ và khả năng kết nối lại được tổng hợp ở Bảng 5.9.



Hình 5.26: Theo dõi trạng thái thiết bị khi kiểm thử độ trễ và kết nối lại

Bảng 5.9: Kết quả quan sát độ trễ và khả năng kết nối lại

Điều kiện	Chiều kiểm thử	Thời gian quan sát	Nhận xét
Gần WiFi	Dashboard -> thiết bị	0,7-1,8 s	Relay và LED phản hồi nhanh
Gần WiFi	Thiết bị -> dashboard	0,7-1,8 s	Trạng thái cập nhật gần như tức thời
WiFi yếu/xa	Dashboard -> thiết bị	4-5 s	Độ trễ tăng do tín hiệu yếu
WiFi yếu/xa	Thiết bị -> dashboard	4-5 s	Dashboard cập nhật chậm hơn

Mất kết nối gần WiFi	Reconnect	Khoảng 30 s	Thiết bị tự kết nối lại
Mất kết nối xa WiFi	Reconnect	Có thể đến khoảng 1 phút	Phụ thuộc cường độ tín hiệu

Cần lưu ý các giá trị độ trễ trong Bảng 5.9 được ước lượng thủ công bằng đồng hồ bấm giây trên điện thoại nên đã bao gồm cả thời gian phản ứng của người đo, khoảng 0,2 đến 0,3 giây, và chỉ mang tính tham khảo về bậc độ lớn. Trên thực tế khi ở gần WiFi, độ trễ gần như tức thì và nhỏ hơn con số ước lượng này. Để có số liệu chính xác hơn có thể quay video tốc độ cao rồi đếm khung hình, đây là hướng hoàn thiện được đề xuất.

5.11 NHẬN XÉT VÀ ĐÁNH GIÁ TỔNG QUÁT

Qua các phép kiểm thử, thiết bị đáp ứng được các yêu cầu chính: đo điện áp, dòng điện, công suất; cấp điện được cho tải thật ở cả hai ổ cắm; đóng/ngắt hai kênh relay; LED báo đúng trạng thái; điều khiển từ dashboard và nút nhấn; thực hiện hẹn giờ/lich bật tắt ở mức kiểm chứng bằng ảnh; và có khả năng kết nối lại khi WiFi gián đoạn.

Hạn chế hiện tại là một số chức năng theo thời gian như hẹn giờ và lịch bật/tắt mới được chứng minh bằng ảnh tĩnh. Để báo cáo cuối thuyết phục hơn, nên bổ sung video demo hoặc bảng đo số lần chạy lặp cho từng chức năng theo thời gian.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế và thi công, đề tài đã hoàn thành được mục tiêu đặt ra ban đầu là xây dựng một ổ cắm điện có khả năng giám sát điện năng và điều khiển thiết bị từ xa qua Internet. Hệ thống hoạt động ổn định qua thời gian chạy thử, đáp ứng được các chức năng cơ bản của một thiết bị nhà và cho thấy tính khả thi khi triển khai thực tế.

Về phần cứng, nhóm đã thiết kế hoàn chỉnh sơ đồ nguyên lý và mạch in hai lớp bằng phần mềm Altium Designer, sau đó đặt gia công và lắp ráp thành sản phẩm thực tế. Mạch tích hợp vi điều khiển ESP32 làm trung tâm xử lý, module đo PZEM-004T để đo các thông số điện, relay hai kênh đóng ngắt độc lập hai ổ cắm, khối nguồn HLK-PM01 kết hợp AMS1117 cấp nguồn ổn định cho vi điều khiển, cùng mạch nạp CH340 và các nút nhấn, đèn báo trạng thái. Toàn bộ được đặt trong vỏ hộp bảo vệ, bố trí tách vùng công suất cao và vùng tín hiệu để bảo đảm an toàn.

Về phần mềm nhúng, firmware trên ESP32 đã thực hiện được việc kết nối WiFi theo cơ chế không chặn, cấu hình qua chế độ điểm truy cập khi chưa có mạng, kết nối tới máy chủ MQTT HiveMQ Cloud có mã hóa, đọc các thông số điện từ cảm biến PZEM-004T và công bố dưới dạng JSON theo cơ chế ngưỡng chênh lệch để tiết kiệm băng thông, nhận và phản hồi lệnh điều khiển từ xa, đồng bộ thời gian theo giao thức NTP. Thiết bị còn tự bảo vệ phần cứng trước các tình huống quá dòng, quá áp, sụt áp và hở tải, đệm dữ liệu đo vào bộ nhớ LittleFS khi mất kết nối rồi đồng bộ lại khi có mạng, đồng thời phát thông báo trạng thái ngoại tuyến để hệ thống nhận biết khi thiết bị mất kết nối.

Về phần giám sát và điều khiển, hệ thống cung cấp giao diện web dashboard cho phép người dùng theo dõi thông số điện theo thời gian thực và điều khiển đóng ngắt

từng ổ cắm từ xa, hỗ trợ nhiều người dùng và nhiều ngôi nhà với cơ chế phân quyền. Ngoài điều khiển trực tiếp, hệ thống còn cho phép hẹn giờ và lập lịch bật tắt tự động, cảnh báo sự cố điện qua email, thống kê điện năng tích lũy theo giờ, ngày và tháng cùng ước tính chi phí tiền điện. Người dùng vừa có thể điều khiển bằng nút nhấn vật lý, vừa có thể điều khiển qua mạng, hai phương thức được đồng bộ với nhau.

Kết quả kiểm thử cho thấy hệ thống đo điện áp với sai số rất nhỏ, chỉ khoảng 0,1% so với công tơ đối chứng, và đo công suất sát giá trị thực sau khi tính đến phần điện tự tiêu thụ của thiết bị, với sai số chỉ khoảng 0,5% ở tải lớn. Điện năng tích lũy hiển thị trên web cùng bậc với giá trị tính từ công suất đo được, với mức sai lệch khoảng 10%. Điều khiển từ xa cho độ trễ nhỏ, gần như tức thì khi mạng tốt và tăng lên vài giây khi mạng yếu, đồng thời cơ chế tự kết nối lại hoạt động tốt khi mất mạng. Các giá trị cụ thể được trình bày chi tiết ở Chương 5.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế. Module PZEM-004T hiện đo chung dòng và công suất của cả hai ổ cắm nên chưa tách riêng được mức tiêu thụ của từng ổ. Khối nguồn công suất nhỏ nên biên dự phòng không nhiều. Mạch chưa có cầu chì bảo vệ quá dòng bằng phần cứng mà mới dựa vào cơ chế bảo vệ bằng phần mềm. Vỏ hộp tự chế nên độ hoàn thiện cơ khí chưa cao. Ở các mức tải nhỏ chỉ vài watt, việc đối chứng độ chính xác còn hạn chế do ampe kìm dao động mạnh ở dòng thấp và công tơ chỉ hiển thị công suất theo bước một watt. Ngoài ra, do điện năng trên web được tính bằng cách tích lũy các mẫu công suất gửi lên, nên khi thiết bị mất kết nối kéo dài giá trị kWh vẫn có thể sai lệch, dù cơ chế đệm dữ liệu offline đã giảm đáng kể ảnh hưởng này.

6.2 HƯỚNG PHÁT TRIỂN

Từ những hạn chế nêu trên, đề tài có thể được tiếp tục hoàn thiện theo nhiều hướng. Trước hết, về mặt an toàn, nên bổ sung cầu chì hoặc aptomat thu nhỏ và linh kiện chống quá áp đột biến để có lớp bảo vệ bằng phần cứng độc lập với phần mềm, tăng độ tin cậy khi xảy ra sự cố.

Về khả năng đo lường, có thể bổ sung thêm module đo cho từng ổ cắm để giám sát riêng mức tiêu thụ của mỗi thiết bị, từ đó phân tích chi tiết thói quen sử dụng điện. Đồng thời nên chọn khối nguồn có công suất lớn hơn để tăng biên dự phòng, giúp hệ thống chạy ổn định hơn khi vi điều khiển và relay hoạt động đồng thời ở mức tải cao.

Về cơ khí, vỏ hộp có thể được thiết kế và in 3D theo kích thước mạch để sản phẩm gọn gàng, chắc chắn và có tính thẩm mỹ cao hơn, tiến gần tới một sản phẩm thương mại.

Về phần mềm và trải nghiệm người dùng, có thể phát triển thêm ứng dụng di động riêng cùng tính năng thông báo đẩy, giúp người dùng theo dõi và điều khiển thuận tiện hơn so với giao diện web, đồng thời bổ sung khả năng cập nhật firmware từ xa qua mạng để bảo trì thuận tiện hơn. Hệ thống cũng có thể tích hợp với các trợ lý ảo phổ biến để điều khiển bằng giọng nói.

Về xử lý dữ liệu, khi lượng dữ liệu tiêu thụ điện được tích lũy đủ lớn, có thể áp dụng các mô hình học máy để dự báo mức tiêu thụ, phát hiện bất thường và đưa ra khuyến nghị tiết kiệm điện cho người dùng. Đây là hướng phát triển có giá trị thực tiễn cao trong bối cảnh nhu cầu quản lý năng lượng ngày càng tăng.

Cuối cùng, về quy mô và bảo mật, hệ thống có thể được mở rộng để quản lý nhiều node trong cùng một ngôi nhà hoặc nhiều ngôi nhà, đồng thời nâng cấp cơ chế bảo mật bằng việc xác thực đầy đủ chứng chỉ của máy chủ và mã hóa thông tin nhạy cảm, thay cho cấu hình bỏ qua kiểm tra chứng chỉ hiện tại.

TÀI LIỆU THAM KHẢO

Tiếng Việt

- [1] Thông tấn xã Việt Nam (TTXVN), “*Tăng trưởng điện thương phẩm bình quân đạt 9,67% mỗi năm*”, VietnamPlus, 17/07/2020. [Trực tuyến]. Địa chỉ: <https://www.vietnamplus.vn/tang-truong-dien-thuong-pham-binh-quan-dat-967-moi-nam-post652267.vnp>
- [2] Bộ Công Thương, Cục Điều tiết điện lực, “*Kết quả kiểm tra chi phí sản xuất kinh doanh điện các năm 2020, 2021, 2022, 2023 của Tập đoàn Điện lực Việt Nam*”, Công thông tin điện tử Bộ Công Thương (moit.gov.vn), 2021–2024.
- [3] TAPIT Embedded and IoT, “*Thiết kế phần cứng thực nghiệm nhà thông minh sử dụng điện toán đám mây IoT*”, TAPIT. [Trực tuyến]. Địa chỉ: <https://tapit.vn/thiet-ke-phan-cung-thuc-nghiem-nha-thong-minh-su-dung-dien-toan-dam-may-iot/>
- [4] Tạ Anh Tú, “*Thiết kế ổ cắm điện thông minh điều khiển qua WiFi sử dụng ADE7753*”, Đồ án tốt nghiệp, Trường Đại học Bách khoa Hà Nội, Hà Nội, 3/2023.

Tiếng Anh

- [5] M. A. S. Md Dzahir and K. S. Chia, “*Evaluating the Energy Consumption of ESP32 Microcontroller for Real-Time MQTT IoT-Based Monitoring System*”, 2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), 2023, pp. 255–261.
- [6] H. J. El-Khozondar, và cộng sự,, “*A smart energy monitoring system using ESP32 microcontroller*”, e-Prime Advances in Electrical Engineering, Electronics and Energy, vol. 9, 100666, 2024.
- [7] S. L. Nalbalwar, D. P. Jayaswal, S. M. Fule, and U. D. Meshram, “*IoT-Based Smart Energy Meter with Real-Time Monitoring and Theft Detection using*

- Arduino and PZEM004T*”, International Journal of Engineering Research and Technology (IJERT), vol. 14, no. 12, 2025.
- [8] Espressif Systems, “*ESP32-WROOM-32 Datasheet*”, Espressif Systems, 2023.
- [9] Peacefair Electronic Technology, “*PZEM-004T V3.0 User Manual (AC Communication Module)*”, Peacefair, 2019.
- [10] OASIS, “*MQTT Version 5.0 — OASIS Standard*”, 2019.
- [11] I. Fette and A. Melnikov, “*The WebSocket Protocol*”, RFC 6455, Internet Engineering Task Force (IETF), 2011. Địa chỉ: <https://datatracker.ietf.org/doc/html/rfc6455>
- [12] Hi-Link Electronic, “*HLK-PM01 AC-DC Power Module*”, trang sản phẩm LCSC Electronics. [Trực tuyến]. Địa chỉ: <https://www.lcsc.com/product-detail/C209903.html>
- [13] Songle Relay, “*SRD-05VDC-SL-C Datasheet (SRD Series Relay)*”, Songle. [Trực tuyến]. Địa chỉ: <https://www.datasheetcafe.com/srd-05vdc-sl-c-datasheet-pdf/>
- [14] Nanjing Qinheng Microelectronics (WCH), “*CH340 USB to Serial Chip Datasheet (CH340DS1)*”, WCH. [Trực tuyến]. Địa chỉ: <https://cdn.sparkfun.com/datasheets/Dev/Arduino/Other/CH340DS1.PDF>
- [15] Advanced Monolithic Systems, “*AMS1117 1A Low Dropout Voltage Regulator Datasheet*”, AMS. [Trực tuyến]. Địa chỉ: <https://datasheet4u.com/part-number/AMS1117.html>
- [16] M. Jones, J. Bradley, and N. Sakimura, “*JSON Web Token (JWT)*”, RFC 7519, Internet Engineering Task Force (IETF), 2015. Địa chỉ: <https://datatracker.ietf.org/doc/html/rfc7519>
- [17] D. Hardt, “*The OAuth 2.0 Authorization Framework*”, RFC 6749, Internet Engineering Task Force (IETF), 2012. Địa chỉ: <https://datatracker.ietf.org/doc/html/rfc6749>

- [18] PostgreSQL Global Development Group, “*PostgreSQL Documentation: Transactions*”. Địa chỉ: <https://www.postgresql.org/docs/current/tutorial-transactions.html>
- [19] TypeORM, “*TypeORM Documentation*”. Địa chỉ: <https://typeorm.io/>
- [20] NestJS, “*NestJS Documentation*”. Địa chỉ: <https://docs.nestjs.com/>
- [21] Vercel, “*Next.js Documentation*”. Địa chỉ: <https://nextjs.org/docs>
- [22] Socket.IO, “*Socket.IO Documentation*”. Địa chỉ: <https://socket.io/docs/v4/>
- [23] littlefs project, “*littlefs: A little fail-safe filesystem designed for microcontrollers*”, GitHub. Địa chỉ: <https://github.com/littlefs-project/littlefs>
- [24] P. Sethi and S. R. Sarangi, “*Internet of Things: Architectures, Protocols, and Applications*”, Journal of Electrical and Computer Engineering, vol. 2017, Article ID 9324035, pp. 1–25, 2017.
- [25] B. Mishra and A. Kertész, “*The Use of MQTT in M2M and IoT Systems: A Survey*”, IEEE Access, vol. 8, pp. 201071–201086, 2020.
- [26] M. Alaa, A. A. Zaidan, B. B. Zaidan, M. Talal, and M. L. M. Kiah, “*A Review of Smart Home Applications Based on Internet of Things*”, Journal of Network and Computer Applications, vol. 97, pp. 48–65, 2017.
- [27] T. Yokotani and Y. Sasaki, “*Comparison with HTTP and MQTT on Required Network Resources for IoT*”, 2016 International Conference on Control, Electronics, Renewable Energy and Communications (ICCEREC), Bandung, Indonesia, 2016, pp. 1–6.
- [28] N. Naik, “*Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP*”, 2017 IEEE International Systems Engineering Symposium (ISSE), Vienna, Austria, 2017, pp. 1–7.
- [29] HiveMQ GmbH, “*MQTT Essentials — A Lightweight IoT Protocol*”, HiveMQ, 2023. [Trực tuyến]. Địa chỉ: <https://www.hivemq.com/mqtt-essentials/>